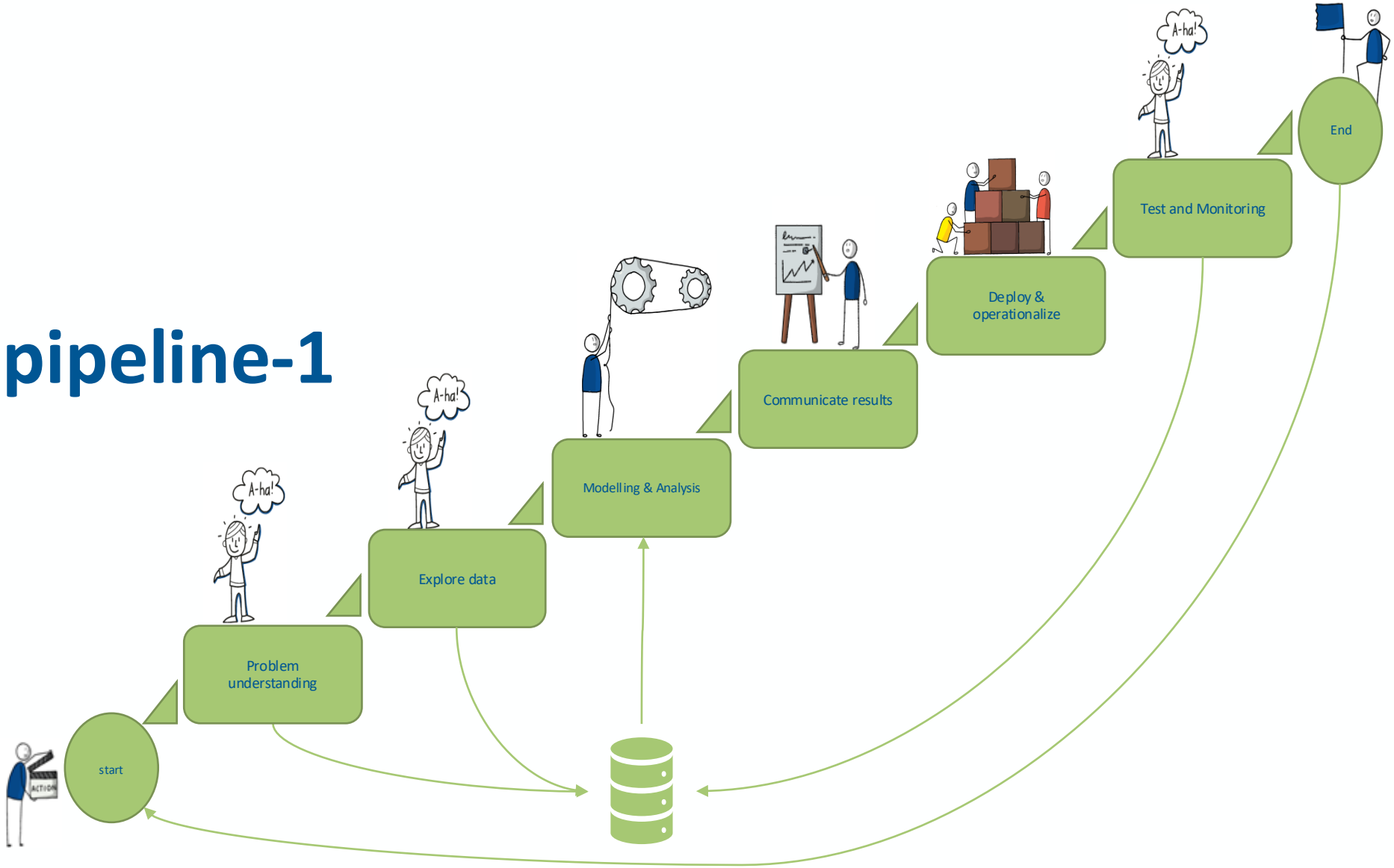
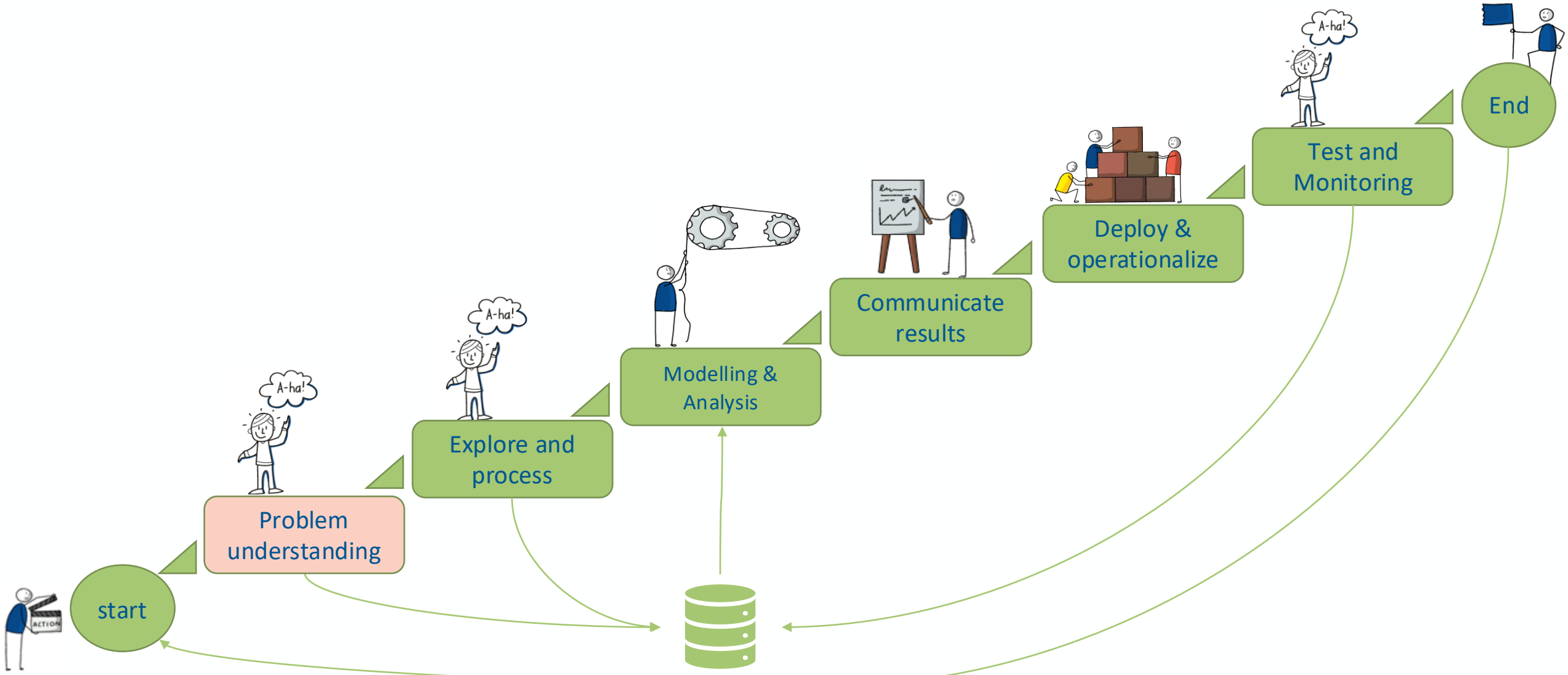
Data Mining pipeline-1

Prof. Dr. Matteo Marouf

21.04. 2026



Data Mining project life cycle may involve all or some of these stages



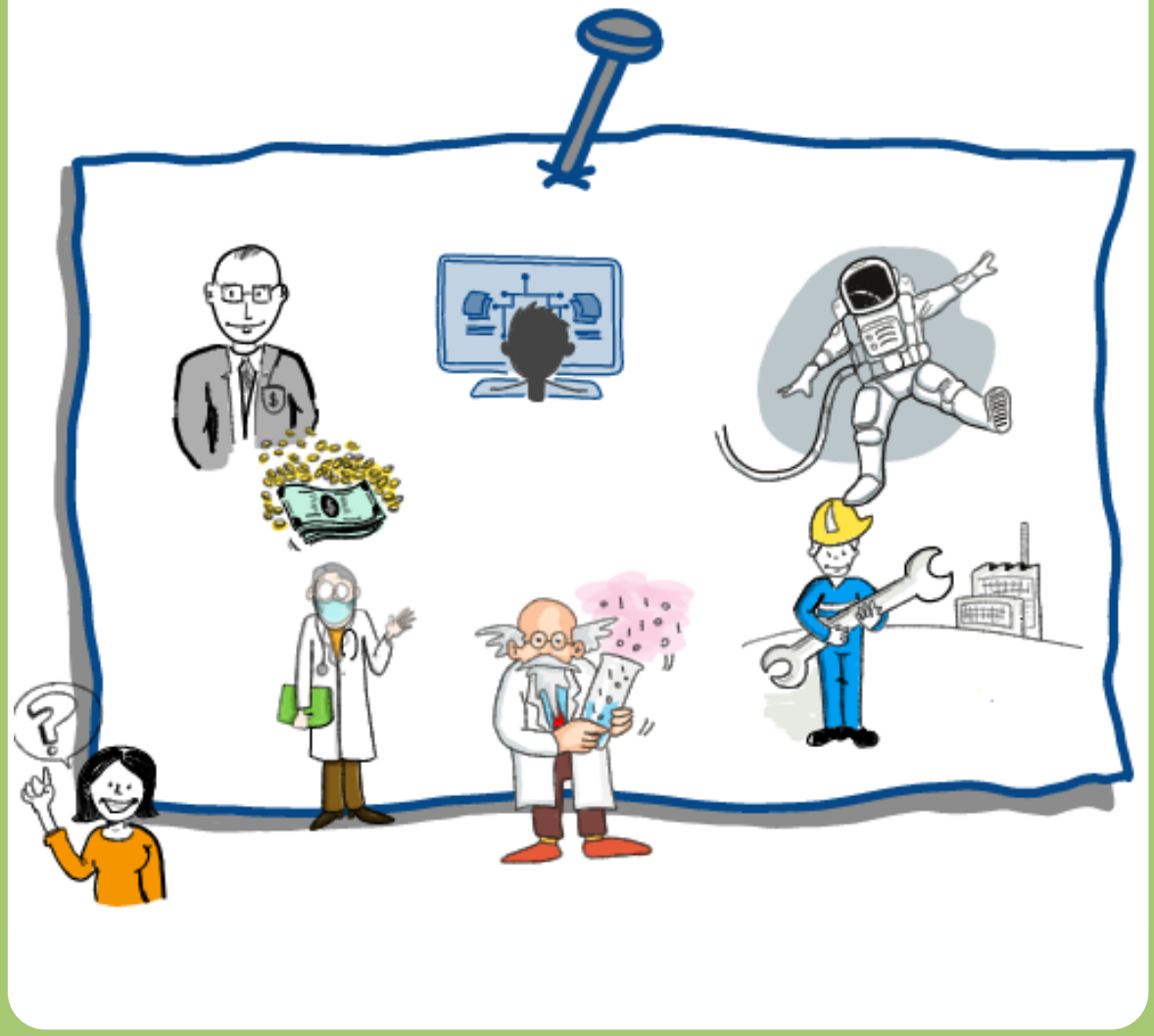
Domain knowledge plays a crucial role in successful data mining project

Domain Knowledge:

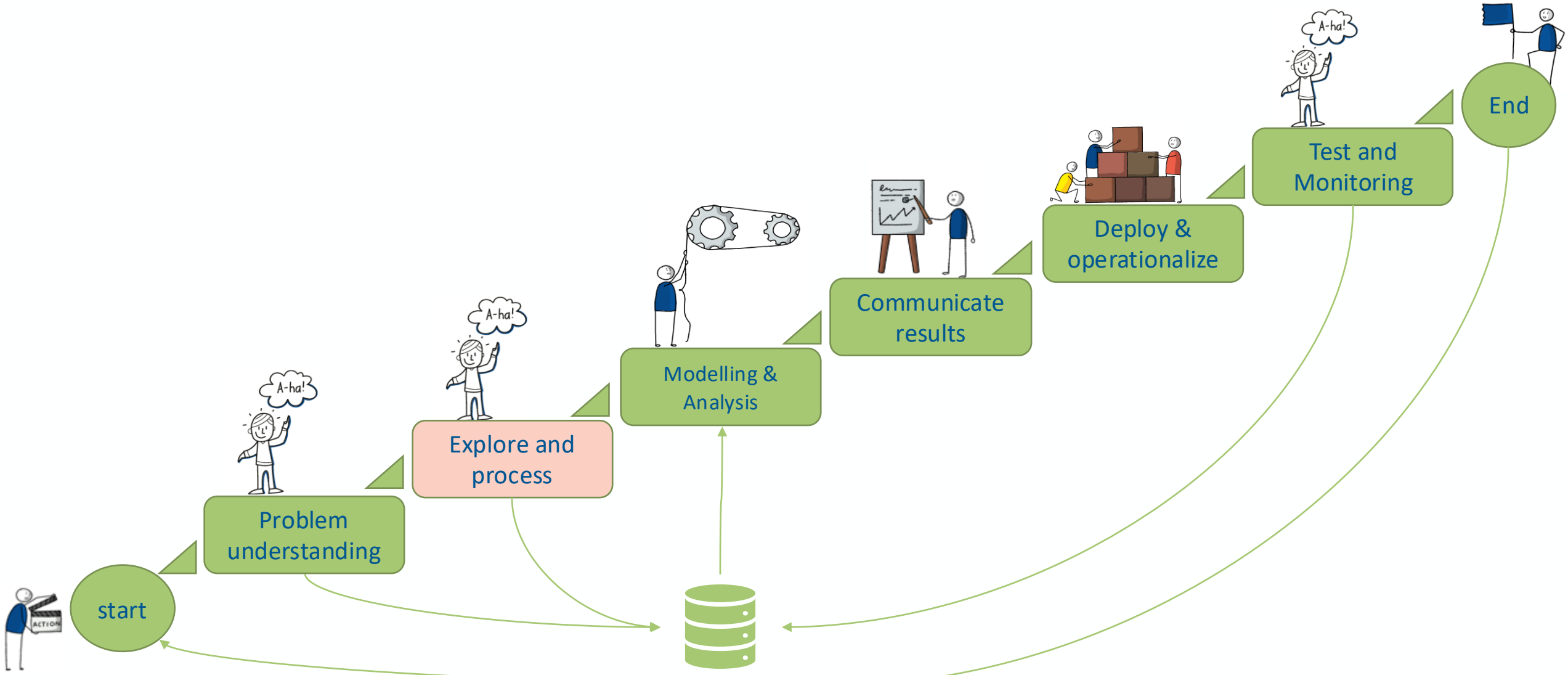
- guides the successful preprocessing
- Influnces the modeling
- Provides the the context to explain the results

The key takeaway

Always dive into the domain behind the data — it's the key to engineering meaningful features, focusing your analysis, and telling a story your audience understands.

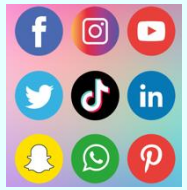


Data Mining project life cycle may involve all or some of these stages



Informative Data representation is a fundamental step in Data Mining

Data exist in different formats



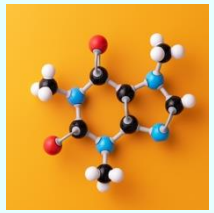
Social media

Imaging

EHR

Genomics

Knowledge graphs



Bioactive molecules

Pharmaceutical packaging machines mainly focus on large-volume products: Quantities of more than 100,000 units are standard. They are not working well for medicines that are produced and packaged only in small quantities, so-called microbatches. This is where conventional packaging machin are mostly inefficient due to the long setup and changeover times. A problem that production experts from Boehringer Ingelheim have been working on in recent years.

Textual data

Vital signals

Data Representation

To train a model, we must create an **informative and numerical representation of the data** that can be manipulated by electronic devices.

Examples

- **Textual data:** Word Embedding is a term used for the representation of words for text analysis, typically in real valued vectors.
- **Images**: Pixel values are used.
- **Molecular structure:** Could be represented as mathematical graph where each atom is a node, and each bond is an edge.

Quiz

Which statement best reflects how data is represented in data analysis?

A) Each dataset has only one possible representation

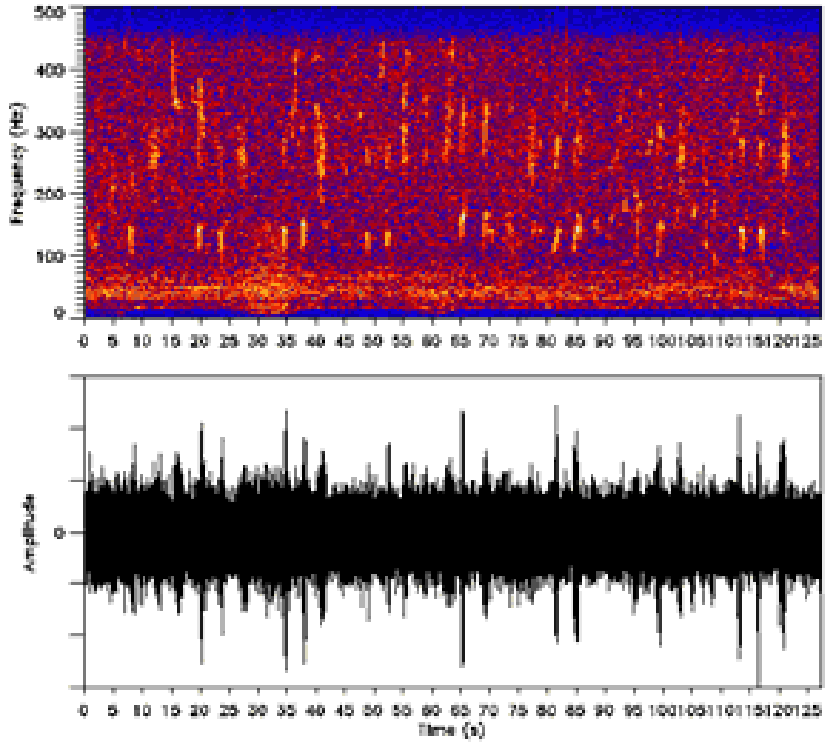
B) There could be different data representation ways depending on context and use

C) Changing representation always changes the underlying meaning

D) Data must be stored in only one format

Examples on finding a proper Data representation

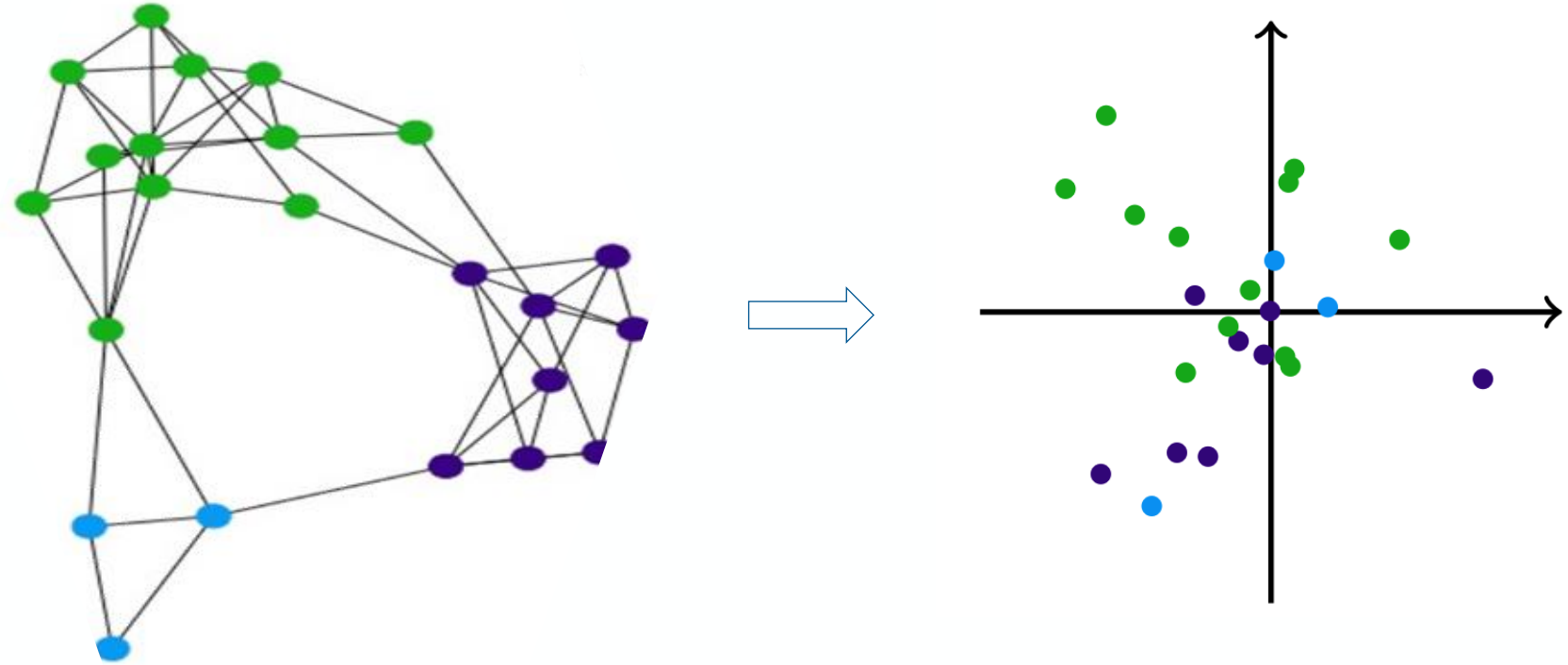
Spectrograms: An audio signal can be transformed into a spectrogram, which represents how its frequency content evolves over time. This representation is particularly useful for speech and music analysis. However, the same signal can also be analyzed directly as a one-dimensional numerical time series or in other domains such as using wavele



https://commons.wikimedia.org/w/index.php?curid=34672291

Examples on finding a proper Data representation

- **Node Embeddings (e.g., Node2Vec) for graph data :** Learn continuous feature representations for nodes in a graph, capturing the network's structural information.



Examples on finding a proper Data representation

- **Term Frequency-Inverse Document Frequency (TF-IDF)** converts textual information into numerical vectors, reflecting the importance of words in documents.

TF

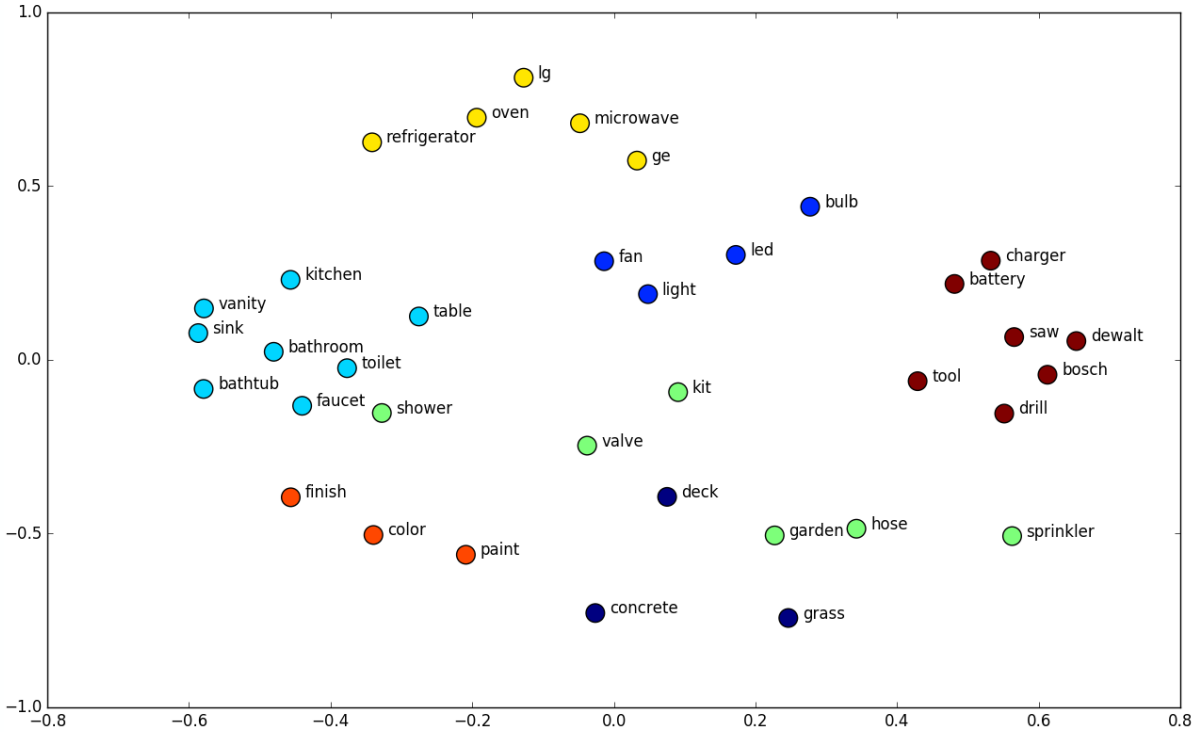
Frequency of a word withing a document

IDF

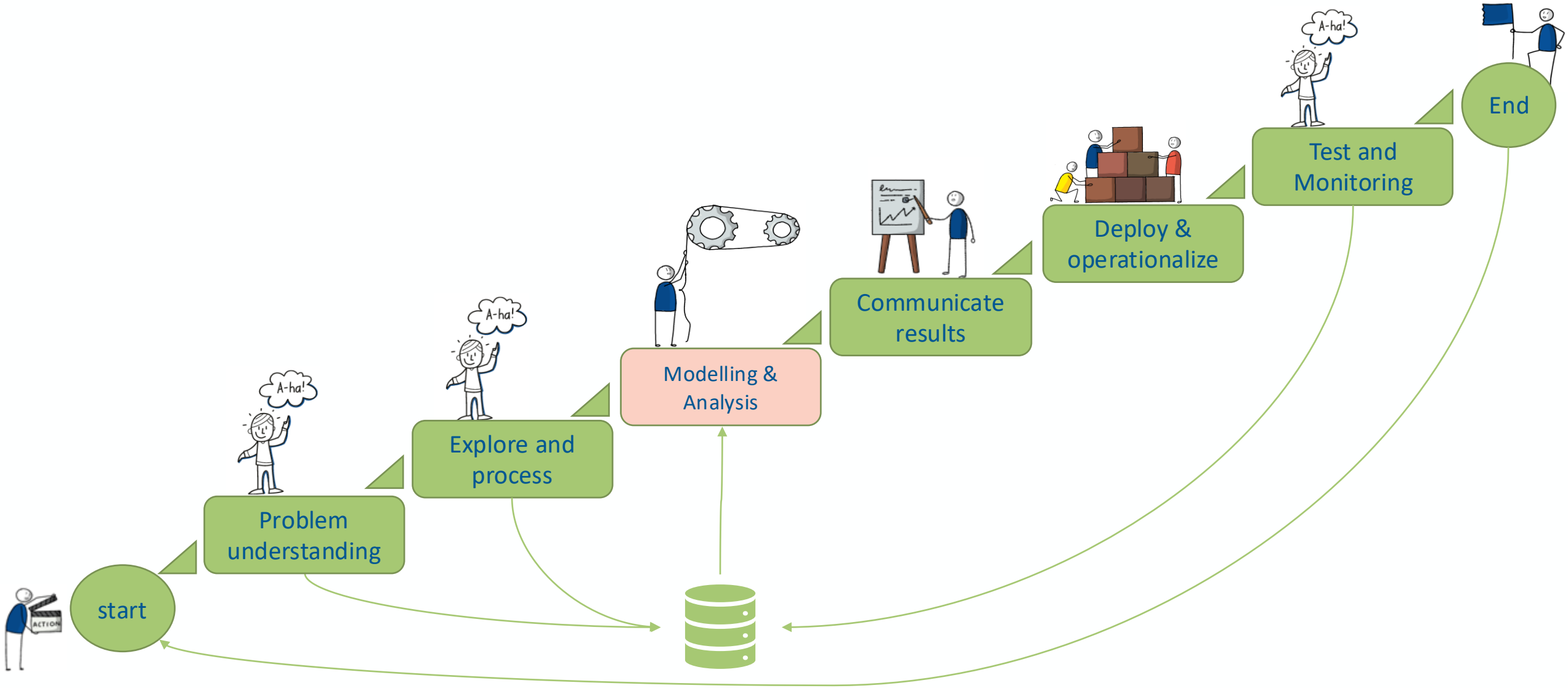
Frequency of a word across the corpus

Examples on finding a proper Data representation

- **Word Embeddings (e.g., Word2Vec, GloVe):** Word embeddings map words into continuous vector spaces where semantically similar words are positioned closely. These vectors capture contextual relationships and can be pre-trained on large corpora.



Data Science project life cycle

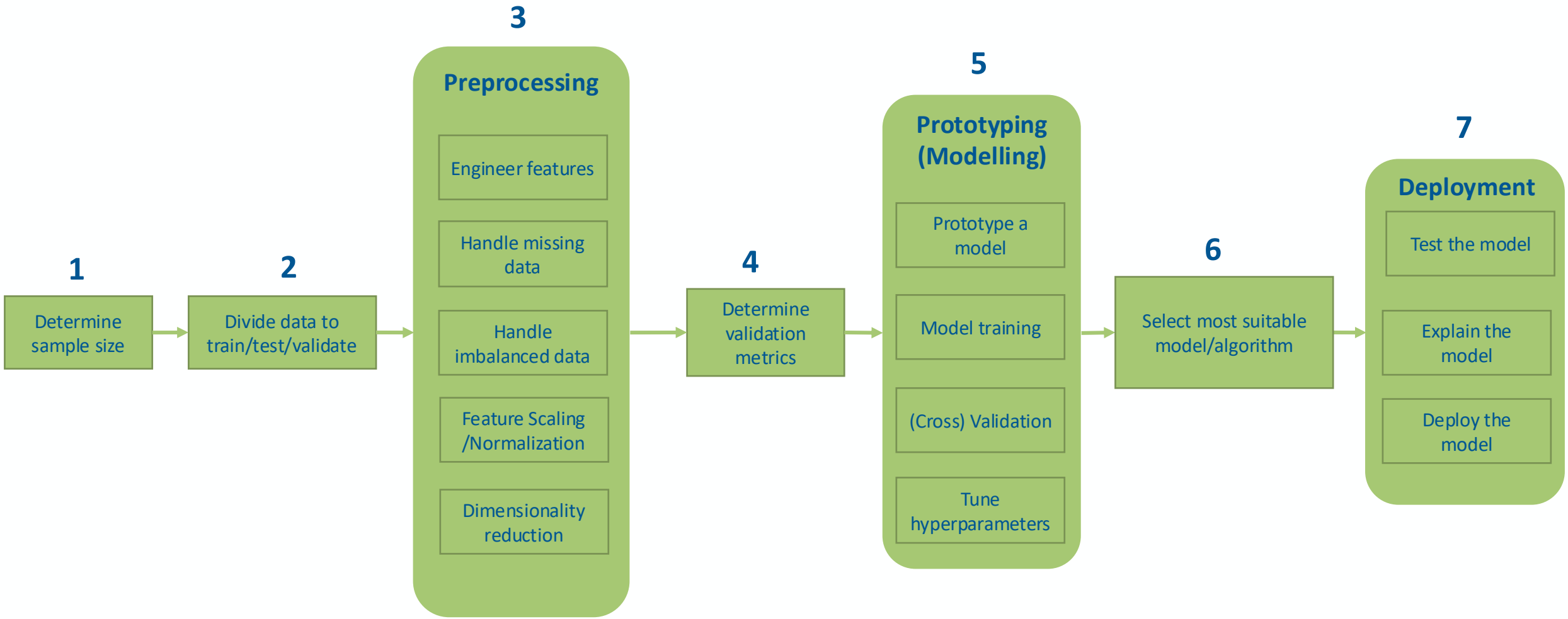


Main topics you may stumble across while working in this field

Classification:	Sorting data into predefined categories (e.g., spam detection).
Association Analysis:	identifying relationships between items in large datasets
Regression:	Predicting continuous values (e.g., stock prices, recovery time for hospitalized patients).
Clustering:	Grouping data based on similarity without predefined labels (e.g., customer segmentation).
Optimization:	AI often finds the best solutions to problems within constraints (e.g., route planning, resource allocation).
Recommendation:	Suggesting items or content based on user preferences (e.g., Netflix or Amazon recommendation engines).
Anomaly Detection:	Identifying unusual patterns or outliers in data (e.g., fraud detection, fault prediction).
Personalization:	Tailoring user experiences or interactions (e.g., personalized marketing, adaptive learning systems).
Decision Support:	Inform decision-making by analyzing large datasets and providing insights (e.g., medical diagnostics, business intelligence).
Simulation:	model complex systems or environments to predict behavior and outcomes (e.g., weather modeling, financial market).
Text mining	Understanding, mining, and generating human language (e.g., sentiment analysis, language translation).

Which steps are usually involved a Data Mining Analysis?

Example: The anatomy of classification using ML most steps are the shared with other analysis types



Sample size discussion

Statistical Power & Accuracy	A larger sample size increases the **statistical power** of models, reducing the risk of **Type II errors** (failing to detect a real effect).
	It enhances **estimation accuracy**, ensuring that patterns and relationships found in the sample reflect the true population.
Generalizability	A small sample may not adequately represent the **population**, leading to biased results.
	A sufficiently large and **diverse** sample helps models generalize well to new, unseen data.
Reducing Variability & Overfitting	Small samples often lead to **high variance**, where minor changes in data can lead to drastically different model outcomes.
	Larger samples help models learn **underlying patterns** rather than memorizing noise, reducing the risk of **overfitting**.
Choice of Algorithms	Some algorithms, like **deep learning**, require **large datasets** to perform well, while others (e.g., decision trees, k-NN) can work reasonably with smaller samples.
	Small samples may force the use of **simpler models** that might not capture complex relationships.
Handling Class Imbalance	In classification problems (e.g., fraud detection, medical diagnosis), a small sample may result in a **class imbalance**, making the model biased toward the majority class.
	A larger dataset allows better handling of rare events and minority class distributions.
Computational Cost vs. Performance Trade-Off	While larger samples improve model reliability, they also increase **computational costs** in terms of processing time and storage.
	In some cases, **sampling techniques** (e.g., stratified sampling, bootstrapping) are used to balance efficiency and accuracy.

Quiz

Which statement is TRUE?

A) Larger datasets always produce more accurate conclusions

B) Increasing sample size eliminates bias

C) Large datasets can make insignificant effects appear important

D) Small datasets are always unreliable

Essential Questions to Guide Your Data Mining Analysis

Ensuring accuracy, generalizability, and computational feasibility from the start.

Determine sample size

Do you have too much data?

- Consider sampling technique to select a representative sample.
- Consider using big data frameworks to facilitate data analysis.
- Consider using learning curves to find the sufficient sample size.

.....

Do you have too little data?

- Consider collecting more data.
- Consider using data augmentation methods.

....

Have you not collected data yet?

- Consider collecting data and evaluate whether it is enough.
- Resort to domain expert when collecting data.

....

Conclusion

Larger sample sizes generally improve model performance, but the optimal size depends on the problem's complexity, algorithm, data noise, and computational resources.

Note on Big Data: Is Bigger Data Always Better?

- Large samples reduce **random noise** and improve detection of real effects
- But they **do not fix bias or poor data quality**
- They can make **tiny, irrelevant effects appear significant**
- More data can lead to **high confidence in wrong conclusion**

Bottomline: In addition to sample size, data scientist should consider Data quality and representativeness

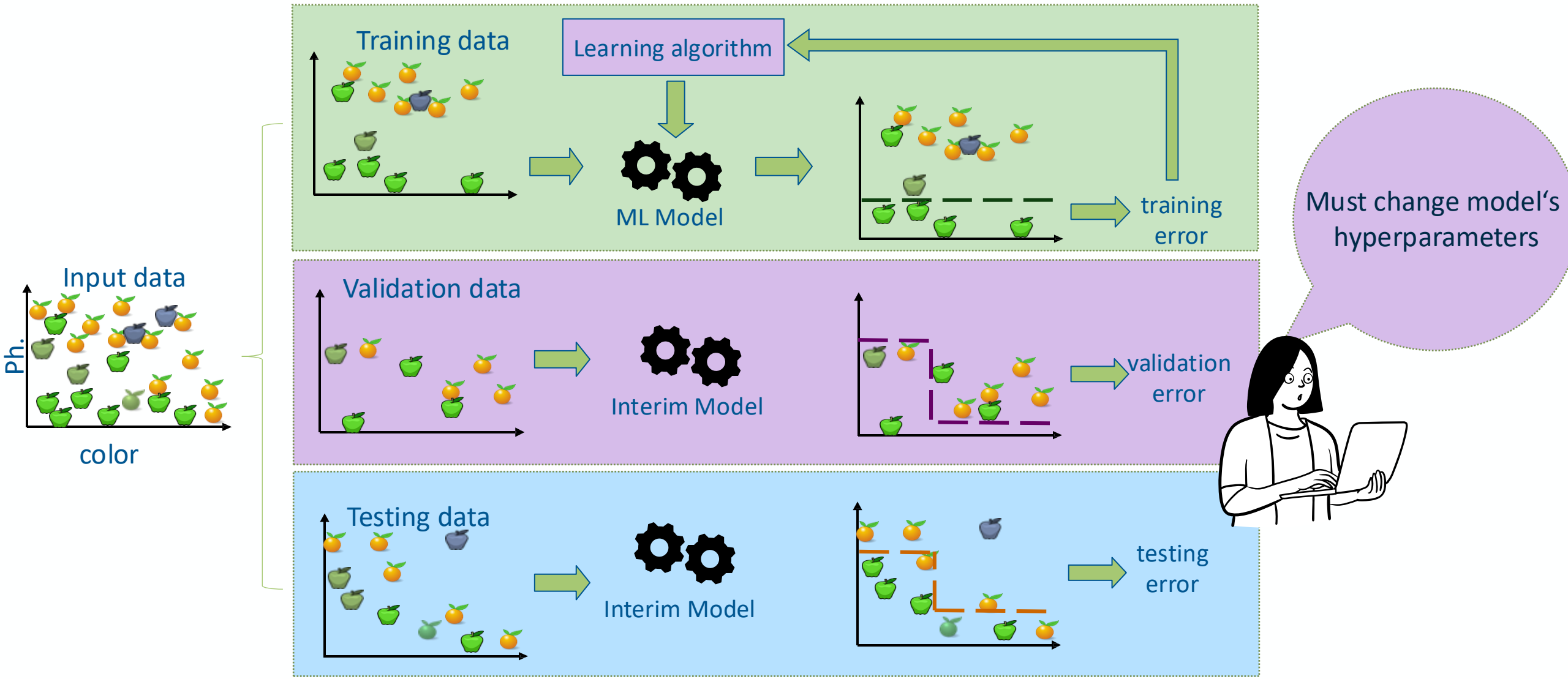
What about Large Language Models?

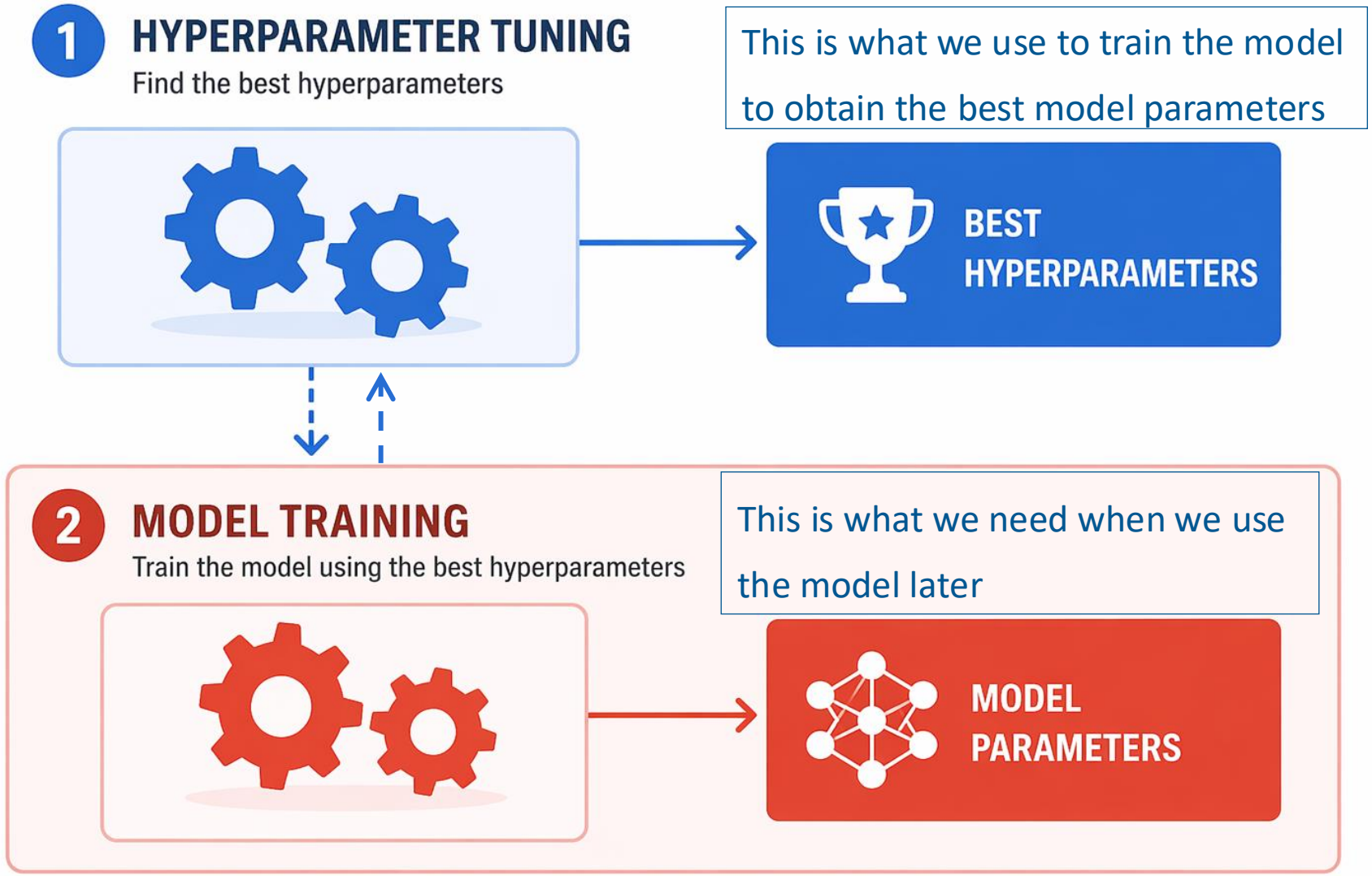
Split to train, test and validate

Determine sample size → Divide data set to train/test/validate →

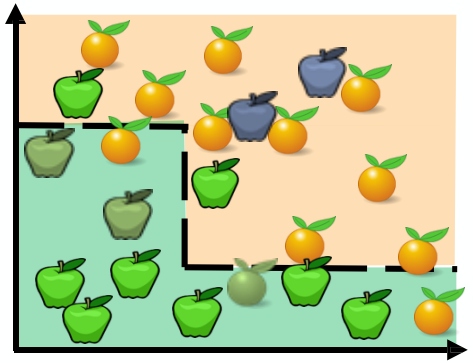
Why?

Example on splitting to Training, Validating, Testing datasets

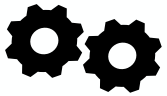



1
HYPERPARAMETER TUNING
Find the best hyperparameters
This is what we use to train the model to obtain the best model parameters
BEST HYPERPARAMETERS
2
MODEL TRAINING
Train the model using the best hyperparameters
This is what we need when we use the model later
MODEL PARAMETERS

Anatomy of Machine Learning: inference



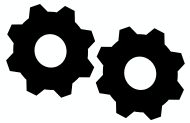
The model is now working sufficiently good for training, validating, and testing data



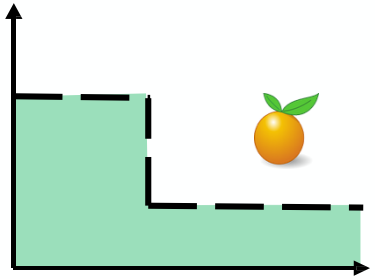
Final Model

INFERENCE

New data point



Final Model



Split to train, test and validate

Determine sample size → Divide data set to train/test/validate →

Maximize generalization

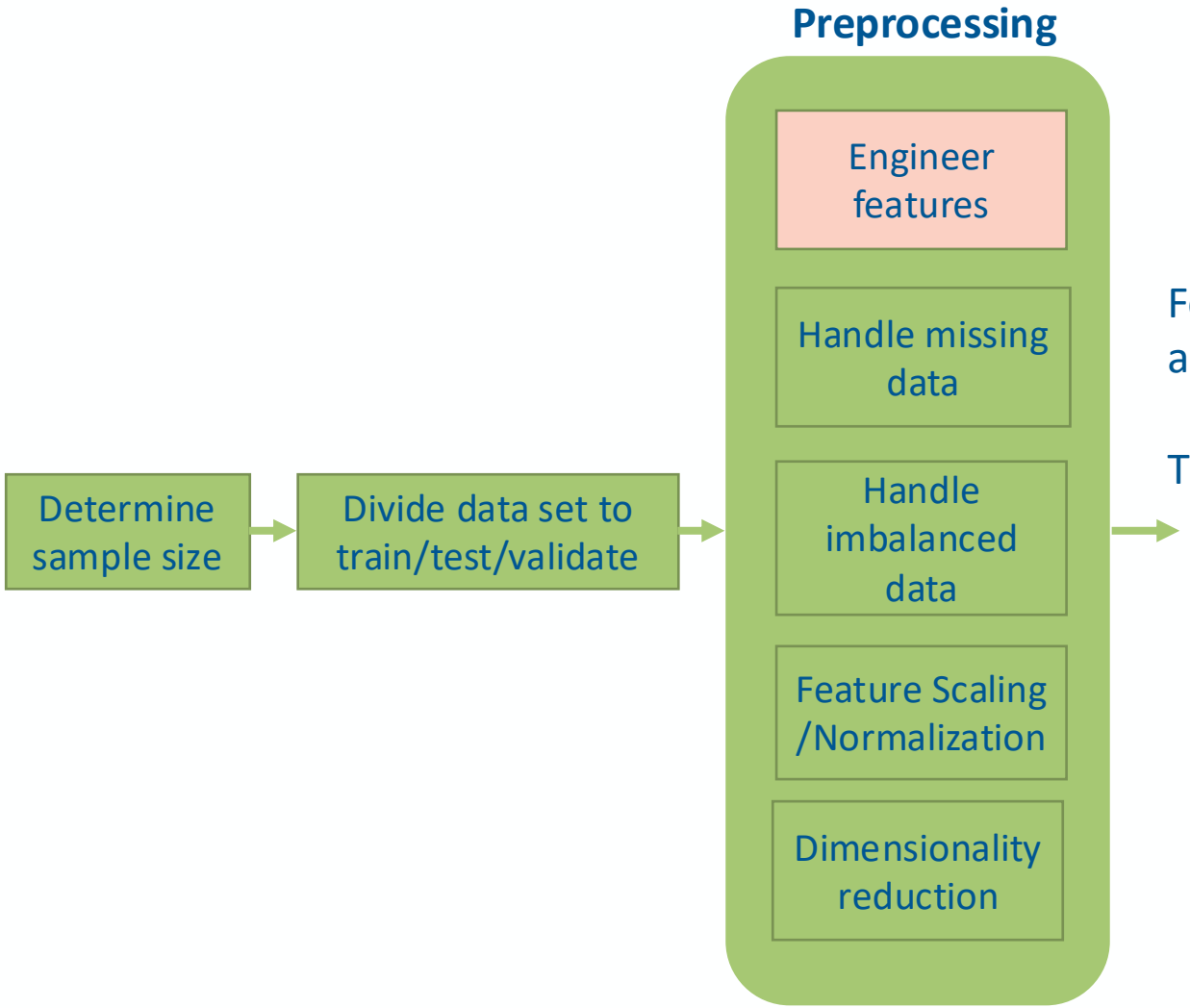
Avoid underfitting

Avoid overfitting

What we build on train data **is good for the train data** but also **good for other unseen data.**

Time for Analogy

Preprocessing: Features engineering



Feature engineering is **critical** for improving modelling accuracy and generating useful insights.

The **choice of technique depends on**:

- The **type of data** (numerical, categorical, text, image).
- The **problem** (classification, regression, clustering).
- The **model** being used (linear models, tree-based models, neural networks).

Features Transformation: Engineering new features

Feature Creation creating new features from existing ones to capture additional patterns in data, e.g. extract speed using time and distance to study rate of movement .

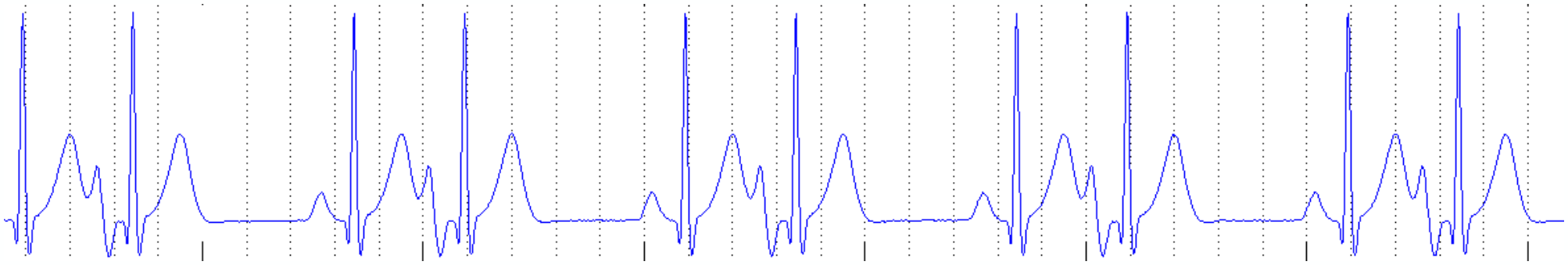
Combining– use an interaction term (e.g., x2,xyx^2, xyx2,xy) e.g. when studying income number of working hours plus income could be converted into income per hour.

Aggregations – Computing statistics (mean, sum, count) over groups.

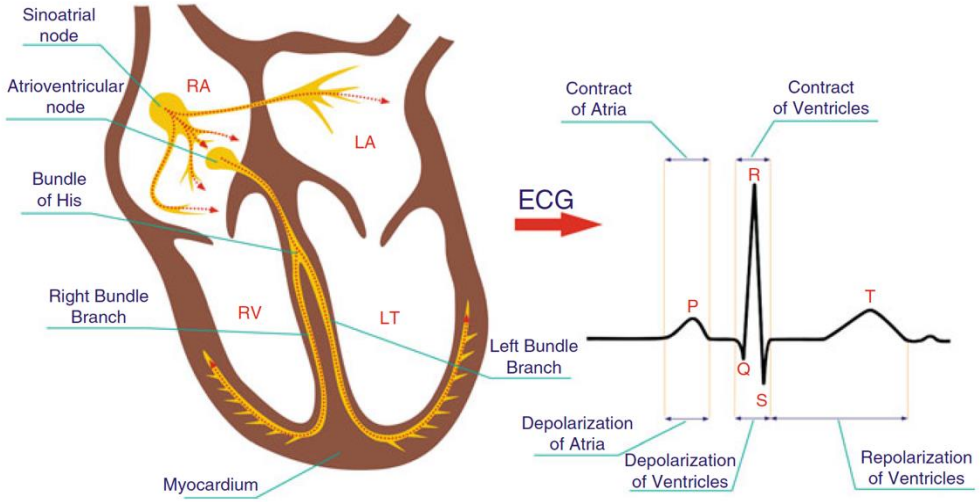
Convert Datetime Features – Extracting hour, day, month, or season from timestamps.

Extract Text Features – Extracting word counts, sentiment, or embeddings from text.

Example: Feature engineering in ECG signals



Successful feature engineering leverages domain knowledge to transform meaning features from raw data.



Example: in ECG signals, features (covariates) are derived by processing the signal to find the different peaks and valleys and compute the main intervals that reflect heart activity.

Feature Encoding

Most algorithms cannot work directly with categorical variables, so we need to convert them into a numerical format

ID	Production Year	Speed	Horsepower	Car Brand	price
0	2018	220	75	Toyota	….
1	2015	240	90	Ford	…
2	2019	190	95	Tesla	…

Example: car band in a data set we want to use to model and predict used-car price

Feature Encoding

Most algorithms cannot work directly with categorical variables, so we need to convert them into a numerical format.

1

Toyota	Ford	Tesla
1	0	0
0	1	0
0	0	1

2

ID	Production Year	Speed	Horsepower	Toyota	Ford	Tesla	price
0	2018	220	75	1	0	0	….
1	2015	240	90	0	1	0	…
2	2019	190	95	0	0	1	…

We first create a code for each category

Replace the categories so that models can process the categorical information numerically.

1. Why don't we just replace brand with different numbers?

a. because numbers can mislead the model
b. Indeed, we can replace them with numbers
c. That categories are unrelated and numbers are
d. It makes training faster

Feature Encoding

Most algorithms cannot work directly with categorical variables, so we need to convert them into a numerical format.

1

Toyota	Ford	Tesla
1	0	0
0	1	0
0	0	1

2

ID	Production Year	Speed	Horsepower	Toyota	Ford	Tesla	price
0	2018	220	75	1	0	0	….
1	2015	240	90	0	1	0	…
2	2019	190	95	0	0	1	…

We first create a code for each category

Replace the categories so that models can process the categorical information numerically.

1. Do we really need all new columns?

A. Yes, always, this reduces overfitting

B. No, one column is redundant

C. Only if using neural networks

D. Removing columns cause non-linearities

Answers

1. **Label encoding introduces an artificial ordinal relationship.**
   - The model thinks that Blue (2) > Green (1) > Red (0)
   - It assumes **order** and **distance**, like “Green is closer to Blue than Red”

   - This can **mislead** models that use math on input values (e.g. linear regression, logistic regression, k-NN)

2. **There is a redundant column:** Keeping the original column can lead to multicollinearity, where redundant information can affect the model's performance (it can assume some weird non-existing relation).

Feature Encoding

dropping one column after encoding prior to modelling

1

Toyota	Ford	Tesla
1	0	0
0	1	0
0	0	1

2

ID	Production Year	Speed	Horsepower	Toyota	Ford	Tesla	price
0	2018	220	75	1	0	0	….
1	2015	240	90	0	1	0	…
2	2019	190	95	0	0	1	…

We first create a code for each category

Replace the categories so that models can process the categorical information numerically.

3

ID	Production Year	Speed	Horsepower	Toyota	Ford	price
0	2018	220	75	1	0	….
1	2015	240	90	0	1	…
2	2019	190	95	0	0	…

Drop the redundant column: Keeping the original column can lead to multicollinearity, where redundant information can affect the model's performance (it can assume some weird non-existing relation).

Features Binning of numerical subranges

Binning (also called bucketing) is a feature engineering technique that groups different numerical subranges into bins or buckets. In many cases, binning turns numerical data into categorical data.

Example: consider a feature named Age with lowest value is 0 and highest value is 130. Using binning, you could represent it with the following five bins:

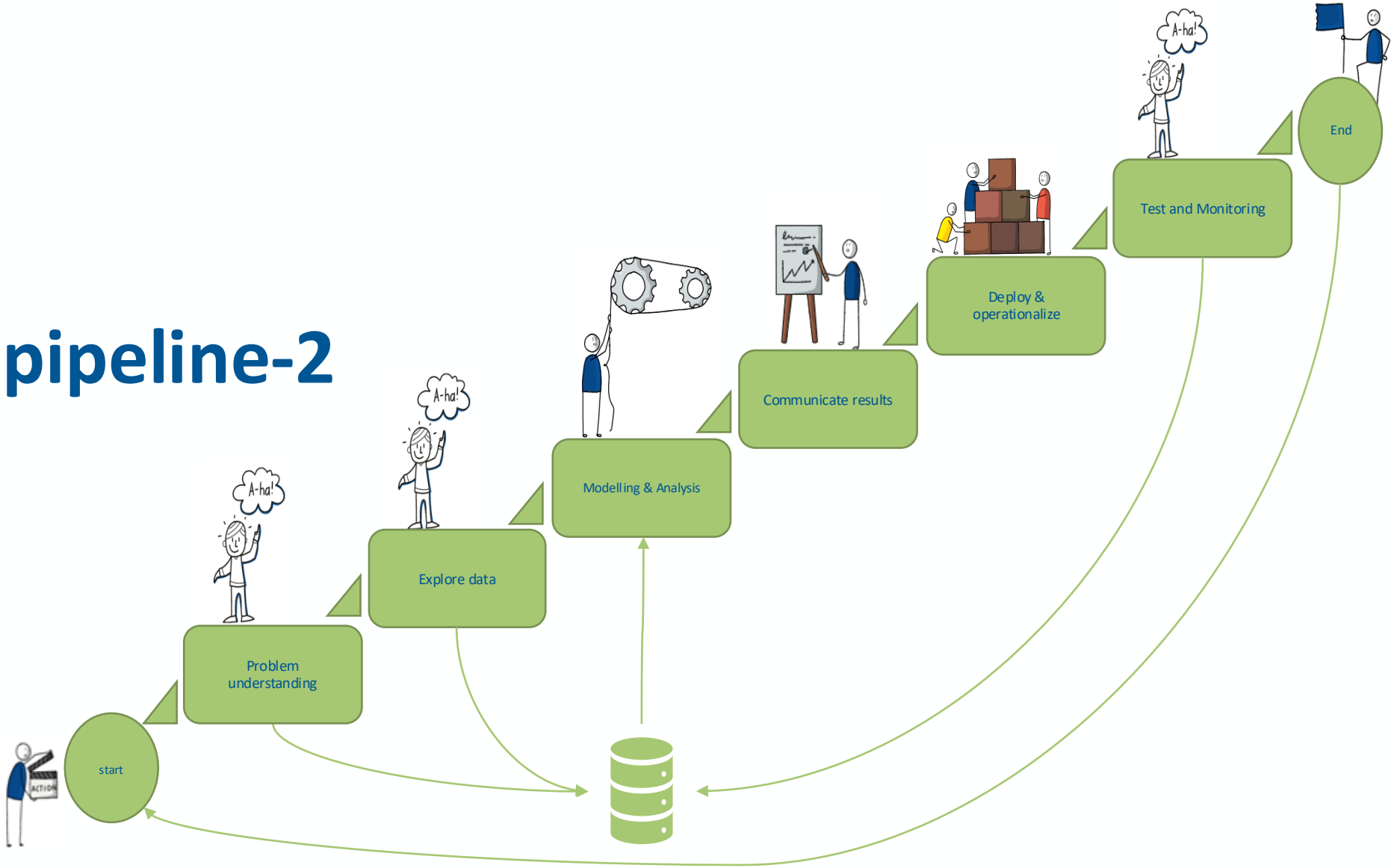
- Bin 1:infant 0 to 1
- Bin 2: toddler 1 to 3
- Bin 3: pre-schooler. 3 to 6
- Bin 4: Grade-schooler 5 to 12
- Bin 5: Teen 12 to 18
- Bin 6: Young Adult 18-21
- …

A model trained on these bins will react differently to X values of 1 and 1.5 since both values are in two different Bins and similarly to 6 and 10 since both are in Bin 4.

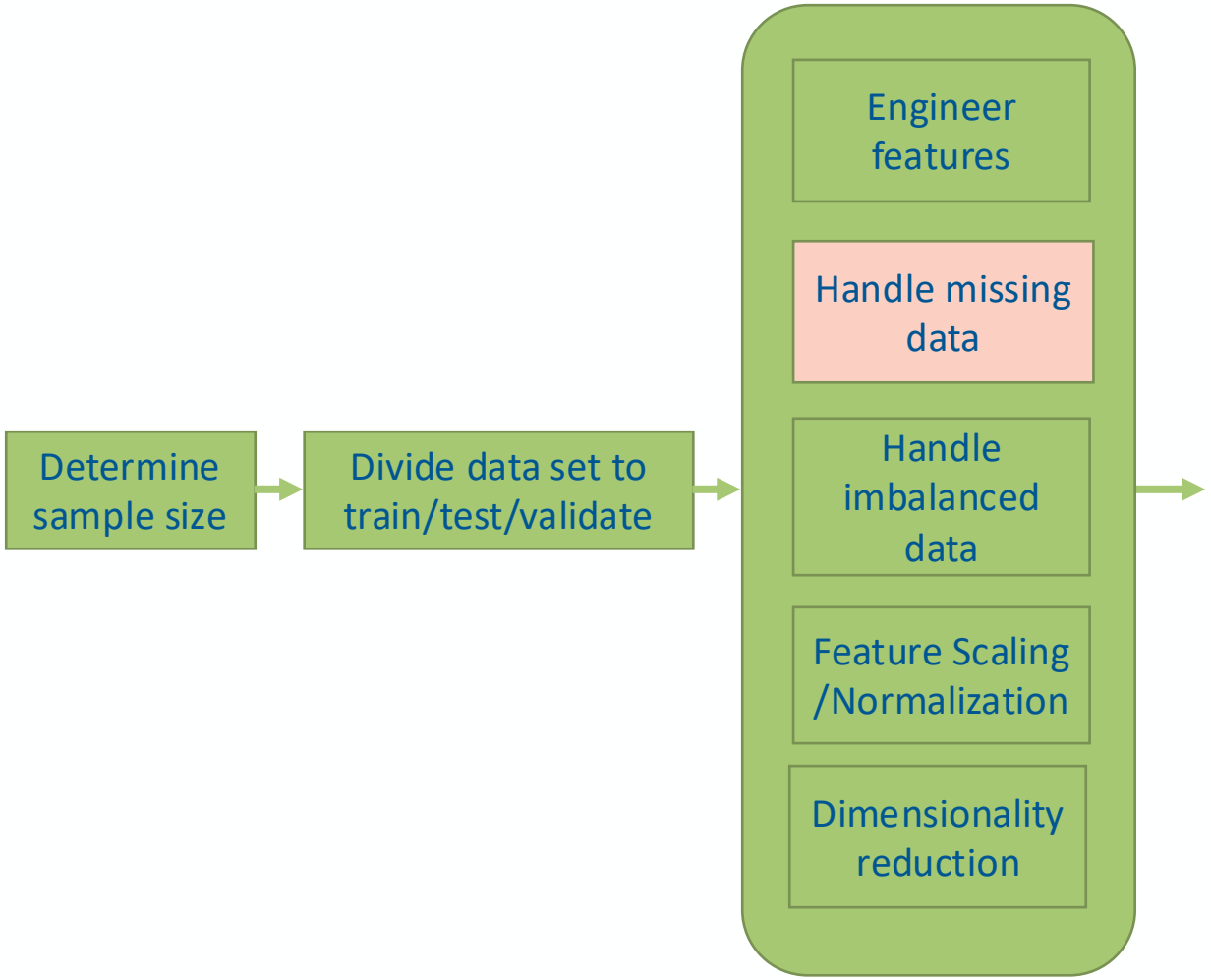
Data Mining pipeline-2

Prof. Dr. Matteo Marouf

28.04. 2026



Preprocessing: strategies to handle missing data



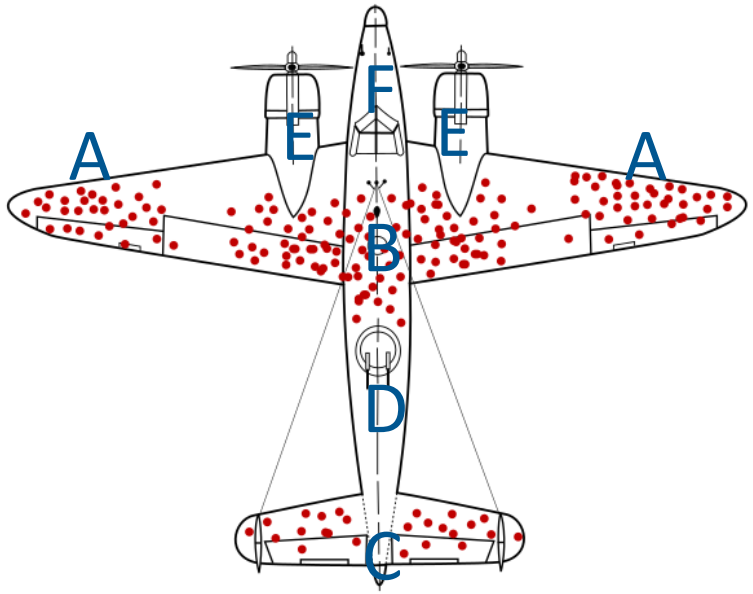
- Define the pattern of missing data.
- Process the missing data (dropping, replacement, imputation…etc.)
- Consider using algorithm which is robust to missing data.

…

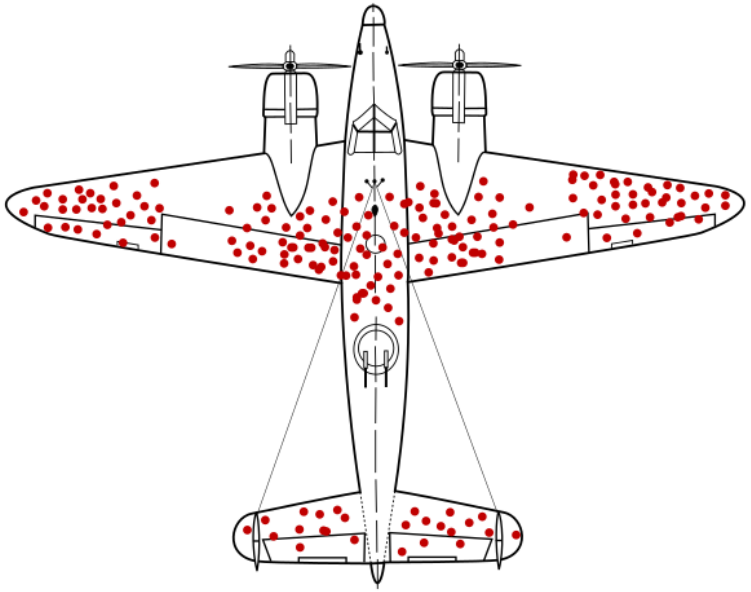
Quiz on missingness

During World War II, engineers analyzed returning airplanes and mapped where they were hit by bullets to suggest proper reinforcement. Which areas do need reinforcement?

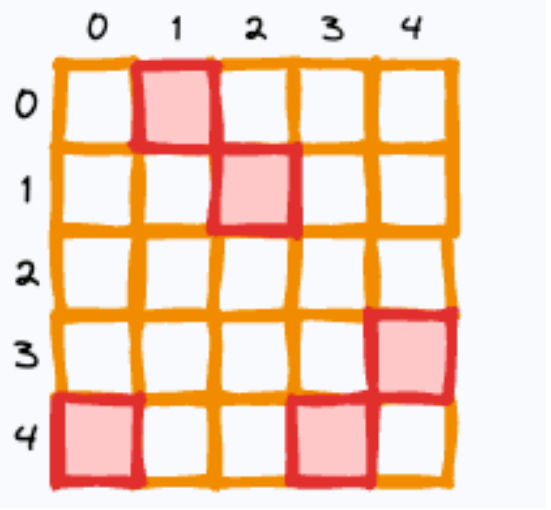
- A&B
- C
- D,E & F
- A,B & E
- D
- F



Preprocessing:
missing data can manifest as missing categories or missing values



survivorship bias
missing data points or categories



Missing values

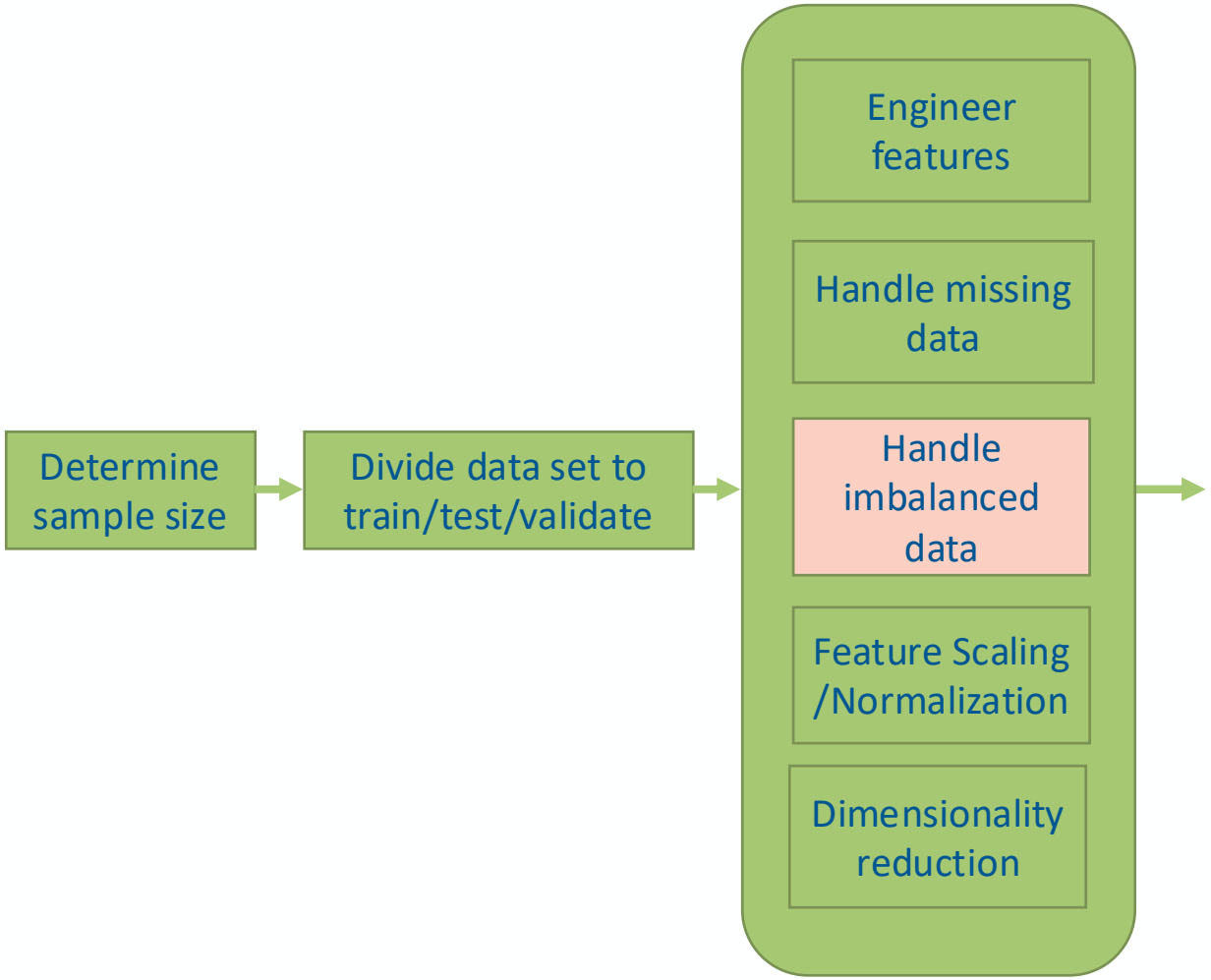
Strategies for handling missing values

- Ignore records with missing data
- Include NA indicator variable that become a part of the overall modelling.
- Do nothing and use method which can use data record with missing data
- **Impute (fill in) missing values (= guess!)**

Strategies for Imputing missing values

- **Manual Imputation**
  Expert or domain knowledge is used to fill missing values.
- **Global Constant**
  Replace with a constant like "unknown" or -999.
- **Statistical Imputation**
  - **Mean/Median of Variable**
    Replace missing values with the **mean or median** of that feature.
  - **Class-wise Mean/Median**
    If labels are available, use the **mean/median within each class**.
- **Model-based Imputation**
  Use other variables to predict missing values **using regression model**, **decision trees**, or other models.

Preprocessing: **strategies to handle imbalanced data**



- **Imbalanced data:** Data set in which some cases or risk categories occur much less frequently than the others.

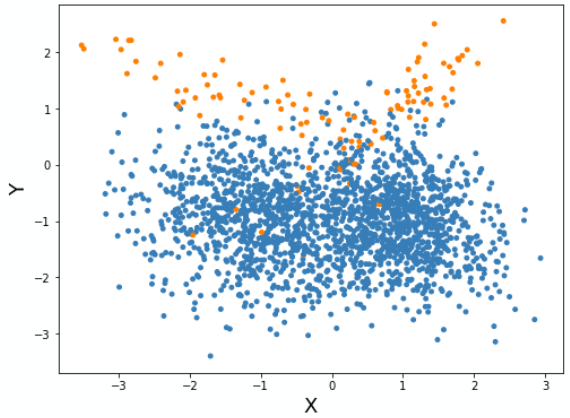
What can we do?

1. Use a model or an objective functions suited to handling imbalanced data
2. Use data resampling techniques
3. Use data augmentation to augment the rare classes
4. Use balanced to suited evaluation metrics

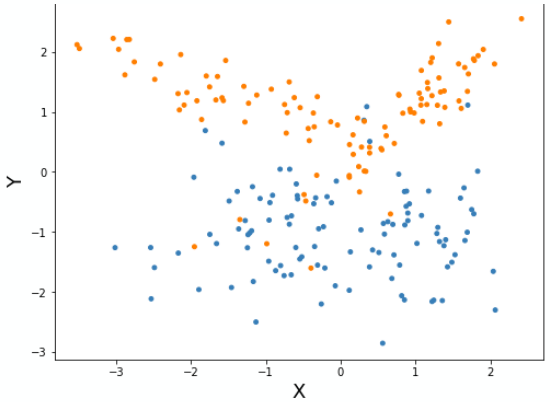
Preprocessing: **strategies to handle imbalanced data**

Determine sample size → Divide data set to train/test/validate → Engineer features / Handle missing data / Handle imbalanced data / Feature Scaling /Normalization / Dimensionality reduction

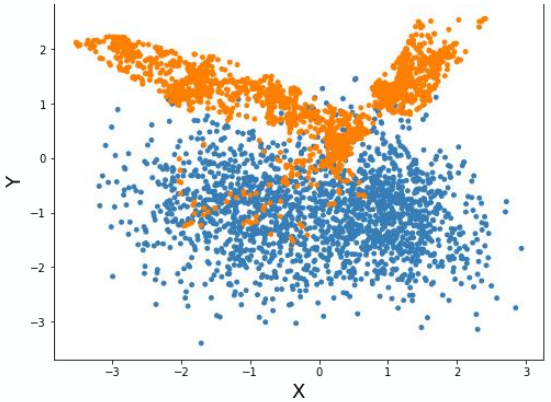
Imbalanced Dataset



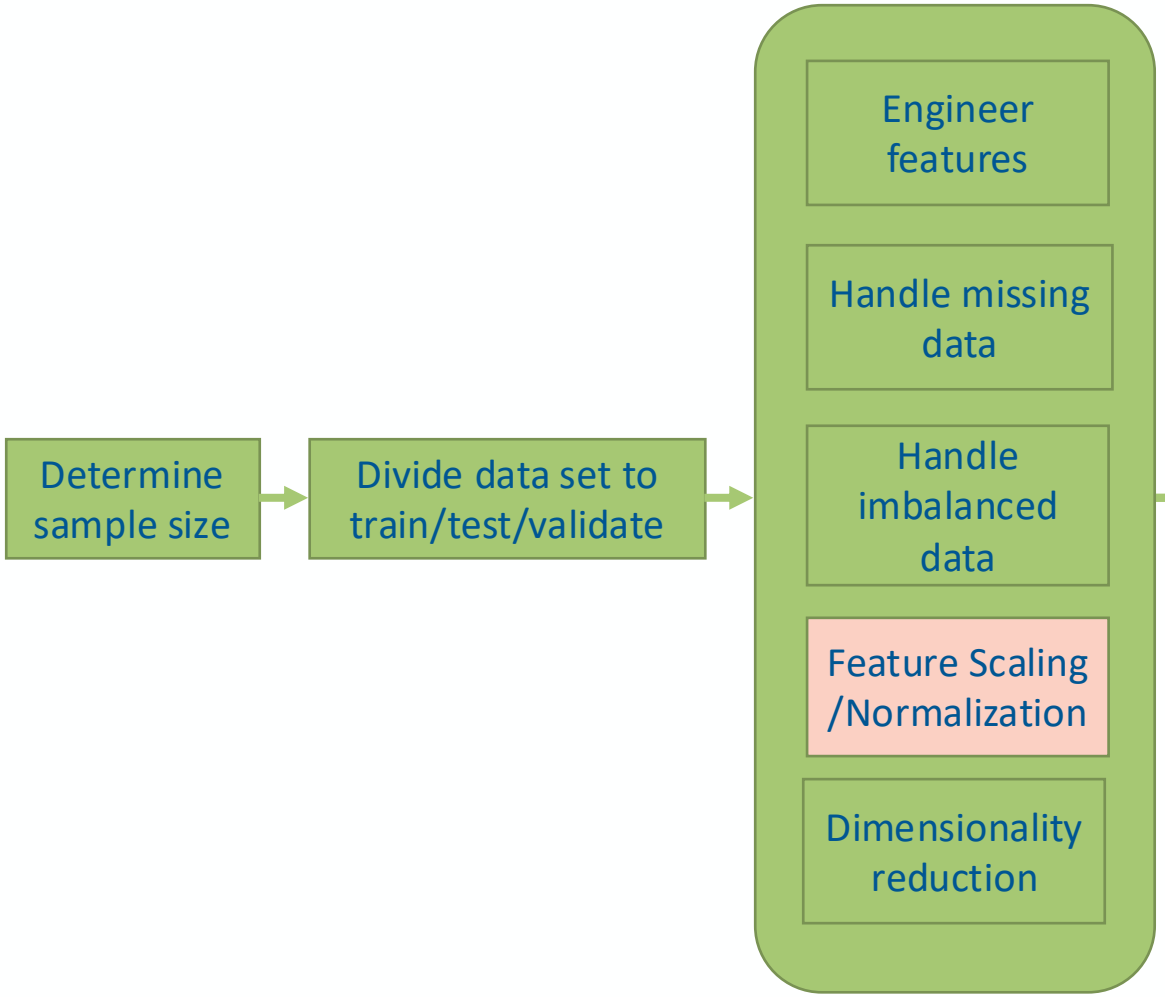
Downsample to balance



Upsample or augment



Preprocessing: Feature Scaling and Normalization



📌 **Why features scaling Matters**

🚀 **Improves model performance** by ensuring that features **contribute equally**.

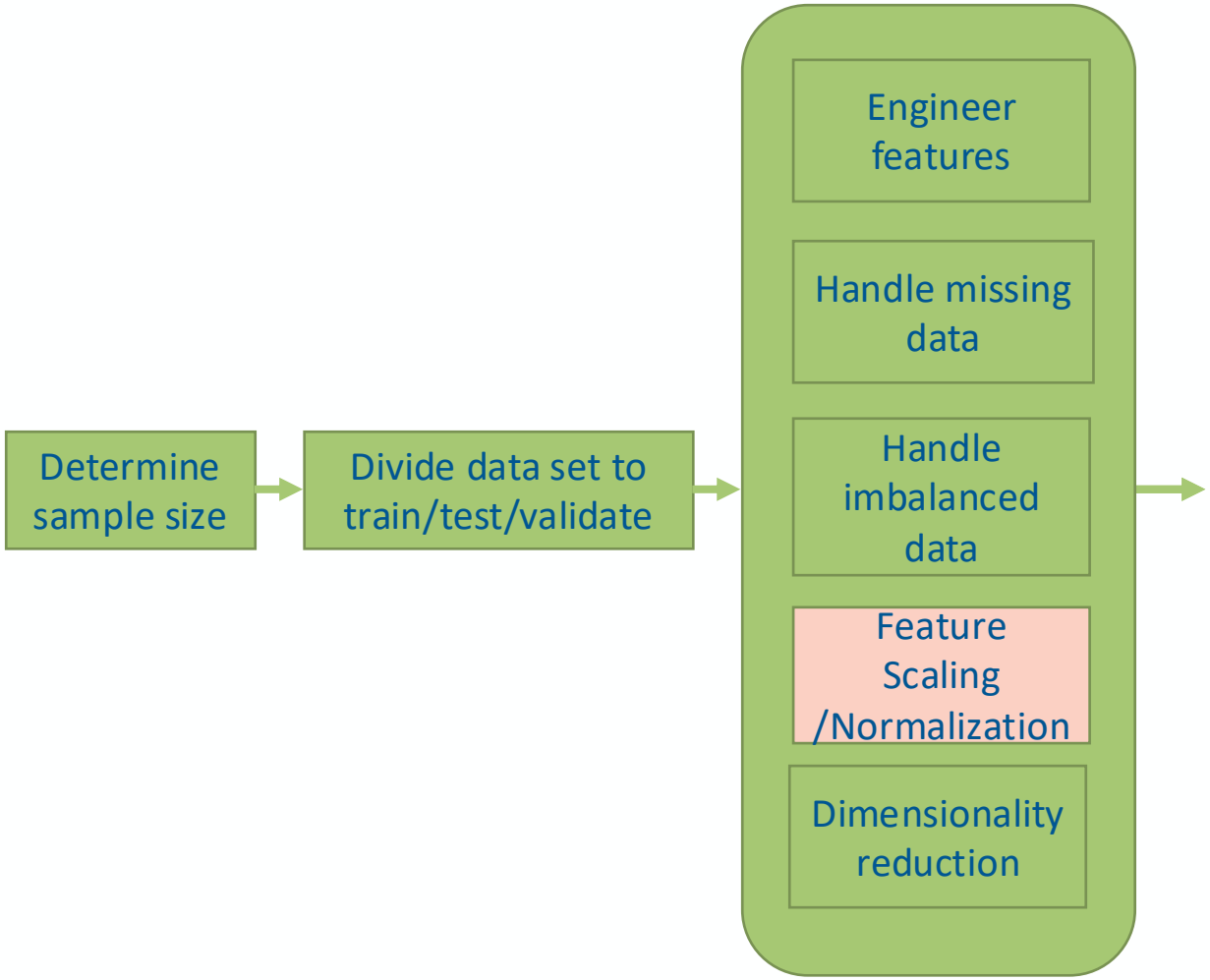
⚖️ Prevents features with larger scales from **dominating others**.

📏 Required for **distance-based algorithms** (e.g., KNN, SVM, clustering).

🔄 **Speeds up convergence** in gradient-based algorithms (e.g., neural networks).

🔥 Helps avoid exploding/vanishing gradients in **deep learning models**.

Preprocessing: Feature Scaling and Normalization



📌 **Why features scaling Matters**

Example data

House	Size (m^2)	Rooms
A	50	2
B	150	3

- Size values are **much larger** than room counts.
- Distance between A and B (simplified):
  - Size difference = 100
  - Room difference = 1
- A model (especially distance-based like KNN or SVM) will treat **size as more important**, even if rooms matter just as much.

➡ The model mostly "sees" the **size difference**, almost ignoring rooms

Common Scaling Methods

Method	Description	Typical Use Case
Min-Max Scaling	Rescales values to a [0, 1] range	Neural networks, image data
Z-score (Standardization)	Centers data to mean 0, std 1	Most ML algorithms, general purpose
Robust Scaling	Uses median and IQR, resistant to outliers	Datasets with heavy-tailed features

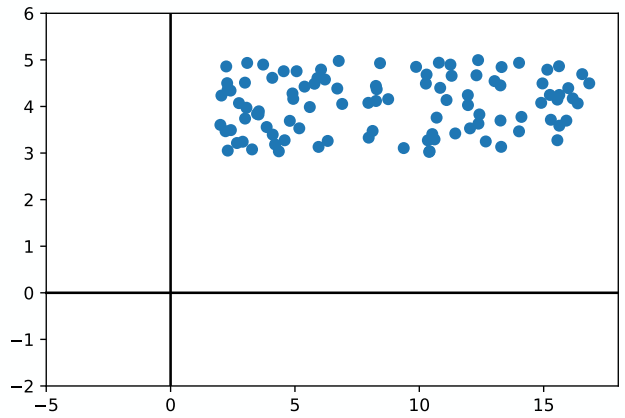
Important when scaling features

✅ Always **fit scalers on training data only**.

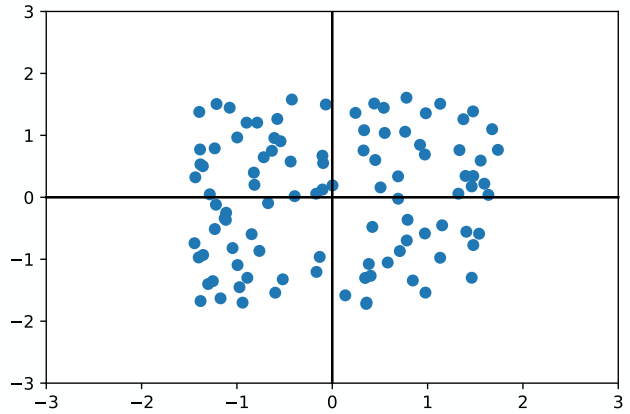
📦 **Normalization parameters (e.g. mean/std) are part of the model** and must be saved and reused in deployment.

🔁 Apply **same transformation to test and production** data.

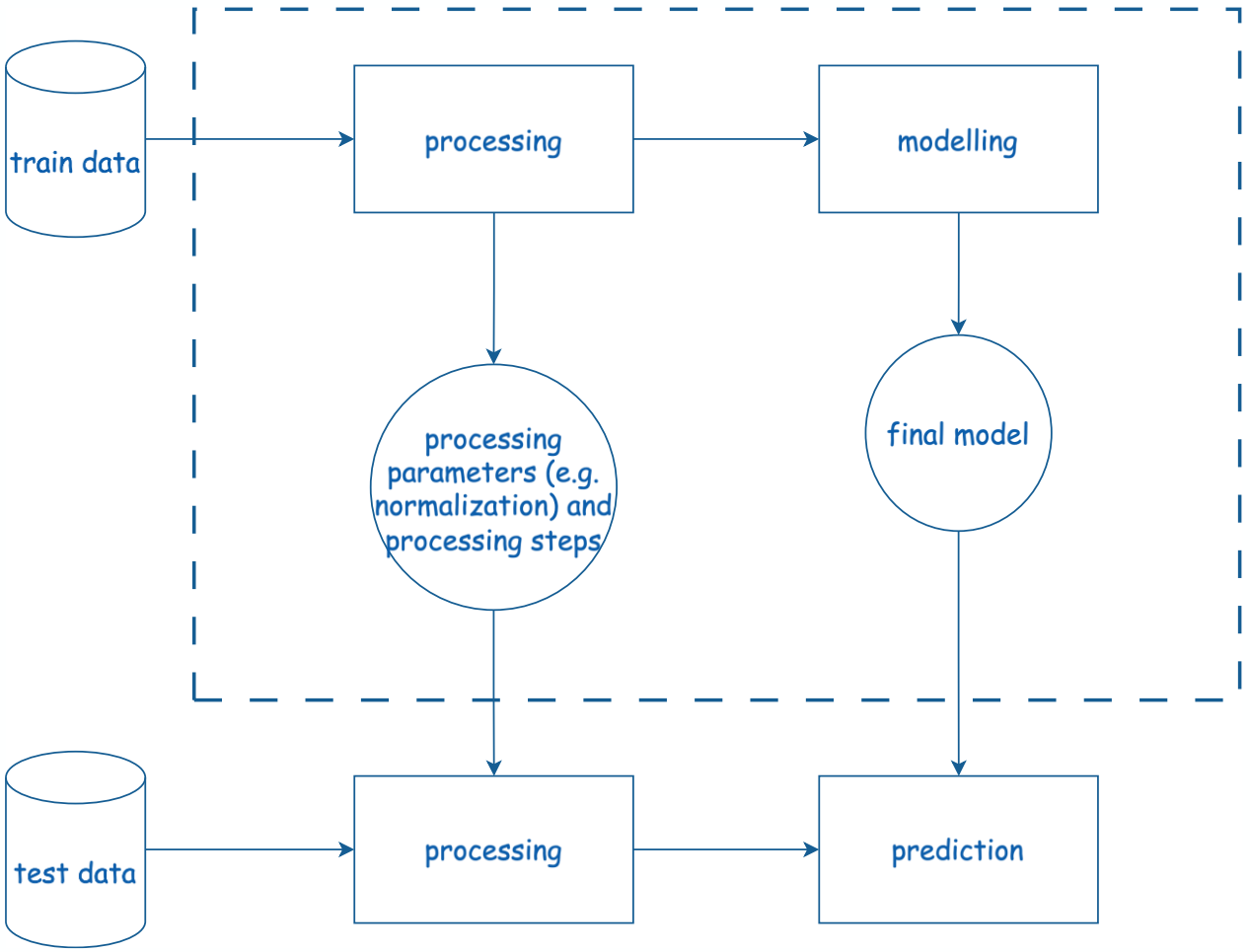
Example: z-score Normalization



$$\mu = \frac{1}{M}\sum_{i=i}^{M} x^{(i)} \qquad \sigma^2 = \frac{1}{M}\sum_{i=i}^{M} (x^{(i)} - \mu)^2$$



$$x_{normalized} = \frac{x - \mu}{\sigma^2}$$

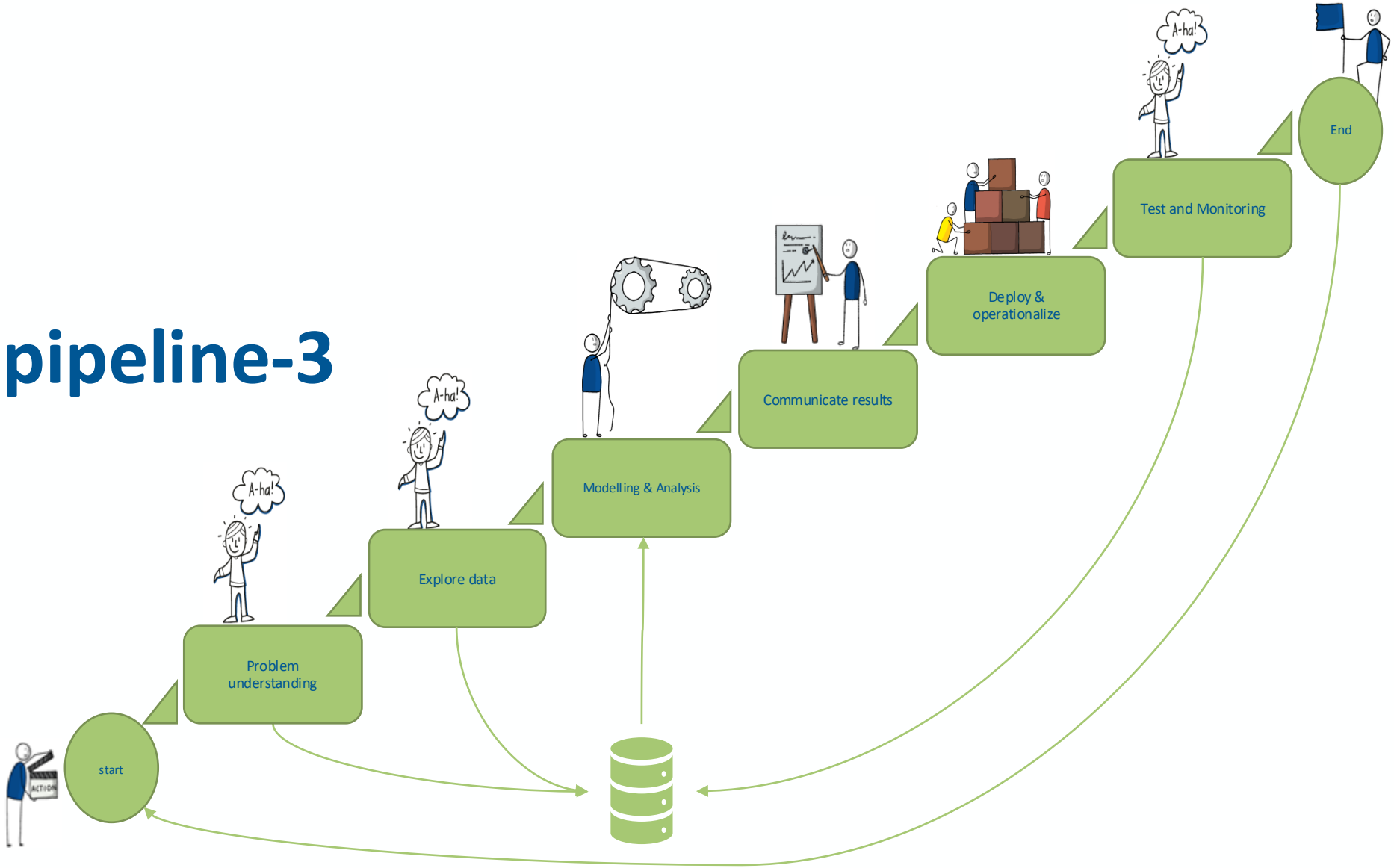


μ and σ become part of the model and must be the same in production

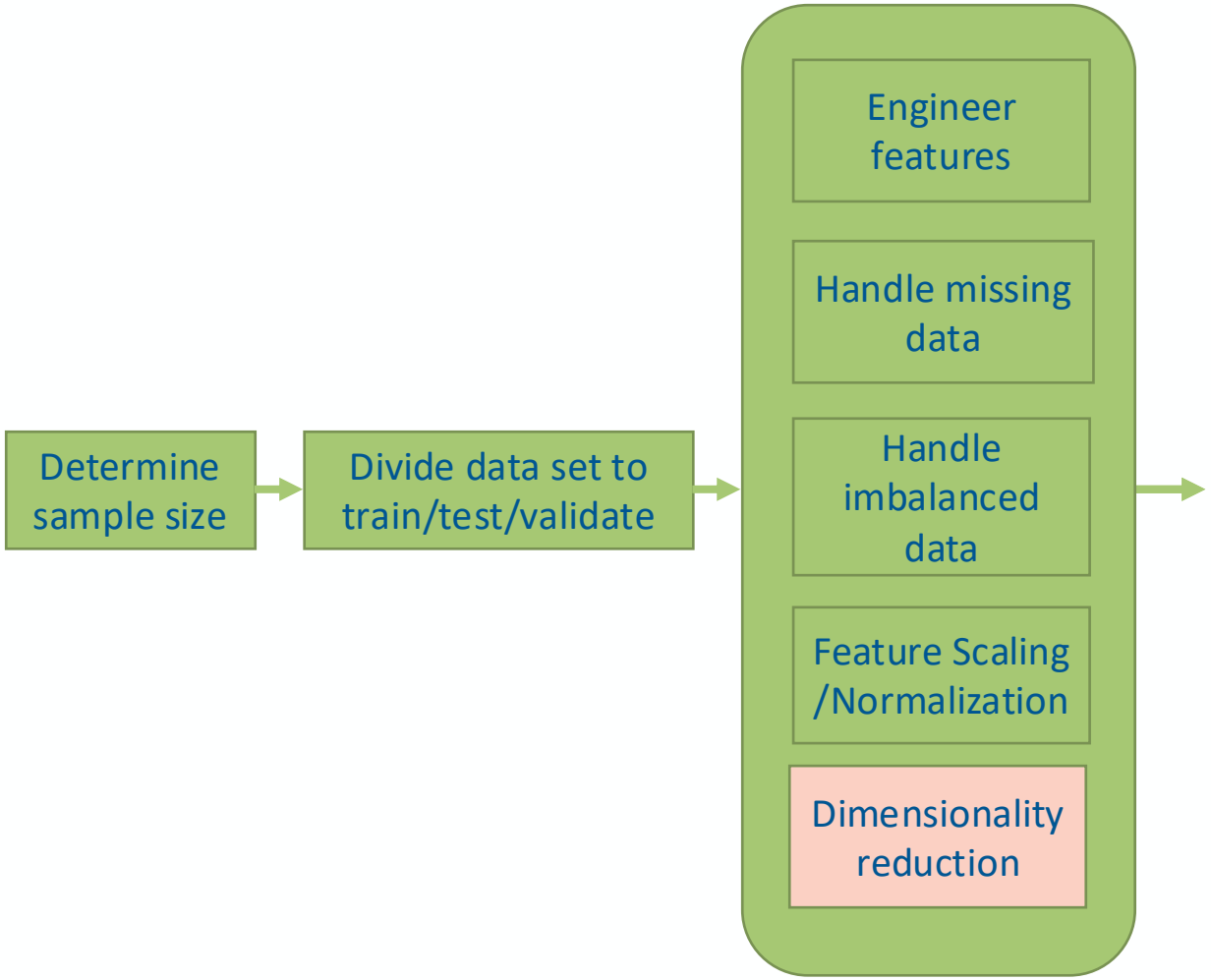
Data Mining pipeline-3

Prof. Dr. Matteo Marouf

05.05. 2026

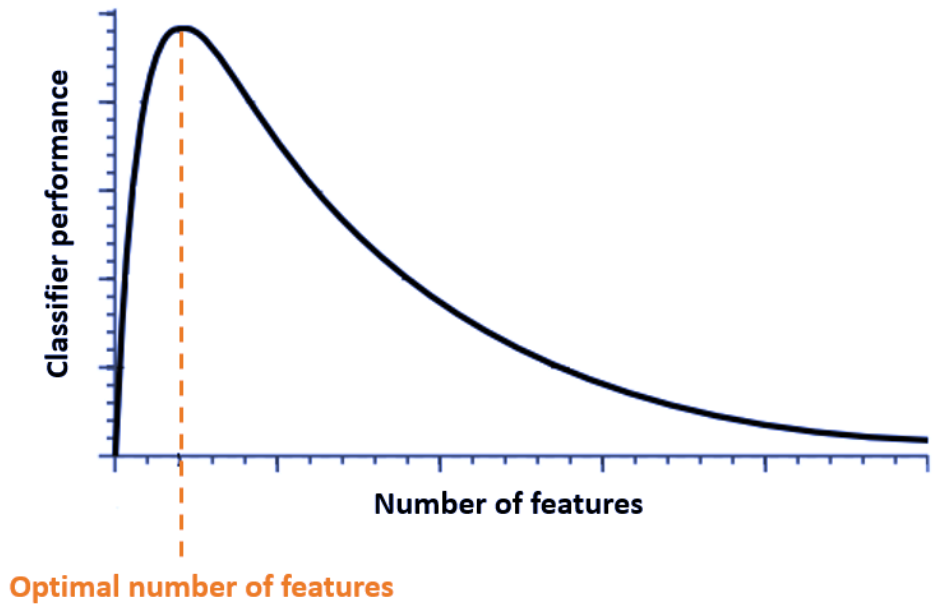


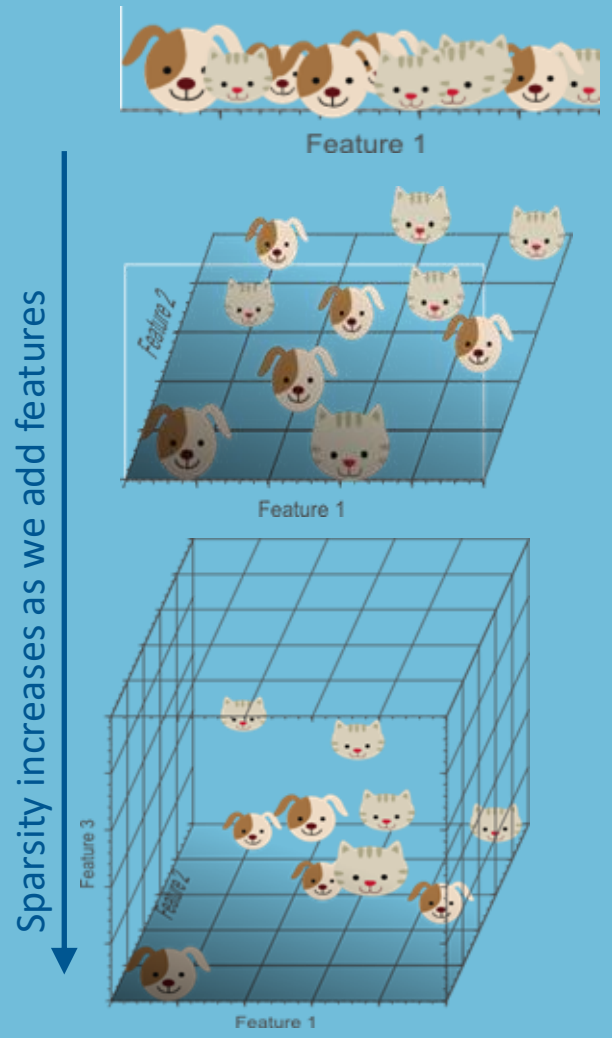
Preprocessing: Dimensionality reduction vs. Feature Selection



- **Feature selection** is simply reducing the dimensionality by excluding given features without changing them.
- **Dimensionality reduction** transforms features into a lower dimension (e.g. from 1000 to 10 features) using **Linear and nonlinear techniques.**

Data Challenge: Curse of Dimensionality in Classical Machine Learning

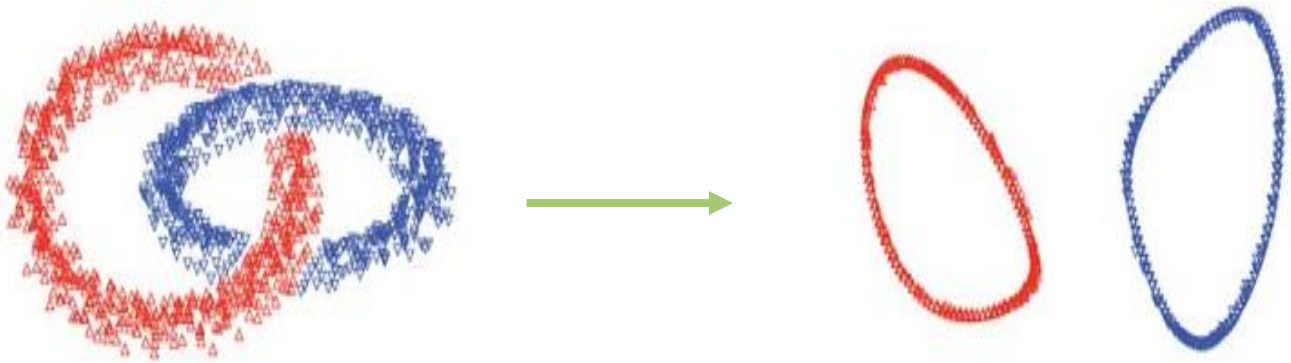




As the dimensionality increases, the classification gets easier until the optimal number of features is reached. Further increasing the dimensionality without increasing the number of training samples makes the classification more difficult and results in a decrease in classifier performance.

Preprocessing: Dimensionality reduction

- Having too many variables leads to Computational complexity
- The performance of machine learning algorithms can deteriorate when exposed to an excessive number of input variables, especially in scenarios where the dataset lacks sufficient data points.



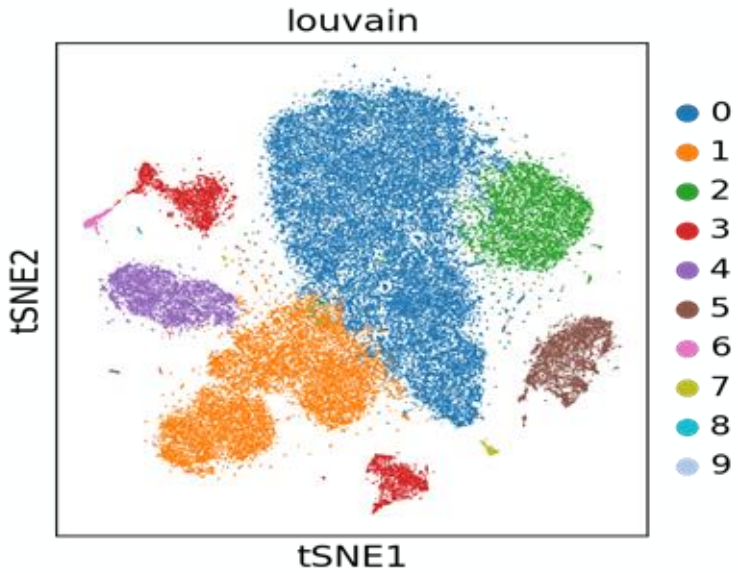
Moving from 3-d to 2-d makes the distinction easier

Reducing the dimensionality helps to separate the different classes or groups in the data and reduces the computational complexity as we process and model lower dimensional data.

Preprocessing: Dimensionality reduction

Dimensionality reduction is also Essential for visualization of high dimensional data

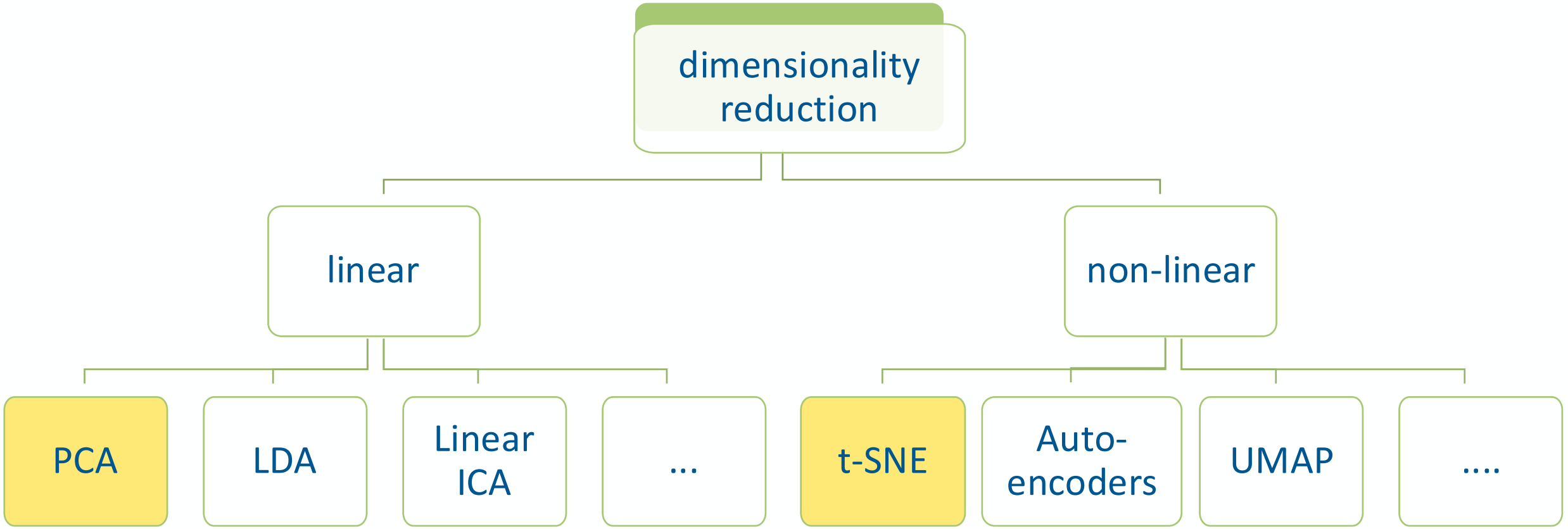
	Gene 1	Gene 2	. . .	Gene N
Cell 1	**3**	**7**	. .	**0**
Cell 2	**0**	**0**	. .	**40**
…	. . .	. . .	. . .	. . .
Cell M	**19**	**0**	. .	**1**



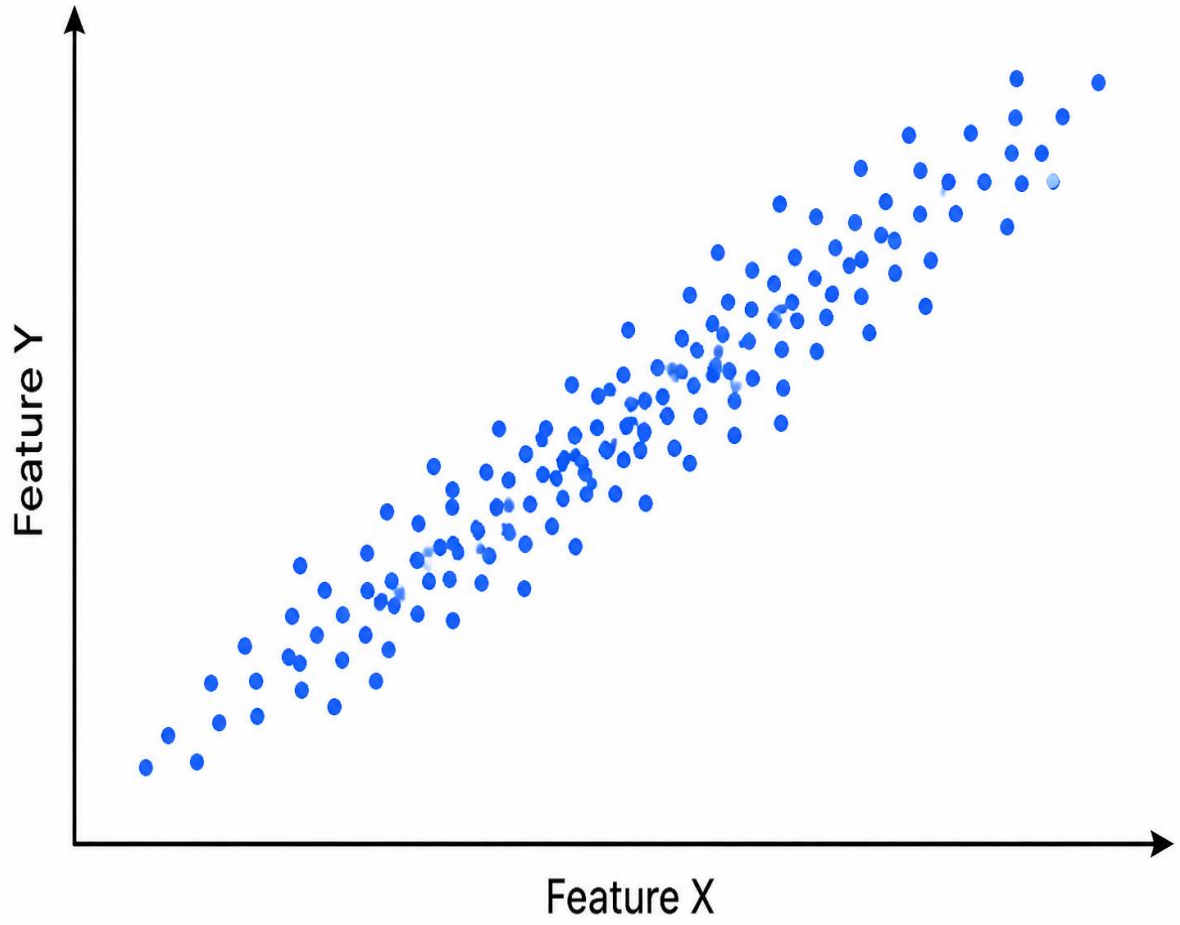
Example: scRNA-seq: millions of cells, up 30.000 measured genes

reducing the dimensionality helps to visualize the genes cells matrix in scRNA-seq

Main types of Dimensionlity reduction

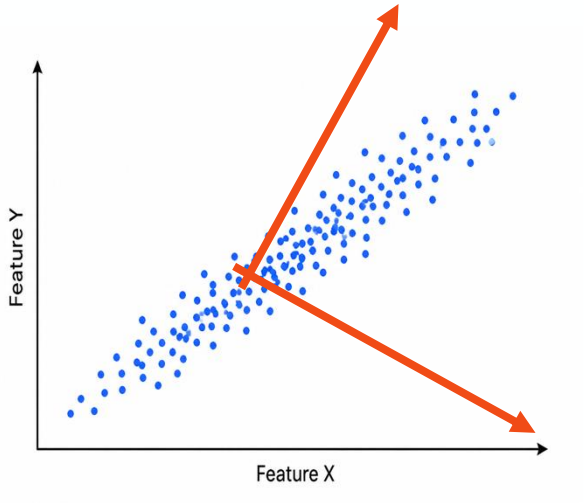


- Imagine a cloud of data points in 3 dimensional space: features X, Y, Z.
- The data often spreads more in some directions than others. This spread reflects the variability.
- This variability of X,Y, Z is what is important for modelling (e.g., classification) but it is is spanned to the 3 dimensions.

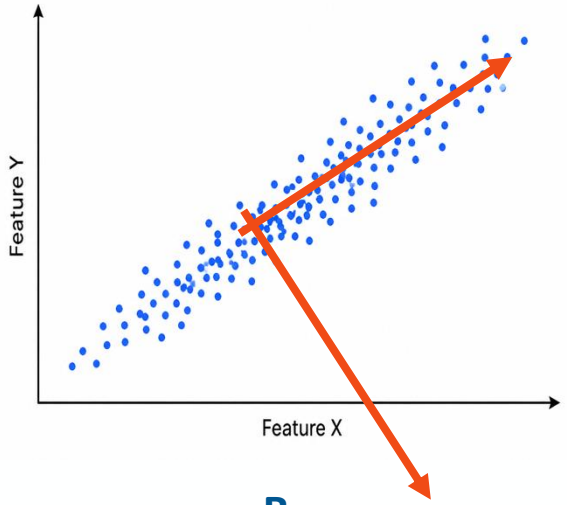


Quiz

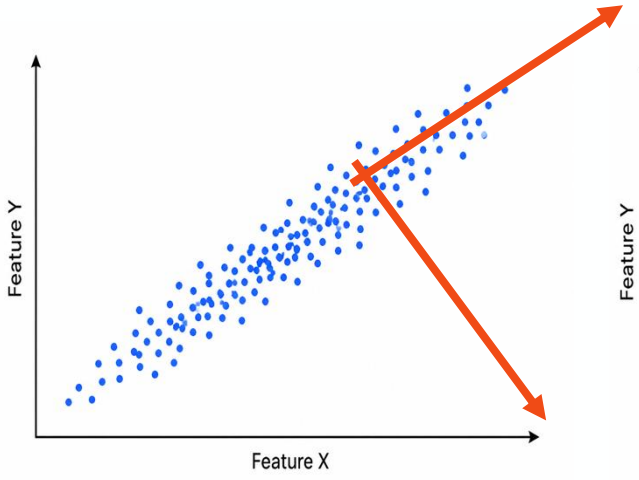
- Can we transform the data into a new orthogonal coordinate system where variance is maximized along successive dimensions in descending order?



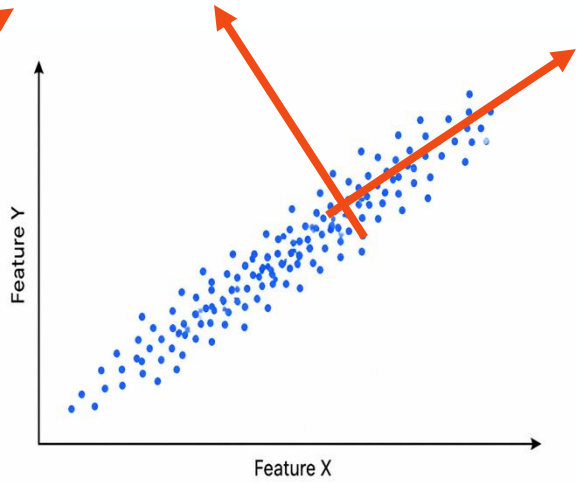
A



B



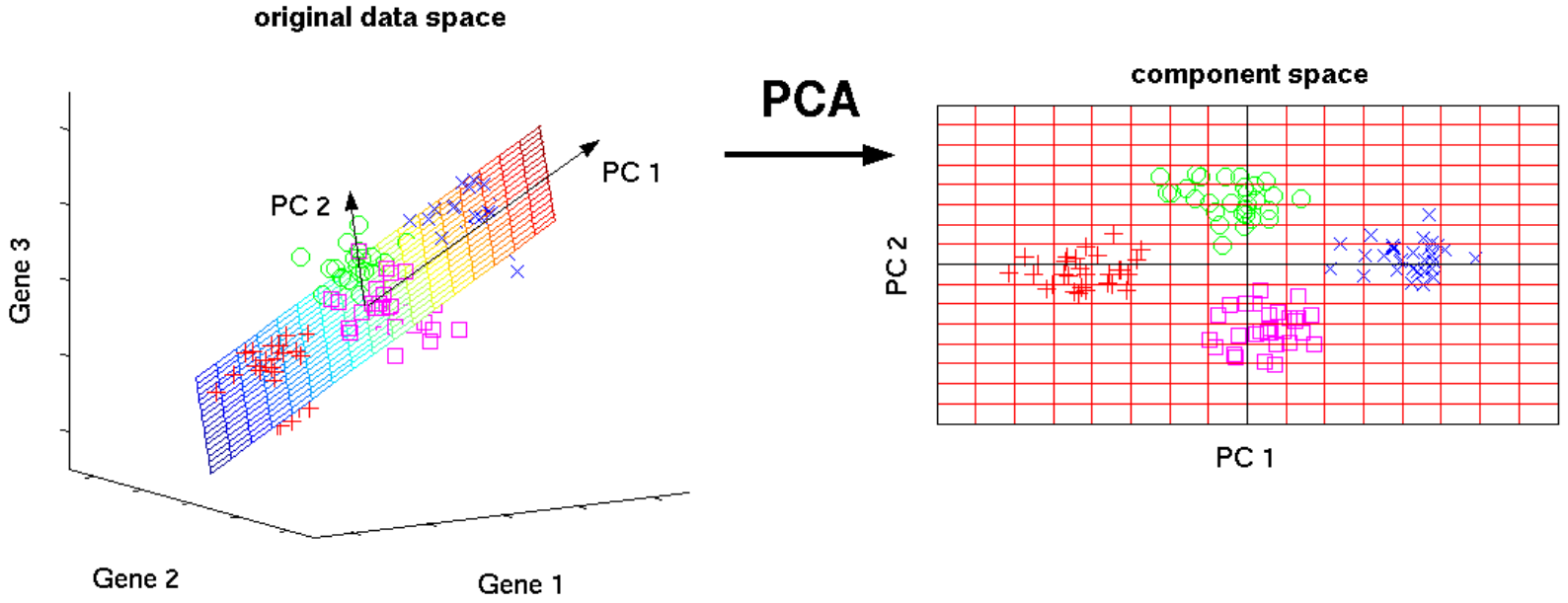
C



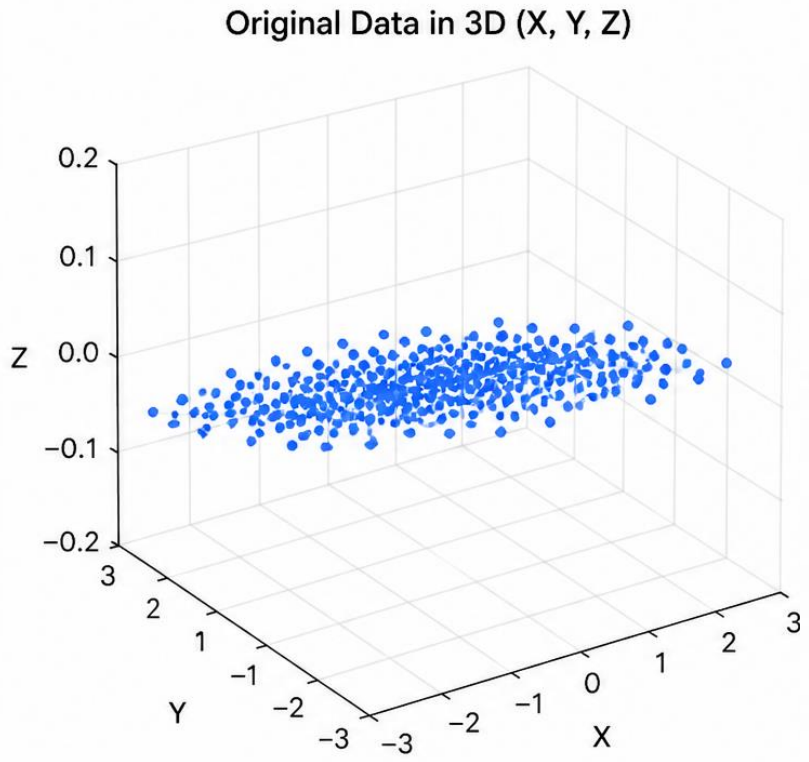
D

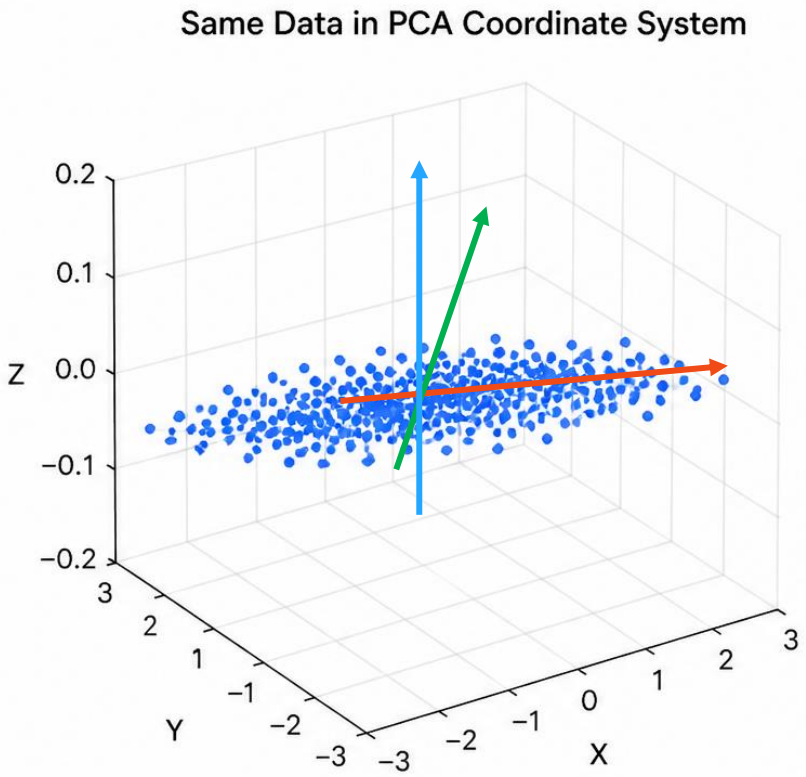
PCA (Principal Component Analysis)

- **PCA (Principal Component Analysis)** is a linear technique used to **reduce the number of features** in a dataset while preserving as much **variance** (information) as possible.
- It **transforms** data into new variables called **Principal Components (PCs)**.
- You use PCA to **simplify complex data without losing the important patterns**.



Approaches to analyse and interpret biological profile data. University of Potsdam, Germany. 2006. Ph.D. thesis

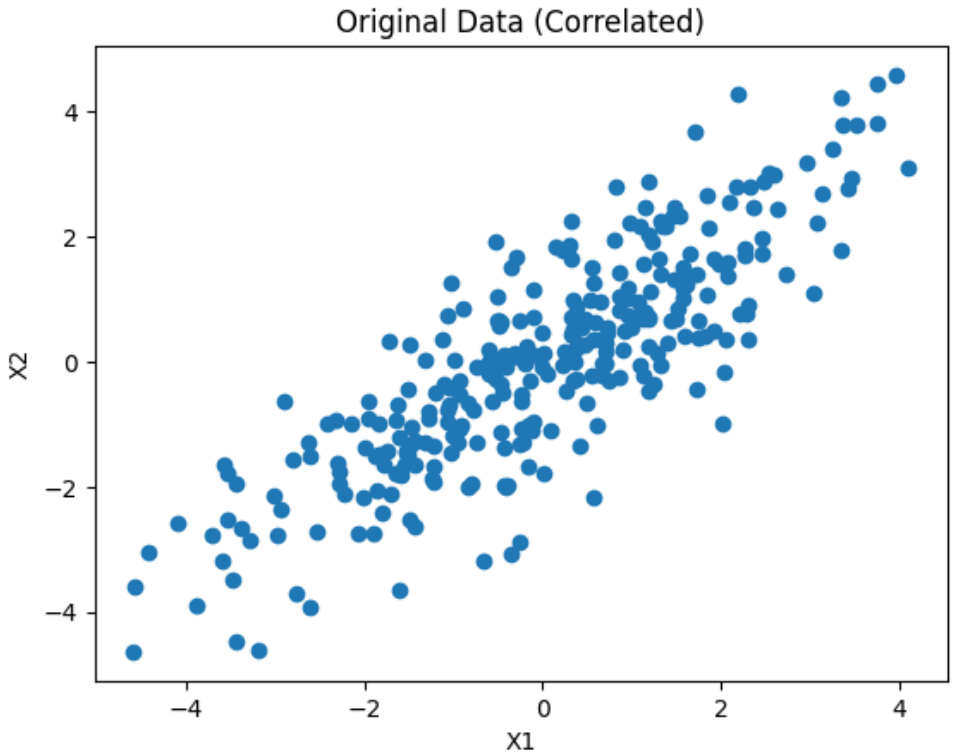


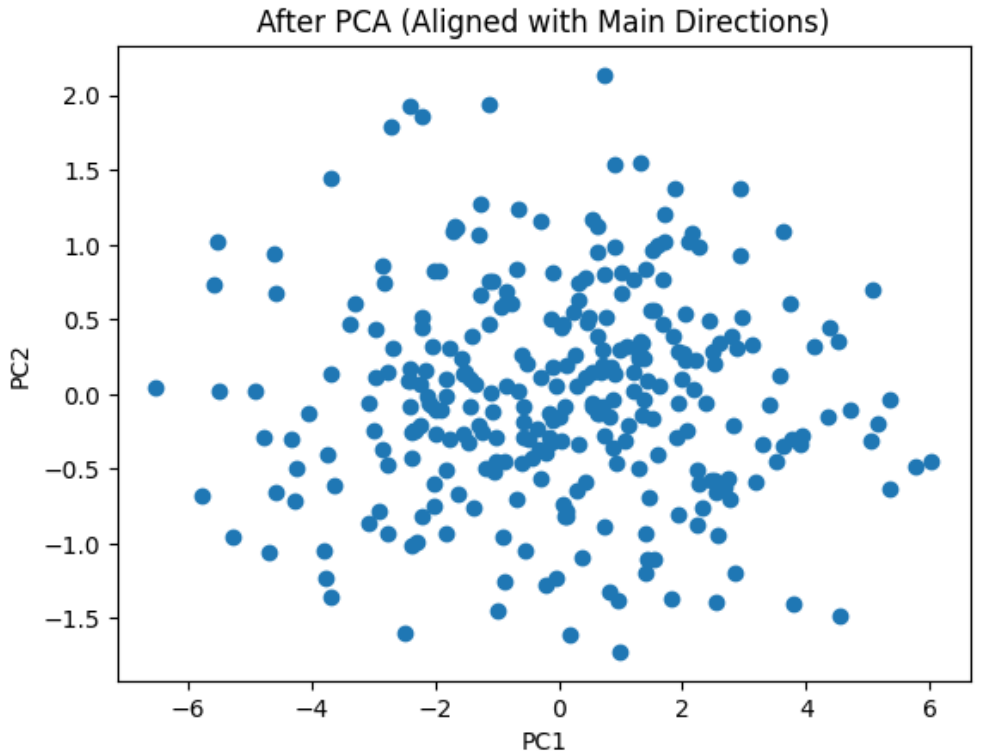


Z has almost no variability
(points lie close to the XY plane)

PC1 (red): direction of maximum variance
PC2 (green): direction of second maximum variance
PC3 (blue): direction of minimum variance
(almost no spread)

- PCA finds **main orthogonal directions of variation**
- Then it **rotates the coordinate system** to align with those directions
- We can remove PC3 without losing a lot of variability, doing so we reduced the dimensionality


Original Data (Correlated)
X1
X2
4
2
0
−2
−4
−4
−2
0
2
4


After PCA (Aligned with Main Directions)
PC1
PC2
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−6
−4
−2
0
2
4
6

The Mechanics of PCA

Standardize the Data

- this shifts data around zero and ensures we focus on the maximum variance of the whole data

Find Principal Components

- It finds new axes that explain the maximum variance.

Project Data onto top Components

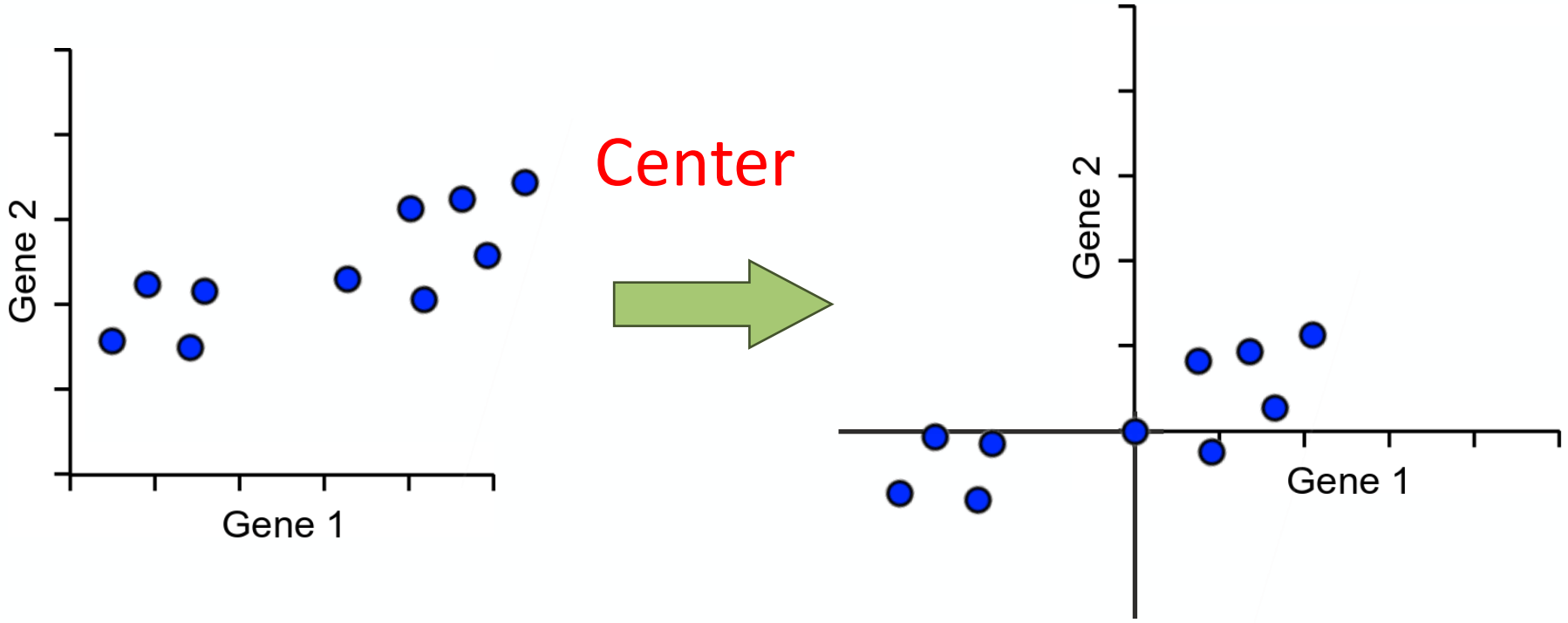
- Data is transformed into a new coordinate system.

$$Cov(x, y) = \frac{1}{(n-1)} \sum (x_i - \bar{x})(y_i - \bar{y})$$

Example: reducing the dimensionality of scRNA-seq

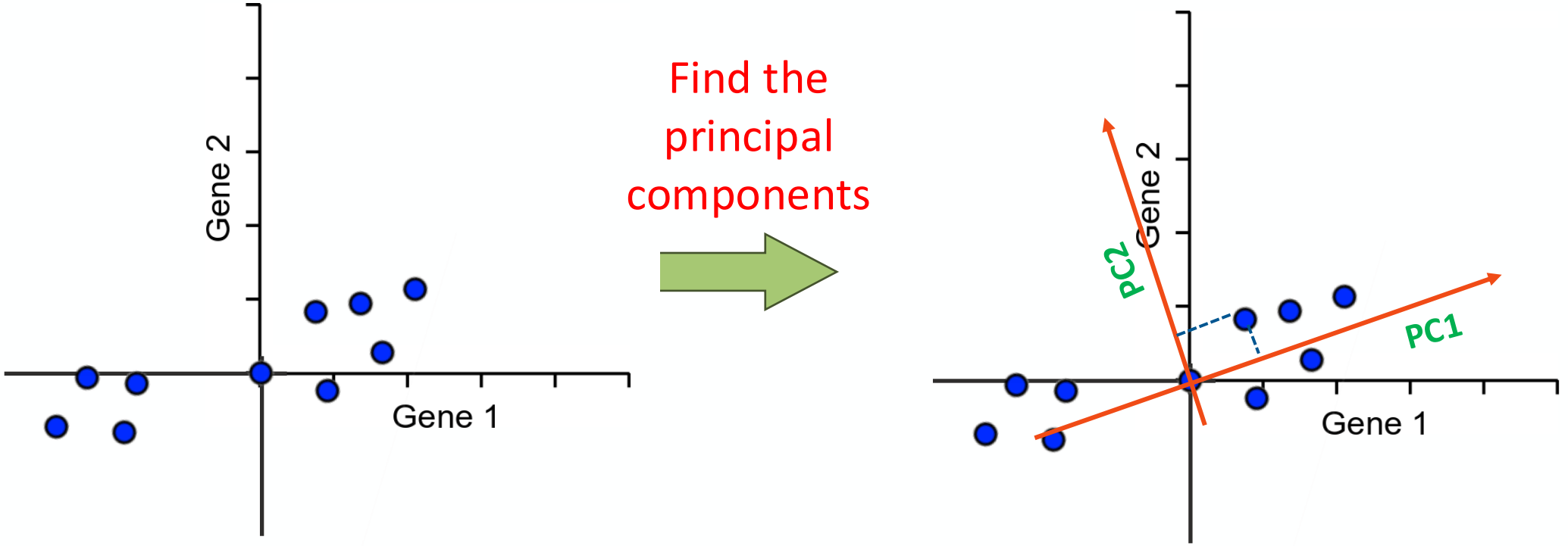
	Gene 1	Gene 2	. . .	Gene N
Cell 1	3	7	. .	0
Cell 2	0	0	. .	40
…	. . .	. . .	. . .	. . .
Cell M	19	0	. .	1

100.000 cells, up 30.000 measured genes per cell
reducing the dimensionality helps to visualize the genes cells matrix in scRNA-seq



- **Standardize the Data** – (e.g., subtracting the mean of each variable from the values), making the mean of each variable equal to zero.

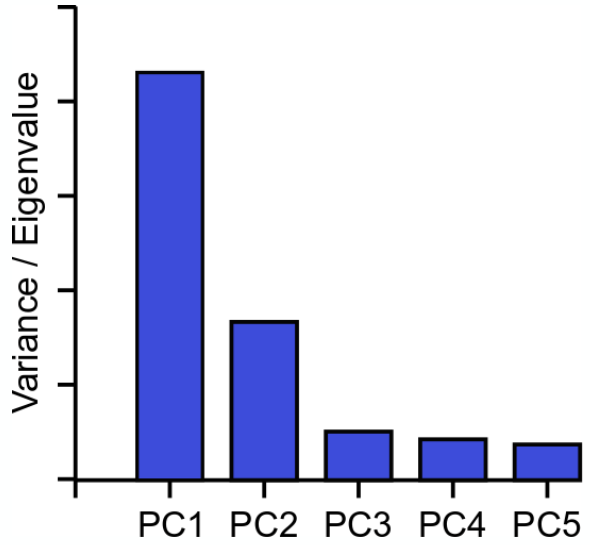
- **Why? Centering ensures** that PCA captures the *true shape and spread* of the data, not the location of the cloud.



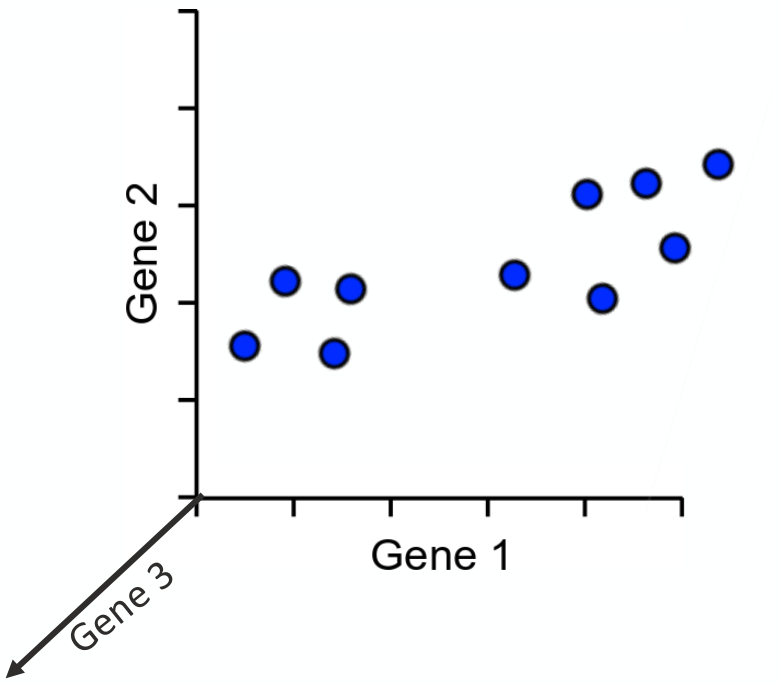
1. **Compute the covariance matrix**
   This matrix shows how features vary with each other.
2. **Calculate eigenvalues and eigenvectors** of the covariance matrix
   - **Eigenvectors** give the coordinate or the direction (principal components).
   - **Eigenvalues** tell you how much variance is along each direction.
3. **Sort eigenvectors by eigenvalues (descending order)**
   The first few eigenvectors (with the highest eigenvalues) are the most important — they capture the most variance.

Explaining Variance (Scree Plot)

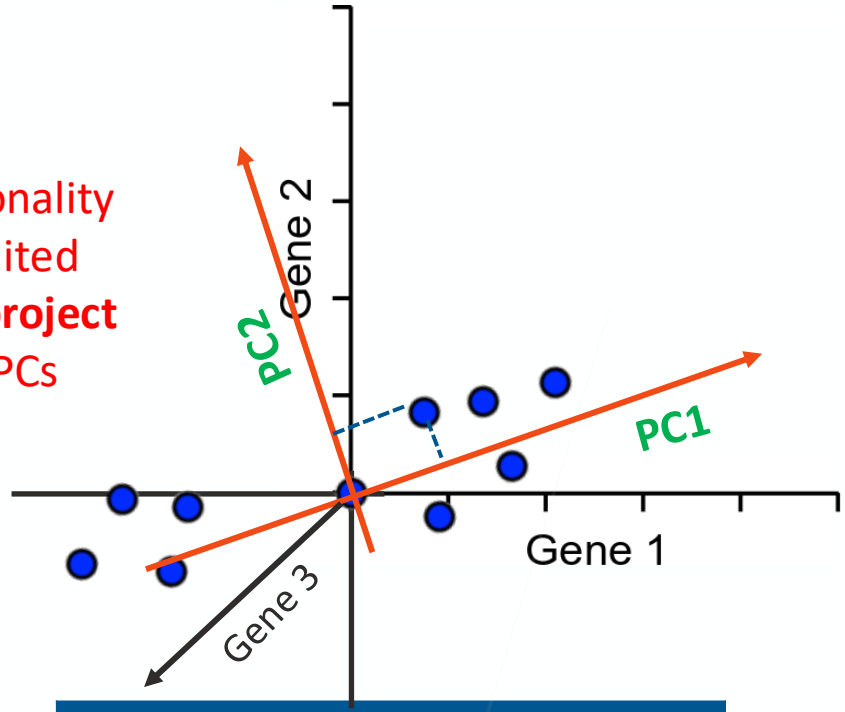
- ✓ Each PC always explains some proportion of the total variance in the data.
- ✓ The first PC will capture the maximum variance it can be captured
- ✓ The second PC will capture the maximum variance an orthogonal dimension can capture.
- ✓ Since we only plot 2 dimensions, so we can plot the first 2 PCs whose variance explain the most
- ✓ Orthogonal directions **don't overlap in what they measure** — they capture **independent modes of variation**.



PCA provides a measure of variance called the 'eigenvalue'



Reduce the dimensionality by keeping only a limited number of PCs and **project** only to the kept top PCs



	Gene 1	Gene 2	Gene 3
Cell 1	**3**	**7**	**1**
Cell 2	**0**	**0**	**12**
…	. . .	. . .	. . .
Cell M	**19**	**0**	**4**

	PC 1	PC 2
Cell 1	**-10**	**4**
Cell 2	**0**	**5**
…	. . .	. . .
Cell M	**19**	**3**

The same idea extends to larger numbers of dimensions (n)

We will get same number of PCs as genes, but we keep only limited number of them and project to reduce dimensionality

	Gene 1	Gene 2	. . .	Gene 30.000
Cell 1	**3**	**7**	. .	**0**
Cell 2	**0**	**0**	. .	**40**
…	. . .	. . .	. . .	. . .
Cell M	**19**	**0**	. .	**1**

	PC 1	PC 2	. . .	PC 10
Cell 1	**-10**	**4**	. .	**0**
Cell 2	**0**	**5**	. .	**0.2**
…	. . .	. . .	. . .	. . .
Cell M	**19**	**3**	. .	**-0.3**

scRNA-seq: 100.000 cells, up 30.000 measured genes

- Reducing the dimensionality helps to keep only top 10 PCs that capture most of the variance in the genes-cells matrix.
- We can use these 10 PCs for further downstream analysis
- We can use the top 2 for visualization

Loadings in PCA

✅ 3. Projection = Dot Product = Linear Combination

Projecting data means taking the **dot product** between the data vector and each eigenvector:

$$PCi = X \cdot vi$$

- X: (centered) data
- vi: eigenvector (direction of principal component)

The Dot product = linear combination of features which looks like this:

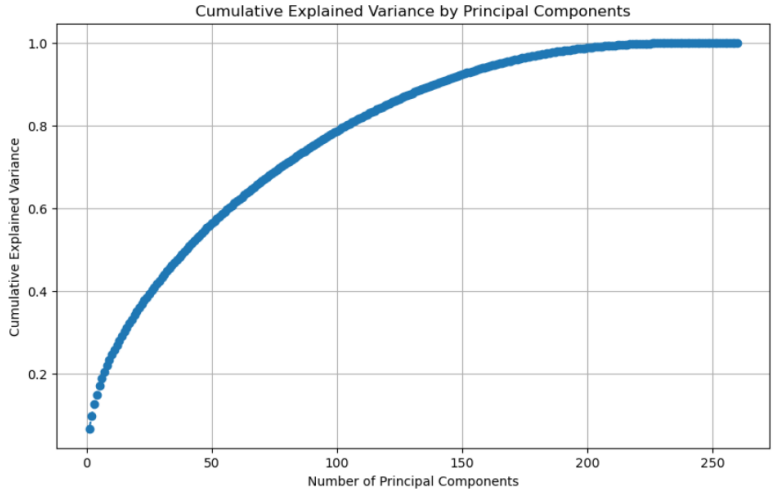
$$PC_1 = a_1x_1 + a_2x_2 + \ldots + a_dx_d$$

- $x1, x2, \ldots, xd$ are the original features,
- $a1, a2, \ldots, ad$ are weights (loadings),
- The result is a **new feature** — a principal component!

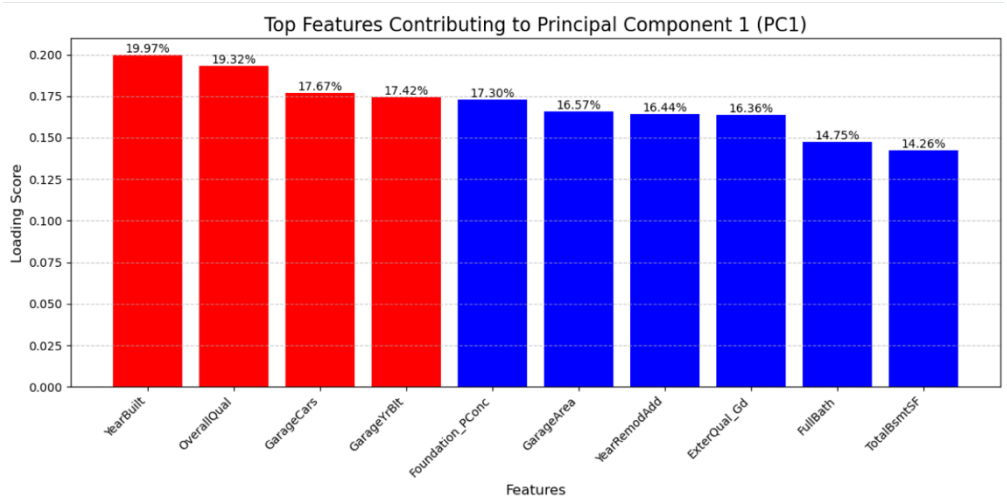
PCA Example

House Price Prediction Data → 81 Columns

Converting Categorical Data → 260 columns

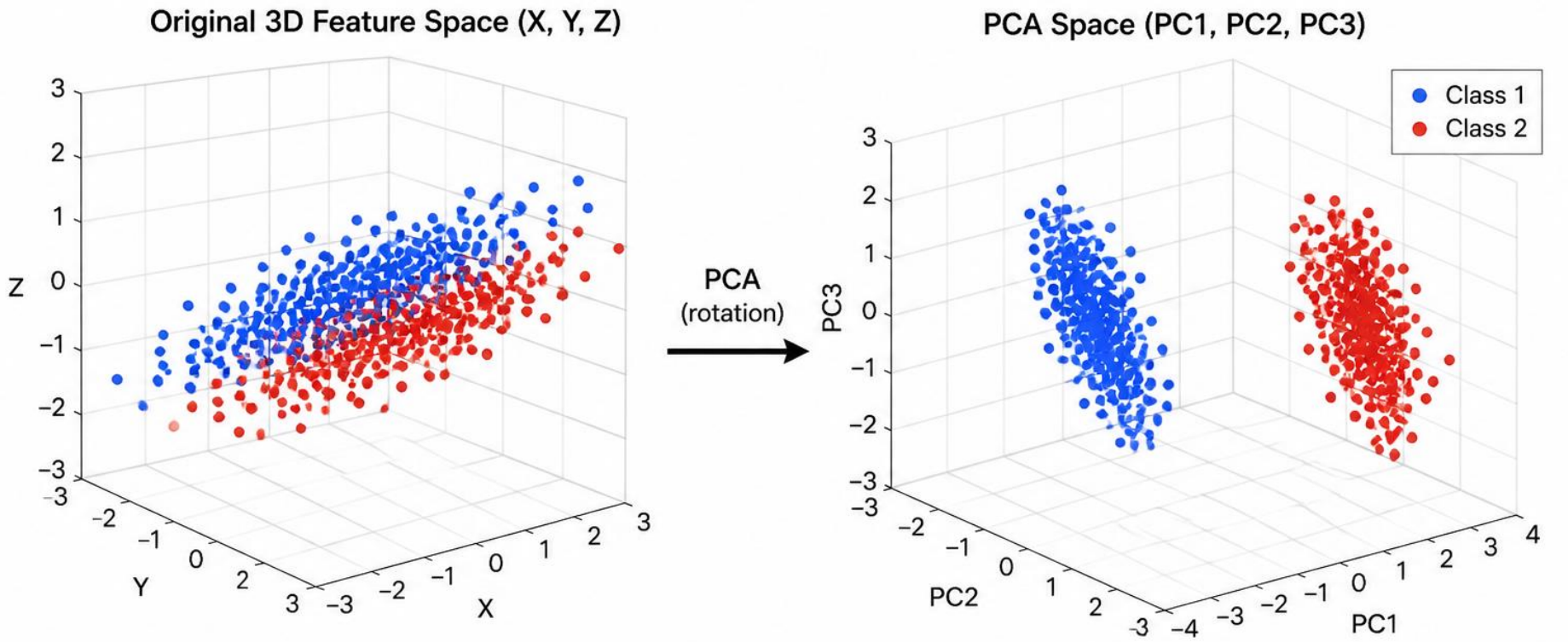


- After running PCA, it's super helpful to know **which original features contribute most to each principal component (PC).**
- Loadings represent the correlations between the original variables and the principal components. They indicate how much each variable contributes to a specific principal component — that tells you what **drives the variance** in your data



https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques/data

PCA captures linear correlations well but….



Original Space

- Classes overlap heavily
- Hard to see a clear boundary
- Separation is not obvious in any single original dimension

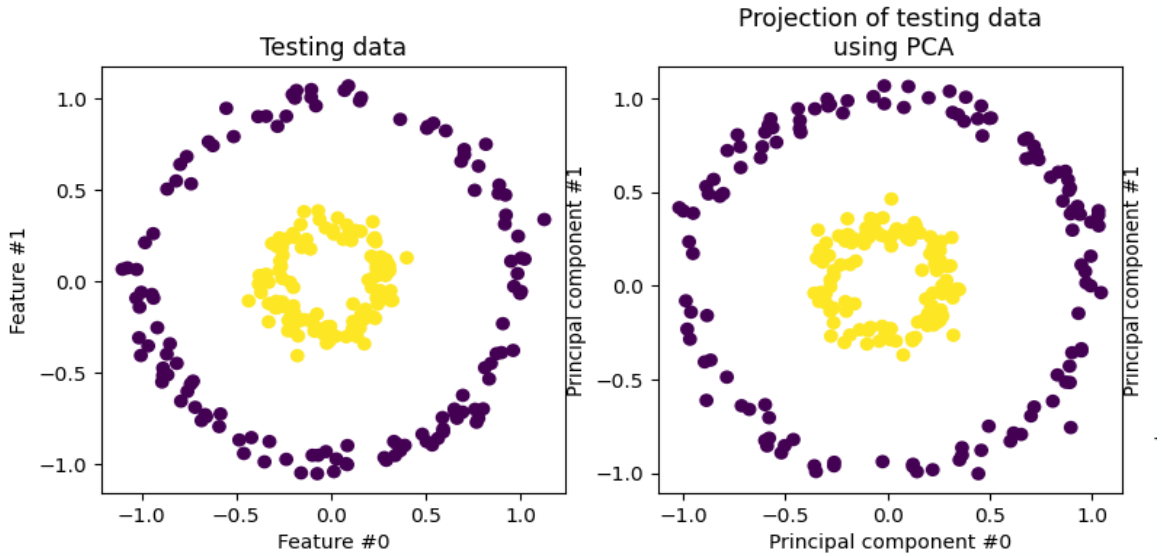
PCA Space

- Classes are clearly separated along PC1
- PC1 captures the most discriminative information
- Easier to visualize and classify

PCA rotates the coordinate system to align with directions of maximum variance, revealing the structure and improving class separability.

…….but it ignores higher-order interactions.

- If the dataset is highly nonlinear, PCA may not be the best approach for dimensionality reduction
- If the data lies on a **curved** or **manifold-like structure,** PCA will **project it onto a straight plane**, destroying the **intrinsic geometry** of the data.



Non-linear data

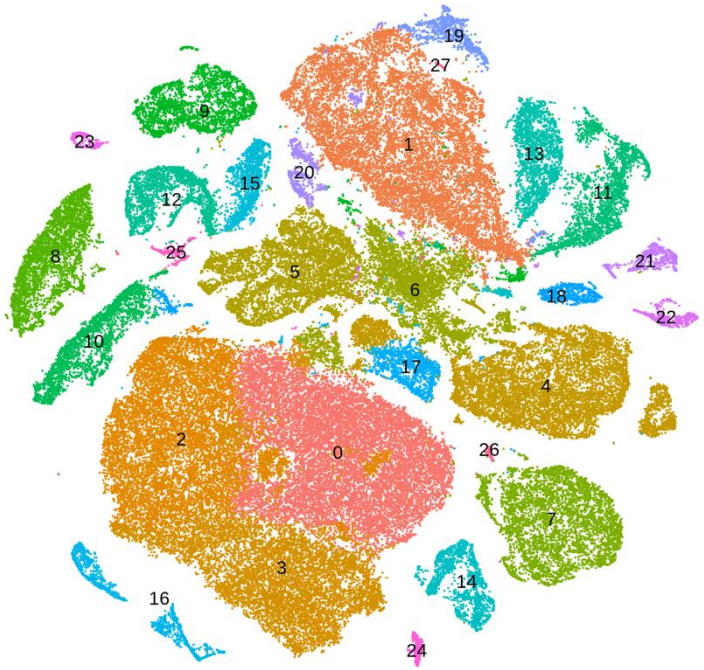
Wang, Quan. "Kernel principal component analysis and its applications in face recognition and active shape models." *arXiv preprint arXiv:1207.3538* (2012).

t-SNE

- t-SNE (T-Distributed Stochastic Neighbour Embedding) is a ML algorithm that is mainly used to visualize high-dimensional data.
- It's a **non-linear dimensionality** reduction technique that can separate data that can't be separated by a **straight line.**
- t-SNE preserves the **local structure** of the data by keeping similar data points close together in a lower-dimensional space (original clustering is preserved)

	Gene 1	Gene 2	. . .	Gene 30.000
Cell 1	**3**	**7**	. .	**0**
Cell 2	**0**	**0**	. .	**40**
…	. . .	. . .	. . .	. . .
Cell M	**19**	**0**	. .	**1**

t-SNE



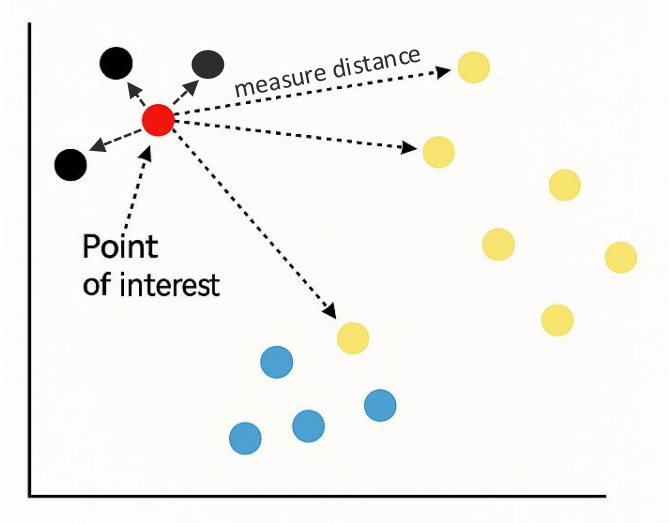
- For each point x_i, t-SNE calculates the probability of another point x_j being its (x_i) **neighbor** if x_j was picked in proportion to a Gaussian cantered at x_i.

$$P_{i|j} = \frac{\exp\left(-\left||x_i - x_j|\right|^2 / 2\,\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\left||x_i - x_k|\right|^2 / 2\,\sigma_i^2\right)}$$

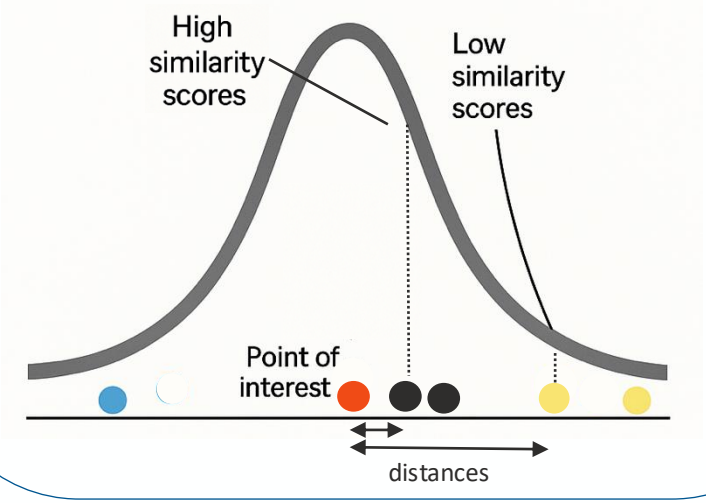
where:

- $\left||x_i - x_j|\right|^2$ is the **Euclidean distance** between points.
- σ_i is the **perplexity-based scaling factor** controlling the neighborhood size (**perplexity controls** how many data points we choose around each data point and then σ_i of the data points is calculated)
- The sum ensures probabilities **normalize to 1**.

Original multidimensional space



Conditional probability in proportion to Gaussian centered at x_i

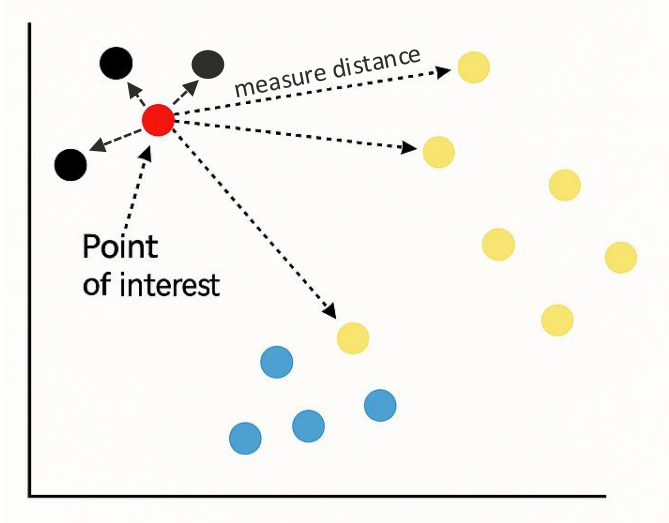


- The resulted matrix is **asymmetric**, but we want a **symmetric joint probability matrix.**

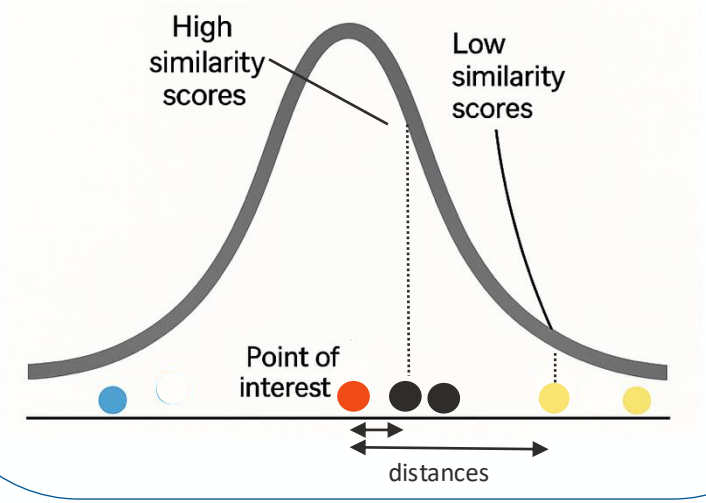
- To create a **symmetric similarity** measure, we define **joint probability** of **similarity** as:

$$P_{ij} = \frac{P_{j|i} + P_{i|j}}{2N}$$

Original multidimensional space

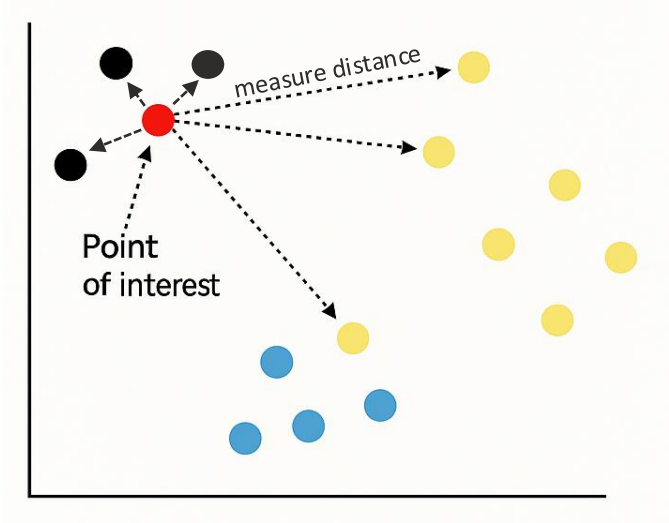


Conditional probability in proportion to Gaussian centered at x_i

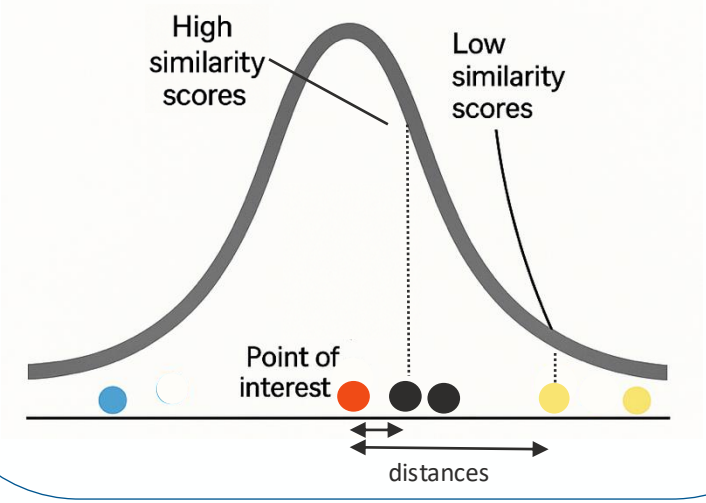


- If two points are close, their joint probability is high
- If two points are far, their joint probability is low
- The matrix of all joint probability forms a symmetric similirity matrix

Original multidimensional space



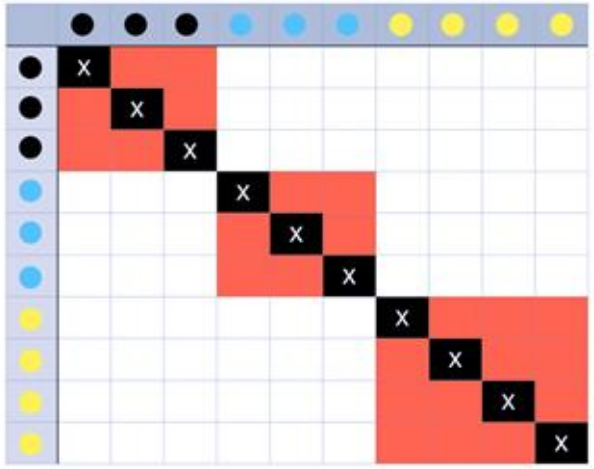
Conditional probability in proportion to Gaussian centered at x_i



How t-SNE works

High dimensional space

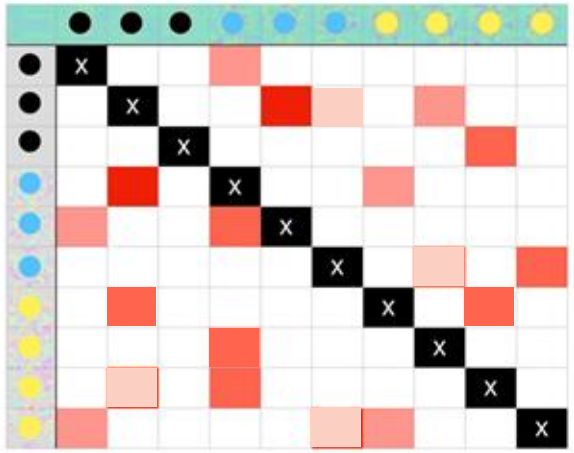
Similarities modelled with Gaussian distribution in the original space



P

Randomly scattering the points into Lower Dimension

Similarities modelled with t-student distribution in the lowered space

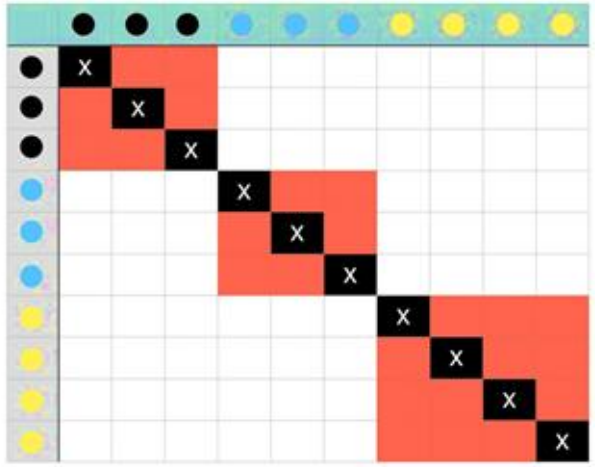


Q

Iterate

- Some points attract.
- Some points repel.

The final similarity matrix after many iterations,



Optimize so that the new dimensional space with a t-student distribution has similarities as equal to the original dimensional space with a gaussian distribution, preserving the neighborhood relationships.

How t-SNE works

P Q

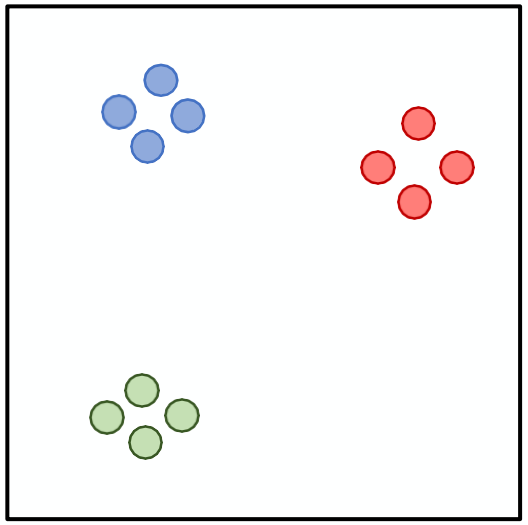
Gradient Descent is used to minimize the Kullback-Leibler (KL) Divergence to measures the **difference between the probability distributions**.

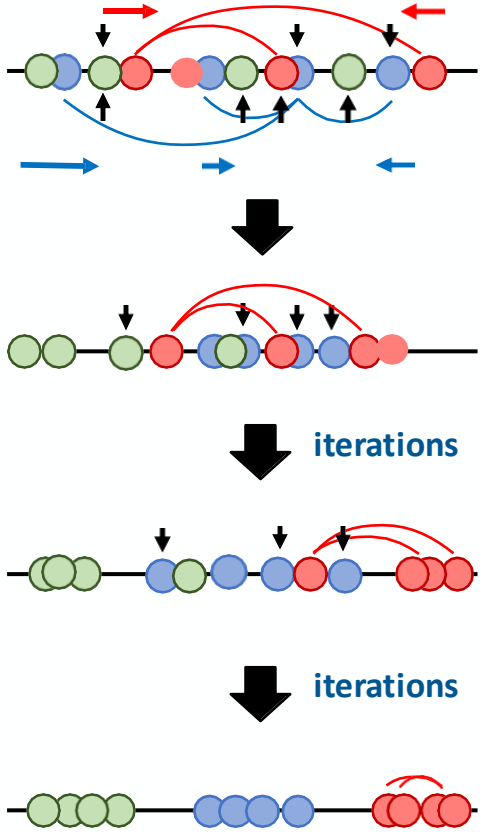
$$KL(P \parallel Q) = \sum_{i \neq j} p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right)$$

Repulsive and attractive forces are applied so that points end up in a low dimensional map that optimizes the KL divergence

How t-SNE works

Here's a basic 2-D Scatter Plot





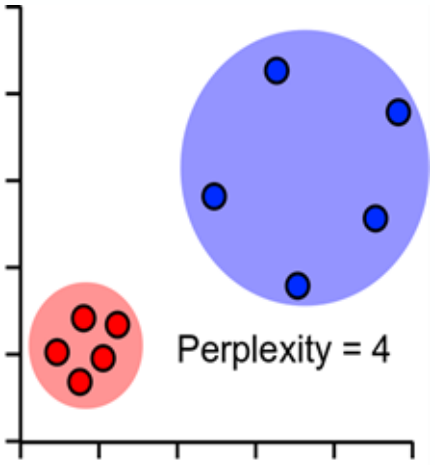
1. Put the points in a random order.

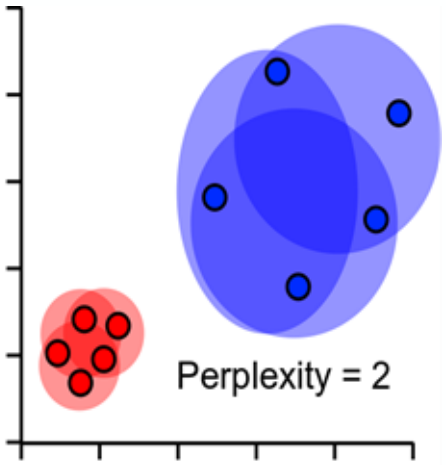
2. Figure out where to move the first point
   - Some points attract.
   - Some points repel.

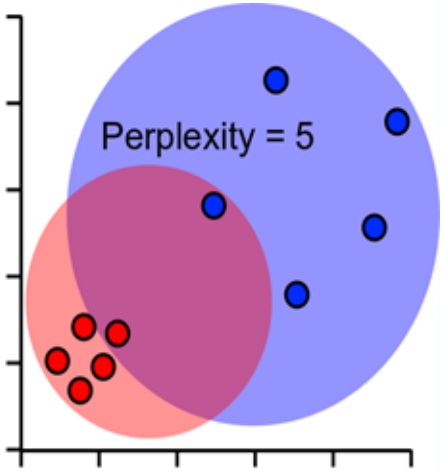
3. Iterate 2nd point till we get the final result.
   -Stop after distances have converged

Perplexity - Hyperparameter

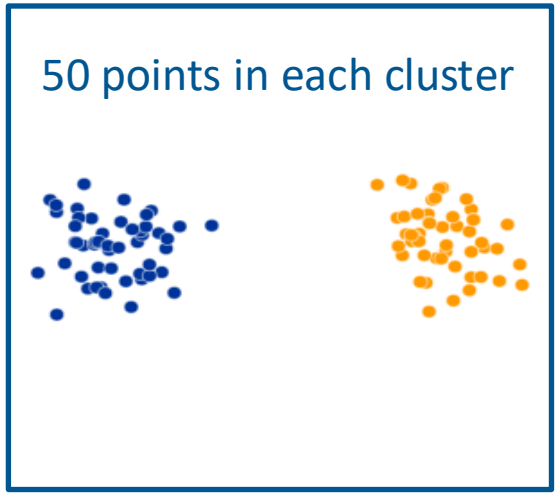
- **Perplexity (a hyperparameter)** controls how many neighbors each point considers. (influences proximity estimation)
- It controls the balance between focusing on local vs. global structure in the data.
- It influences how t-SNE defines "neighborhoods" when calculating the similarities between data points



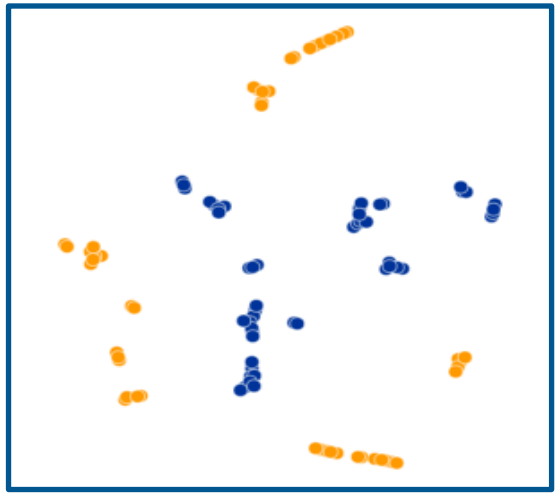




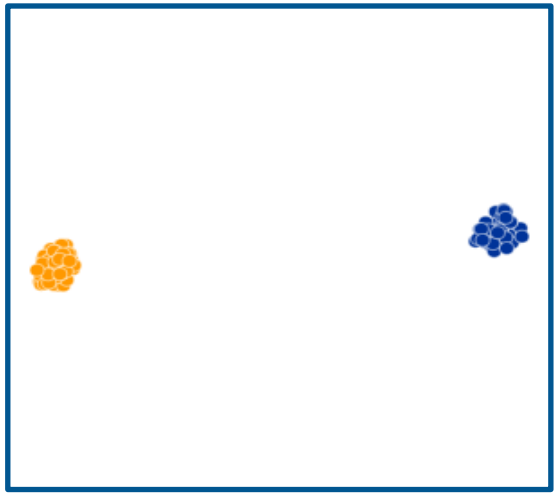
Quiz: t-SNE Perplexity
What is a good perplexity?



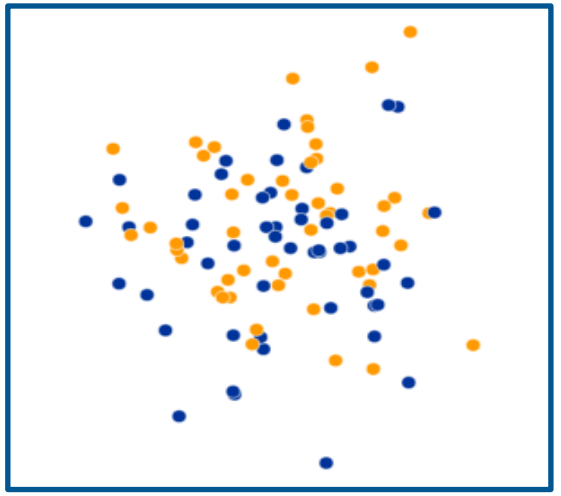
Original (2 dimensional)



A: Perplexity = 2



B: Perplexity = 30



C: Perplexity = 100

Comparision between t-SNE and PCA

Aspect	PCA	t-SNE
Purpose	Dimensionality Reduction and feature transformation	Data visualisation in low dimensions (typically 2D)
Type of algorithm	**Deterministic**	**Stochastic**
Approach	Linear transformation (distance makes sense)	Probabilistic (local similarity) Embedding
Speed	Fast	Slow
Noisy data	Helps against noisy data	Can’t cope with noisy data
Structure Preserved	Global	Local Only

So what?

Sometimes, it is useful to combine PCA and t-SNE to get the best of both worlds. This is done by applying PCA followed by t-SNE if needed.

Why Learn how PCA and t-SNE work?

Understanding **PCA** and **t-SNE** helps you develop the mindset of a machine learning practitioner. These techniques introduce you to essential ML concepts like:

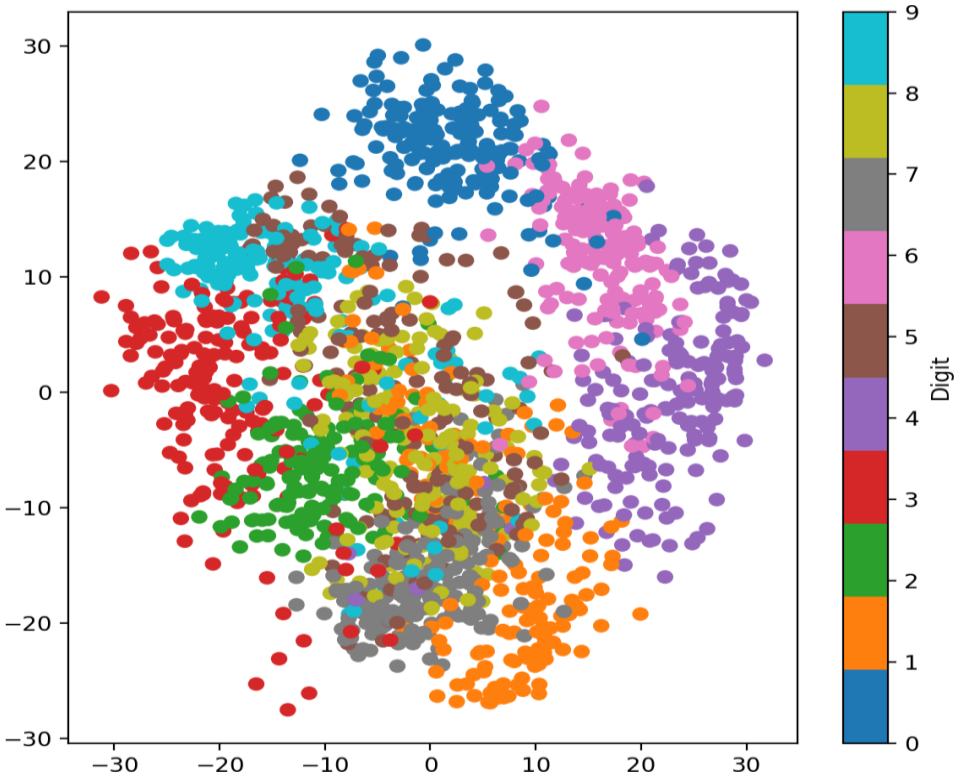
- 📐 **Dimensionality & Transformations** –reduce complexity while preserving structure

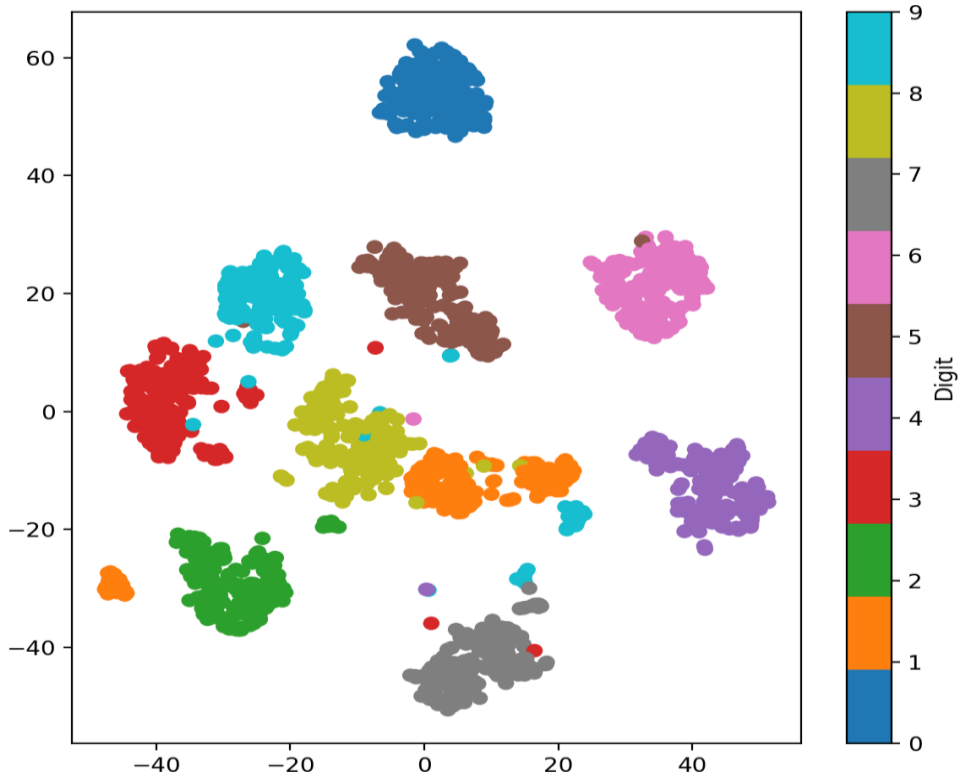
  🛠️ **Hyperparameter tuning** – and how choices affect results
- 🔄 **Optimization** – including **gradient descent**, used in t-SNE
- 📊 **KL Divergence** – for comparing probability distributions

For further details on t-SNE visit: How to Use t-SNE Effectively

Quiz

Which figure is an outcome of tSNE and which figure is an outcome of PCA?

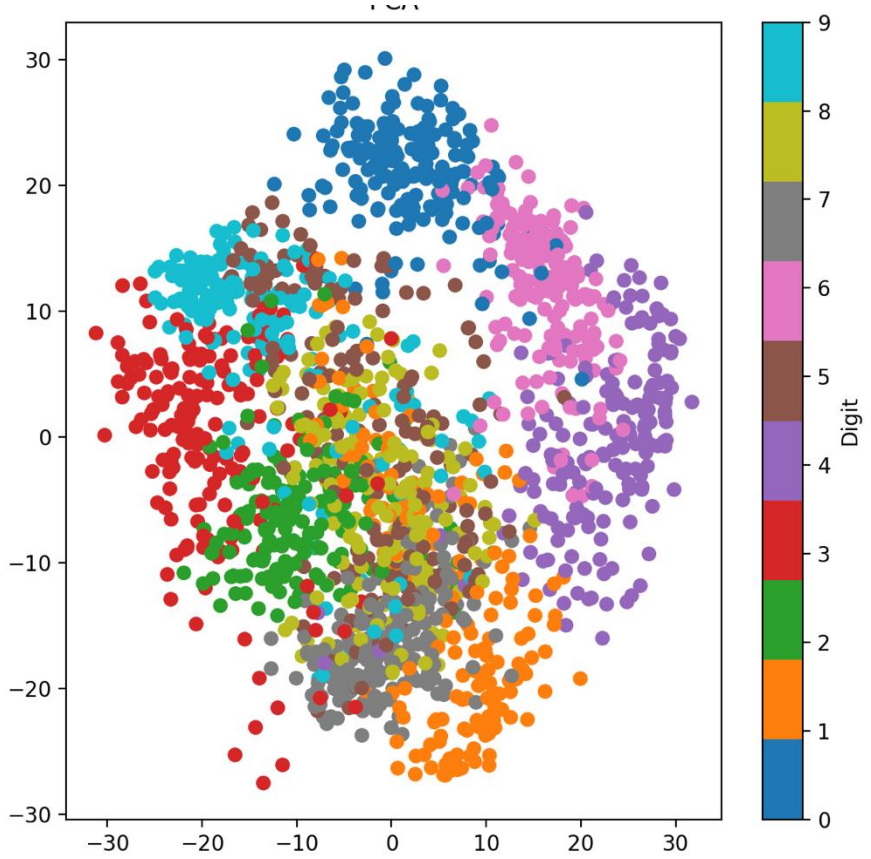




- A PCA tSNE
- B tSNE PCA

Difference tSNE vs. PCA

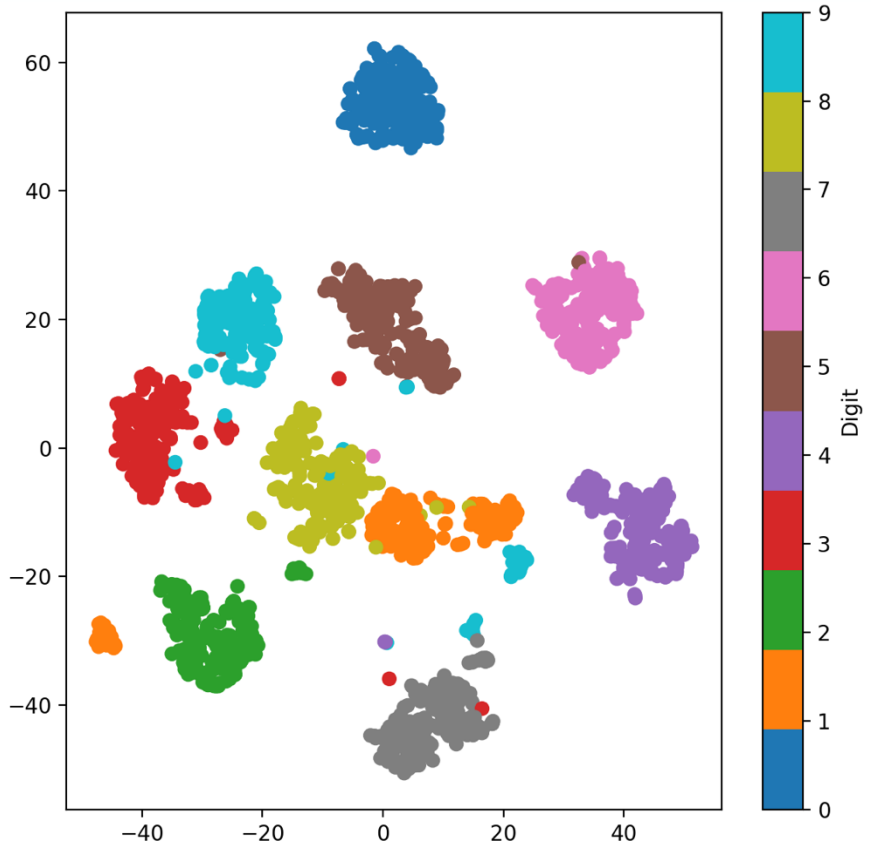
PCA



- Preserves **global variance** Distances between far-apart points are meaningful Good for understanding overall geometry
- Clusters often **overlap** Some separation, but messy Global structure is meaningful (relative distances matter)

Difference tSNE vs. PCA

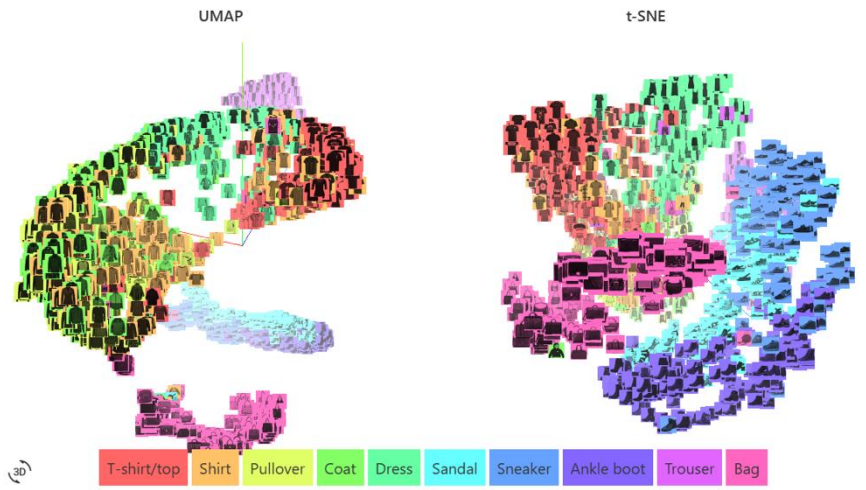
tSNE



- Preserves **local similarity**
- Excellent at grouping similar digits (e.g., all “3”s together)
- Clear **cluster separation by digit (0–9)** Visually intuitive grouping Distances between clusters are *not* globally meaningful
- But cluster spacing can be misleading (clusters may look equally far apart even if they aren’t)

UMAP: Uniform Manifold Approximation and Projection

- 👉 Once you're comfortable with PCA and t-SNE, it’s worth exploring **UMAP** — another powerful tool for visualizing and understanding data.



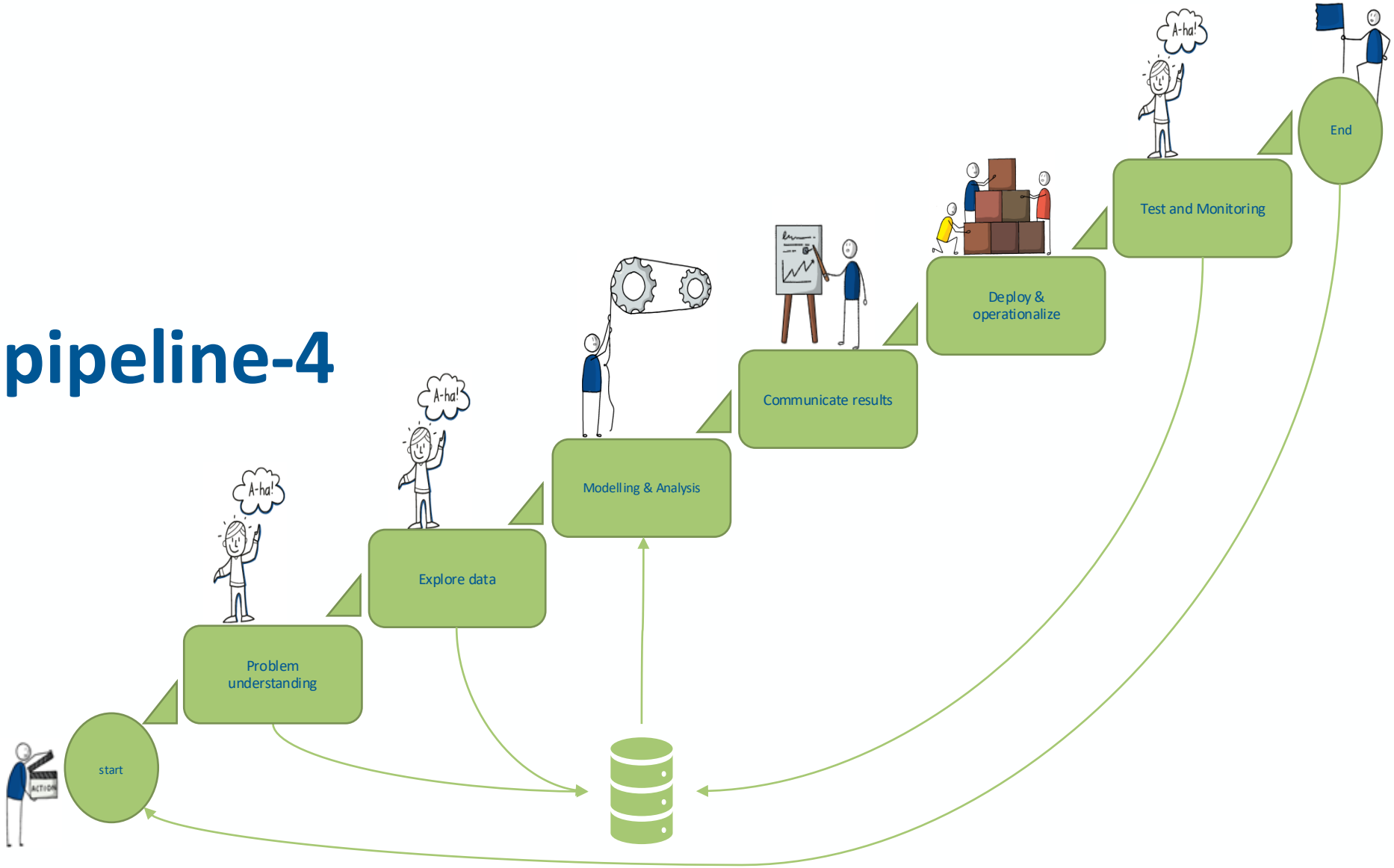
784-dimensional Fashion MNIST dataset

Understanding UMAP: Andy Coenen, Adam Pearce | Google PAIR

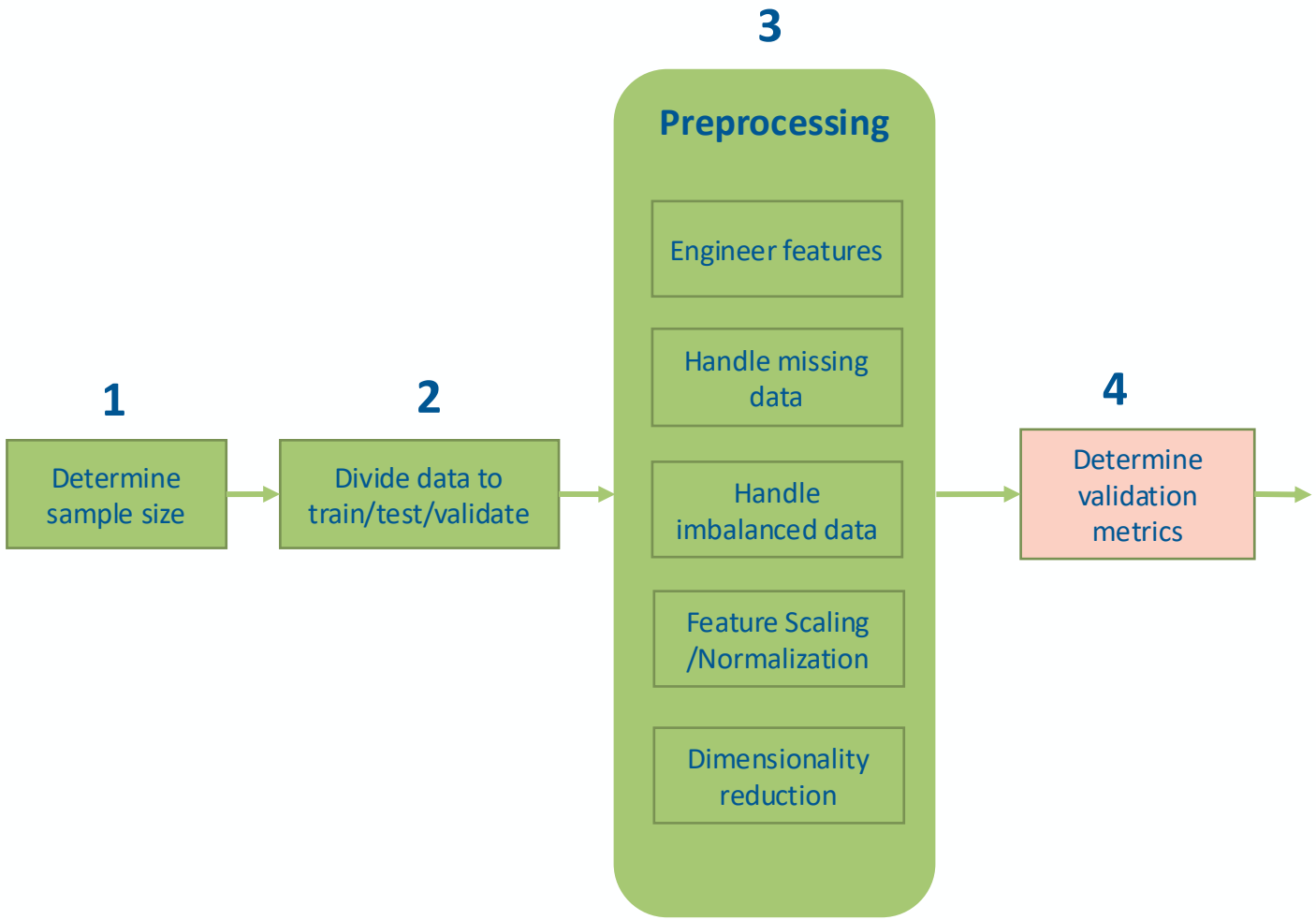
Data Mining pipeline-4

Prof. Dr. Matteo Marouf

11.05. 2026



Determine Validation metrics



- Problem specific and required domain knowledge
- Methodology specific
- It is not always about accuracy

Confusion matrix for two classes classification

E.g. Classification of pathological images to benign vs. malignant tissue

		Predicted	
		Positiv	**Negative**
Truth	**Positive**	True positive (TP)	False negative (FN)
	Negative	False positive (FP)	True negative (TN)

Confusion matrix for two classes classification

E.g. classify pathological images to benign and malignant

Malignant tissue classified malignant

Malignant tissue classified Benign

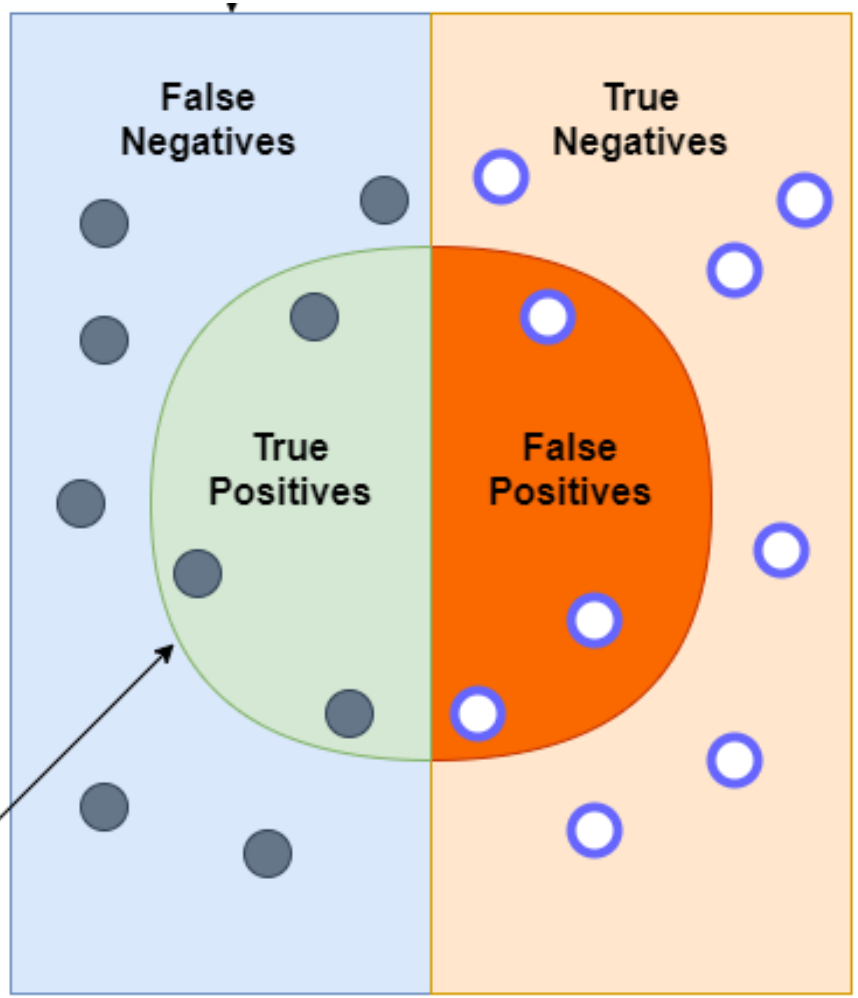
Truth \ Predicted	Positive (Malignant)	Negative (Benign)
Positive (Malignant)	True positive (TP)	False negative (FN)
Negative (Benign)	False positive (FP)	True negative (TN)

Benign tissue classified Malignant

Benign tissue classified Benign

Confusion matrix for two classes classification

E.g. classify pathological images to benign and malignant

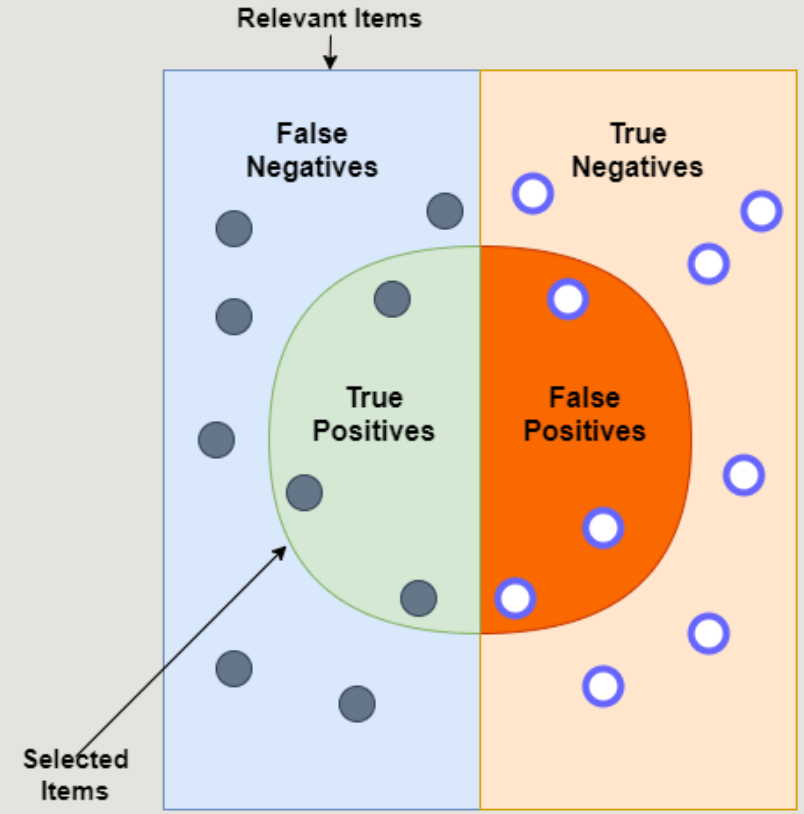


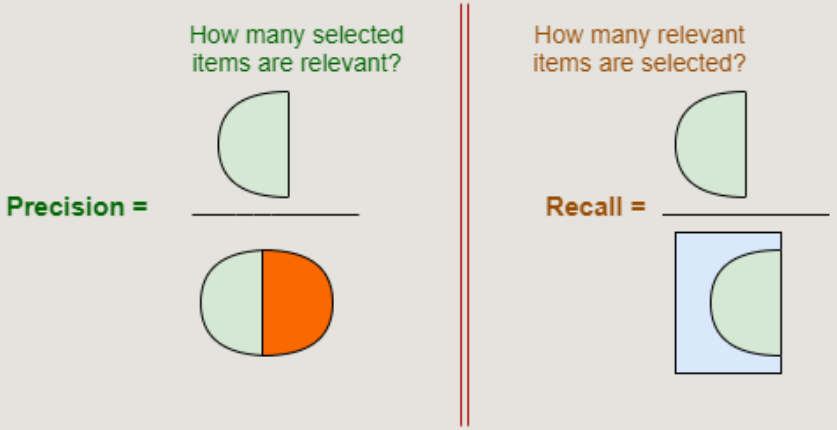
The circle represents all cancer patients

Percision

What fraction of our positive predictions was correct?

$$\frac{TP}{TP + FP}$$

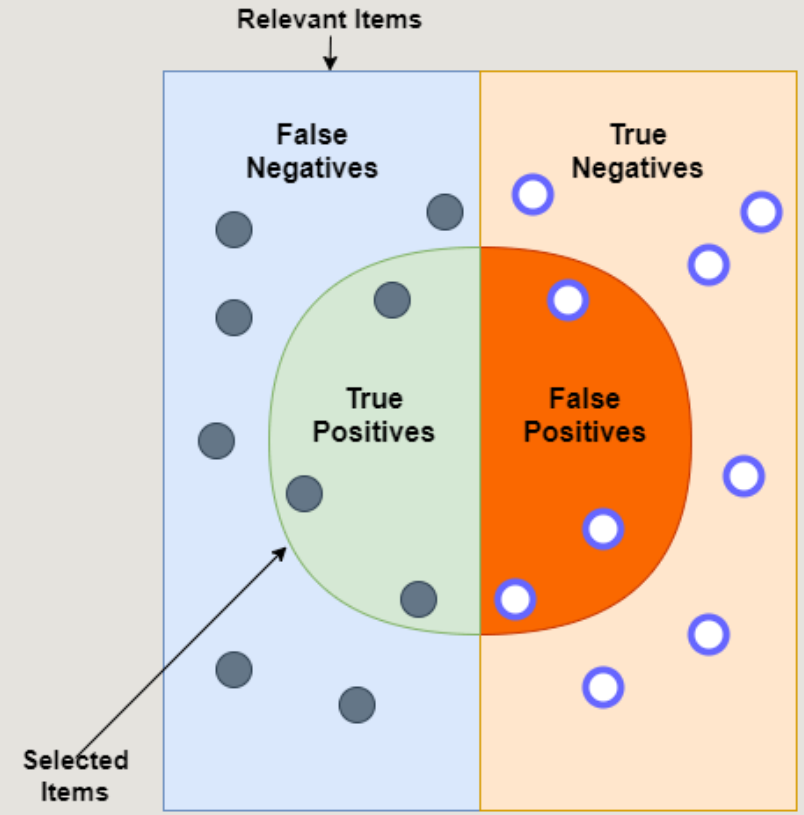


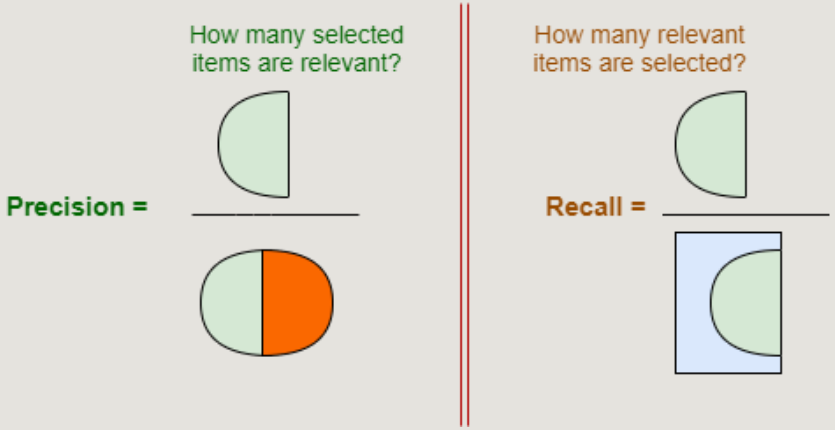


Recall
(also called Sensitivity or True positive Rate)

What fraction of actually positive examples did we find?

$$\frac{TP}{TP + FN}$$

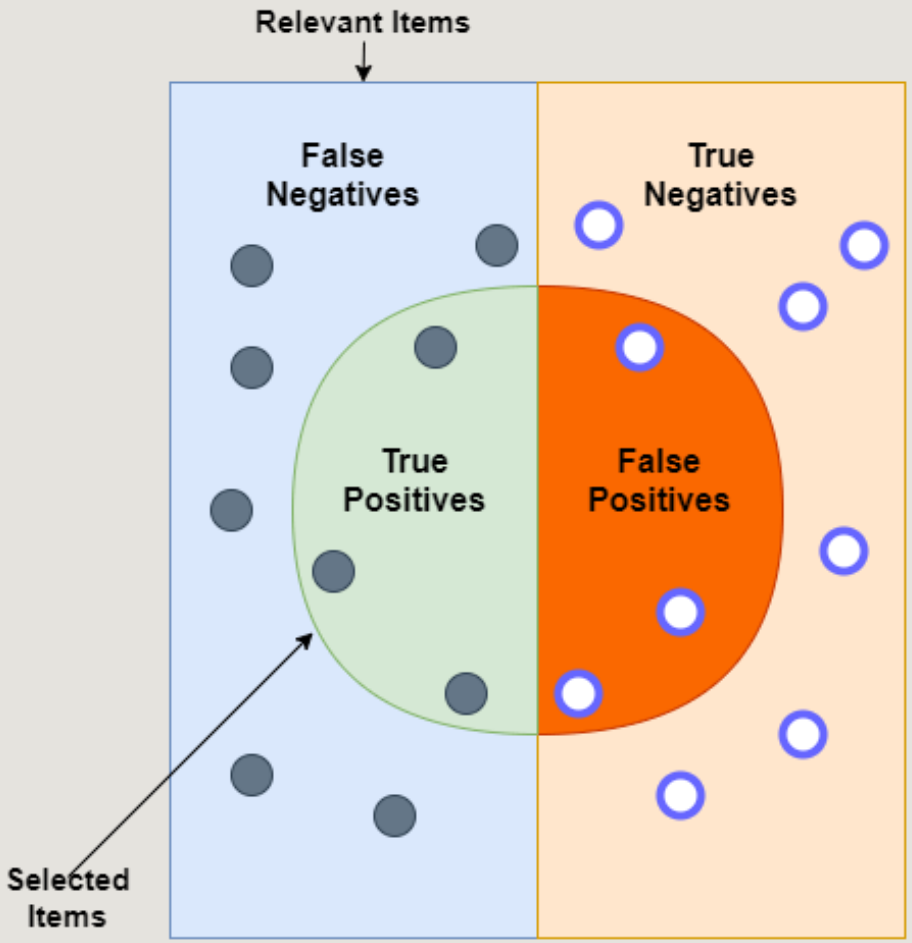




Accuracy

What fraction of our predictions was correct?

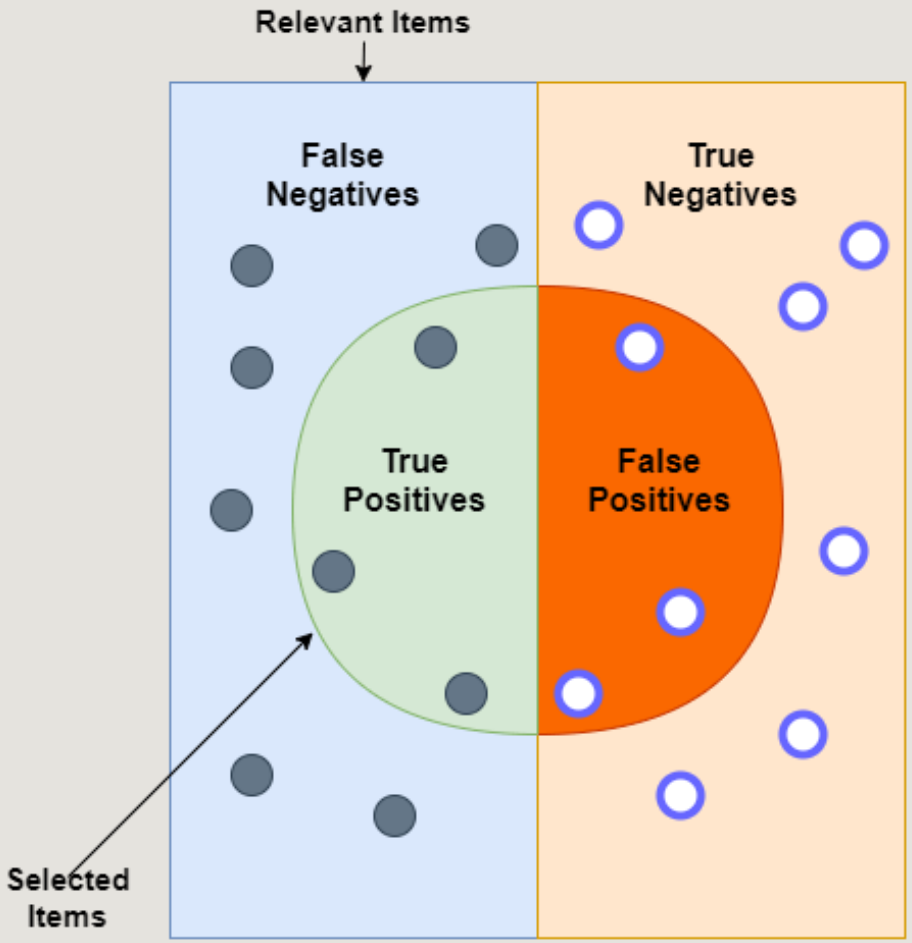
$$\frac{TP + TN}{TP + TN + FP + FN}$$



Specificity
(True Negative Rate)

What fraction of actually negative examples did we find?

$$\frac{TN}{TN + FP}$$



- **A medical test dataset contains:**
  - 5% positive cases
  - 95% negative cases
- A classifier predicts **every patient as negative**.

- Which statement is true?

A. Recall = 100%

B. Specificity = 0%

C. Specificity = 100%

D. Precision = 100%

A spam filter predicts **every email as spam (positive)**.

Which metric is guaranteed to be 100%?

A. Accuracy

B. Recall

C. Specificity

D. Precision

Quiz

- Suppose a rare disease infects **one** out of every 1000 people in a population. Some company published a test for this disease: if a person has the disease, the test comes back positive 99% of the time. On the other hand, the test also produces some false positives: 2% of uninfected people(test positive although healthy).
- If someone just tested positive, what are his chances of having this disease?

Define P: positive test D: having disease H or D′: Healthy

a. 4.72%
b. 2%
c. 99%

Quiz

- Suppose a rare disease infects **one** out of every 1000 people in a population. Some company published a test for this disease: if a person has the disease, the test comes back positive 99% of the time. On the other hand, the test also produces some false positives: 2% of uninfected people(test positive although healthy).
- And someone just tested positive. What are his chances of having this disease?

Define P: positive test D: having disease H or D': Healthy

P(D) = .001
P(H) = 0.999
P(P|D) = .99
P(P|H) = 0.02

$$P(D|P) = \frac{P(P \mid D) \times P(D)}{P(P)}$$

$$= \frac{P(P \mid D) \times P(D)}{P(P \mid D) \times P(D) + P(P \mid H) \times P(H)}$$

$$= \frac{.99 \times .001}{.99 \times .001 + .002 \times .999} = .0472$$

A single metric can be misleading, so what?

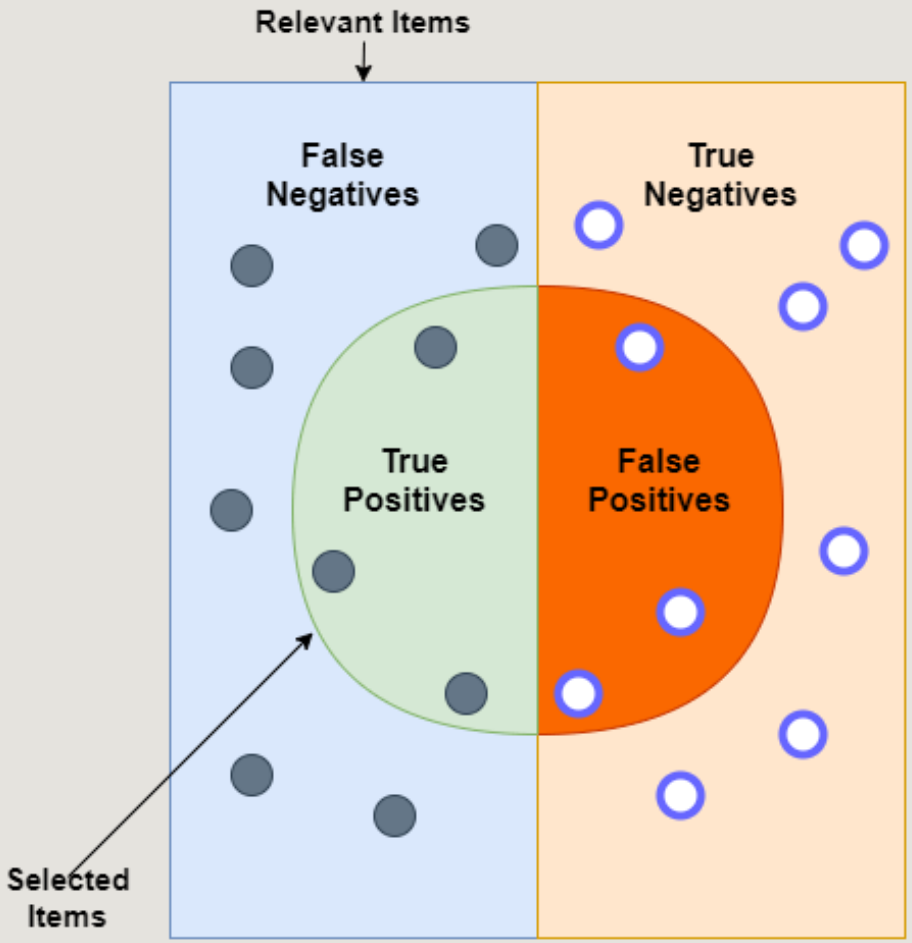
- *If we always predict positive, we get recall of 100%*
- *If we always predict negative, specificity is 100%*
- *If 98% of examples is negative, always predicting "negative" leads to 98% accuracy*

F1 score

Preferred when Predicting positives is important and positive class is rare relative to the negative class.

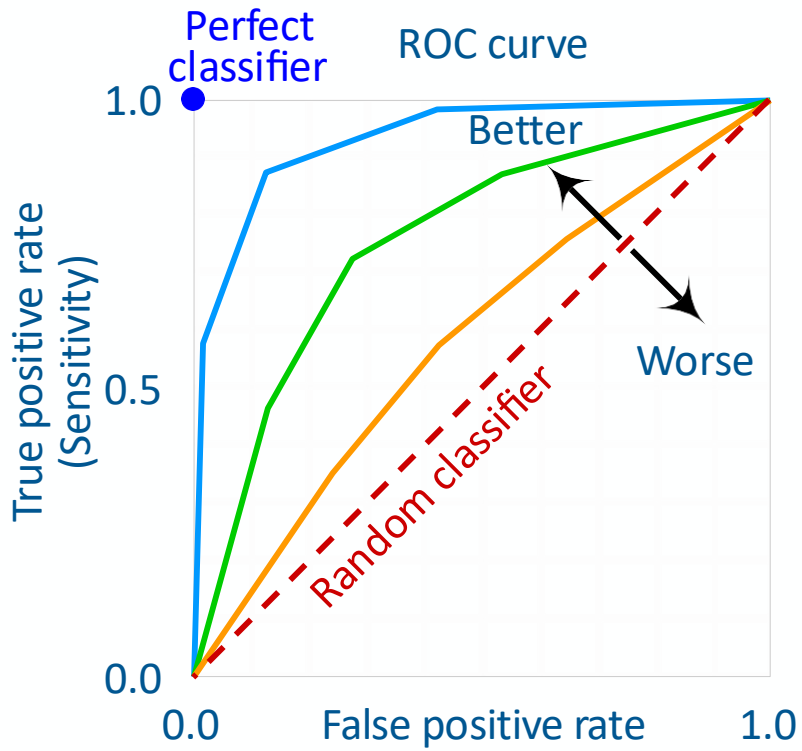
$$F1 = \frac{2.\ Percision.\ Recall}{Percision\ +\ Recall}$$

- **Interpretation of F1 Score Values:**
  - F1 Score = 1: Perfect precision and recall.
  - F1 Score = 0: Worst possible score; the model failed to identify any true positives.
  - 0 < F1 Score < 1: Indicates a balance between precision and recall. The closer to 1, the better the balance.



Area Under Curve

Receiver operating characteristic (ROC) curve is a graphical representation of a binary classifier's performance across all possible classification thresholds.



We plot it for all thresholds for each model and the select the best performing model and the most suitable threshold for our application

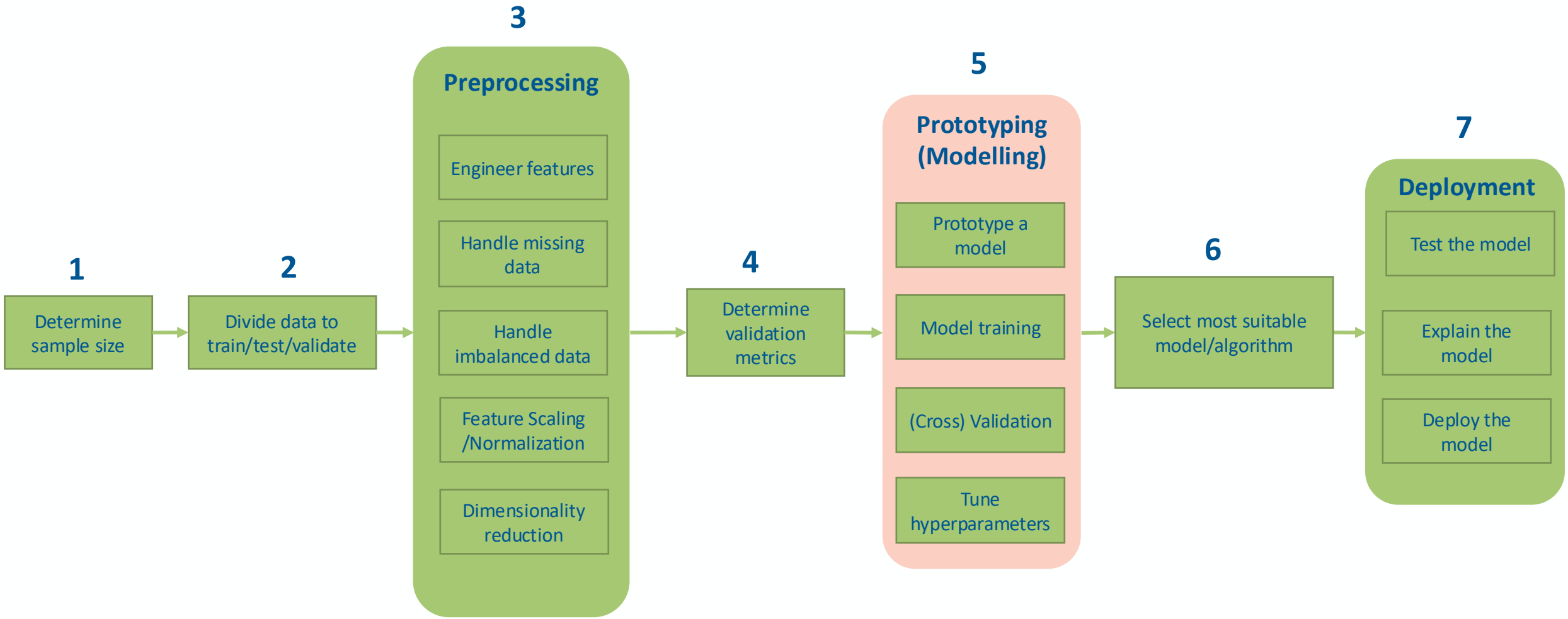
The ROC curve lets you **choose the balance between detecting as many positives as possible and avoiding too many false alarms**.

Validation & evaluation metrics

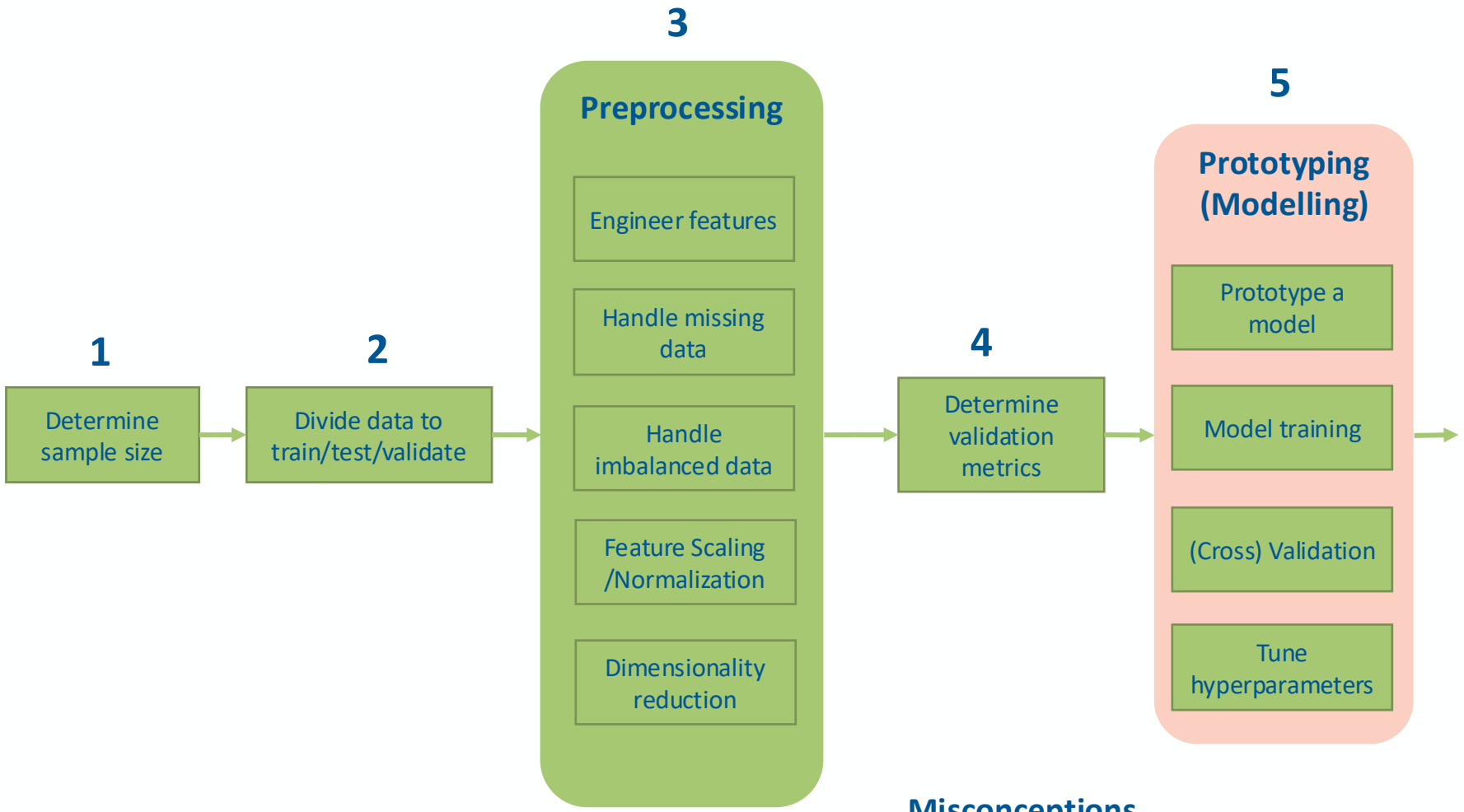
Why is it important to think twice about these metrics?

- Training objective (cost function) is only a proxy for real world objectives.
- Not all errors are equal
- Additional Metrics help capture a business goal into a quantitative target
- Metrics can account for class imbalances or artificially upweight (amplify the importance of) certain important class relative to others

The anatomy of classification using ML



Prototyping



Prototyping is rapid, iterative process in which robust solutions are tested and identified.

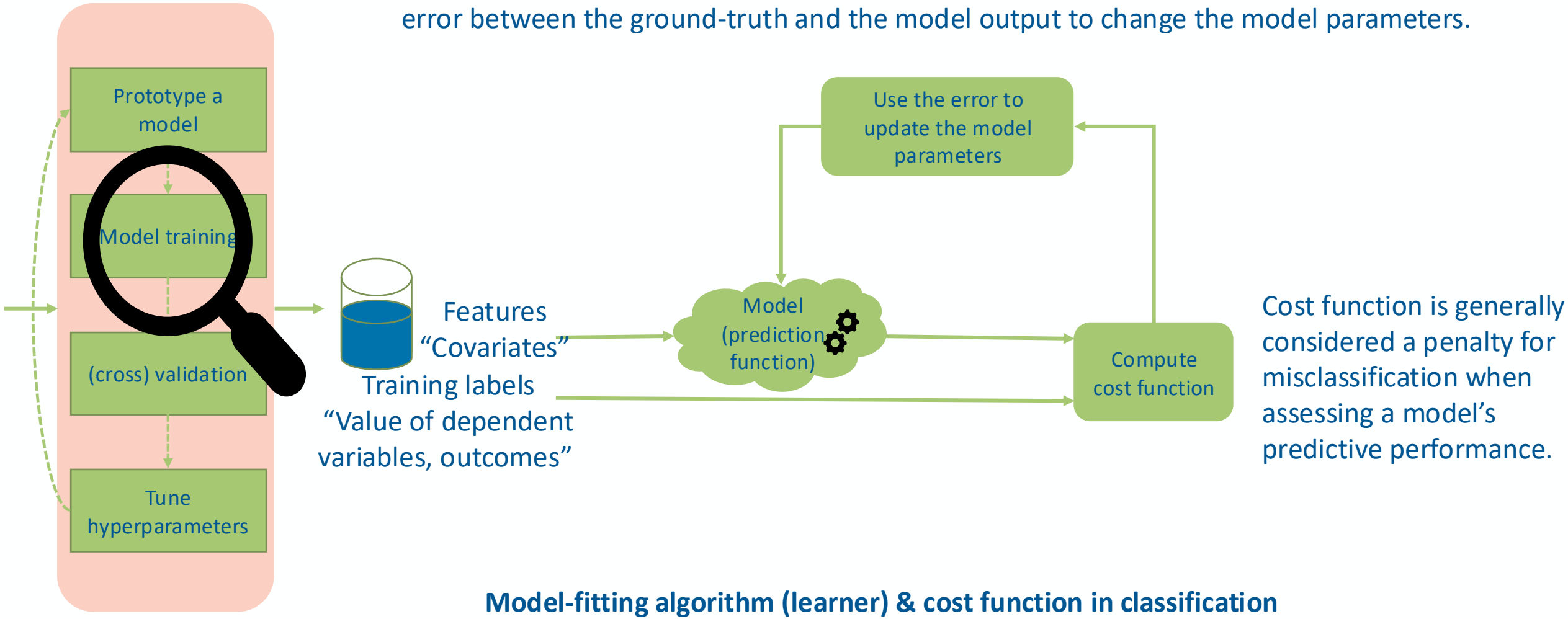
Misconceptions

- We select the best model in terms of accuracy….etc.
- Machine learning models are complicated: we usually start with the simplest model and build on it

Training: objective function (also called cost in supervised learning)

Prototyping (modelling)

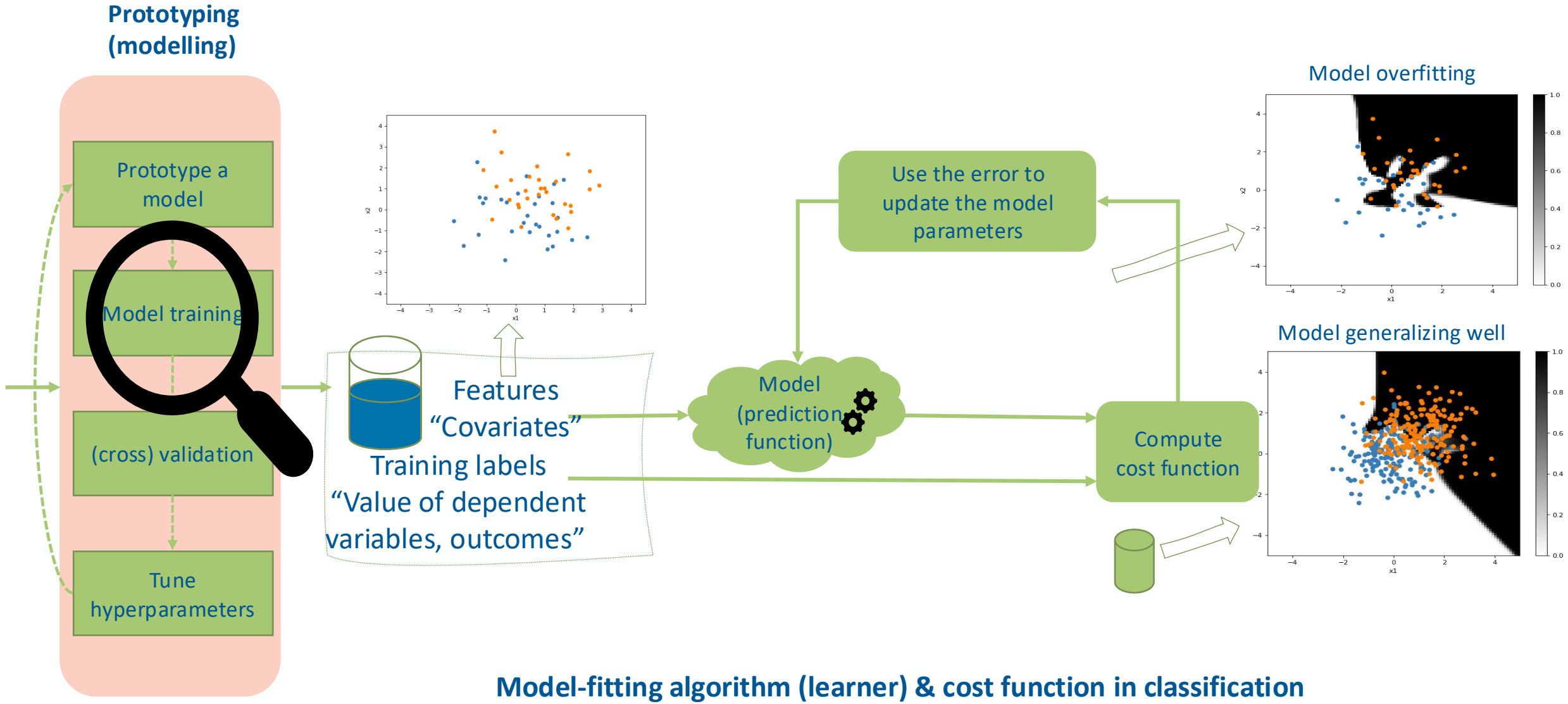
The learning algorithm iteratively calculates the loss function of the training data and uses the error between the ground-truth and the model output to change the model parameters.



Cost function is generally considered a penalty for misclassification when assessing a model's predictive performance.

Model-fitting algorithm (learner) & cost function in classification

Training: overfitting (high variance and little bias)



Model-fitting algorithm (learner) & cost function in classification

Training: underfitting (small variance and high bias)

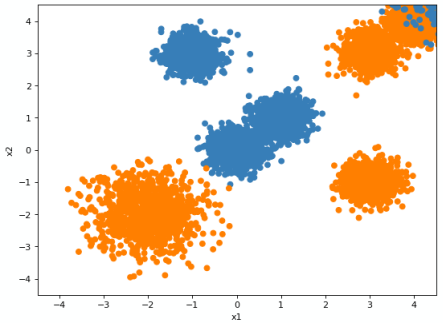
Prototyping (modelling)

Prototype a model

Model training

(cross) validation

Tune hyperparameters



Features "Covariates"

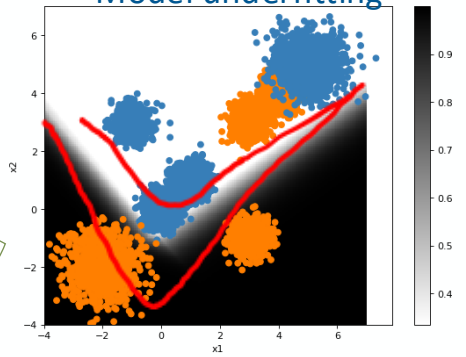
Training labels "Value of dependent variables, outcomes"

Model (prediction function)

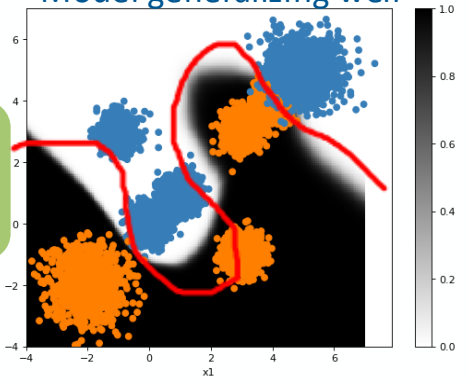
Use the error to update the model parameters

Compute cost function

Model underfitting

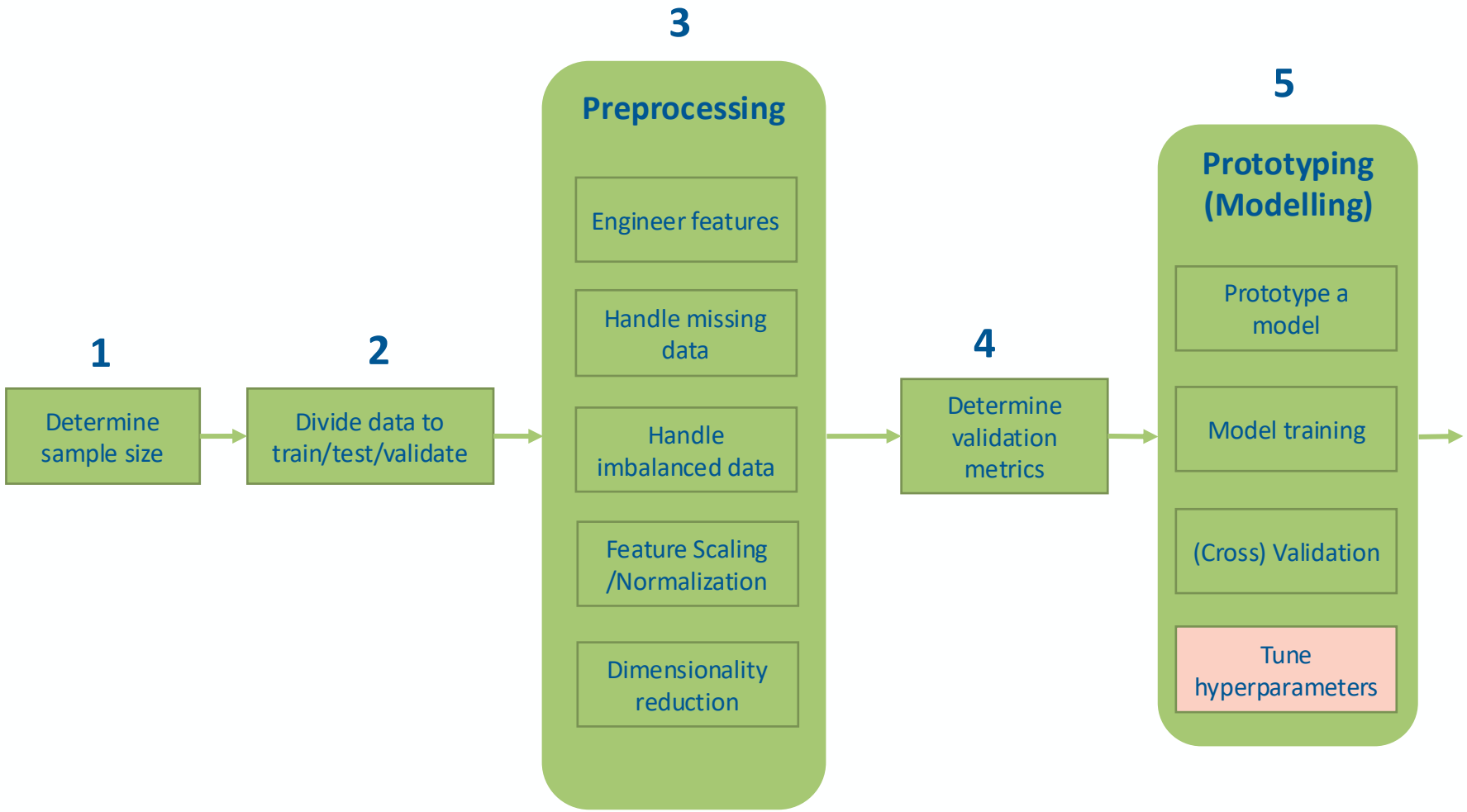


Model generalizing well



Model-fitting algorithm (learner) & cost function in classification

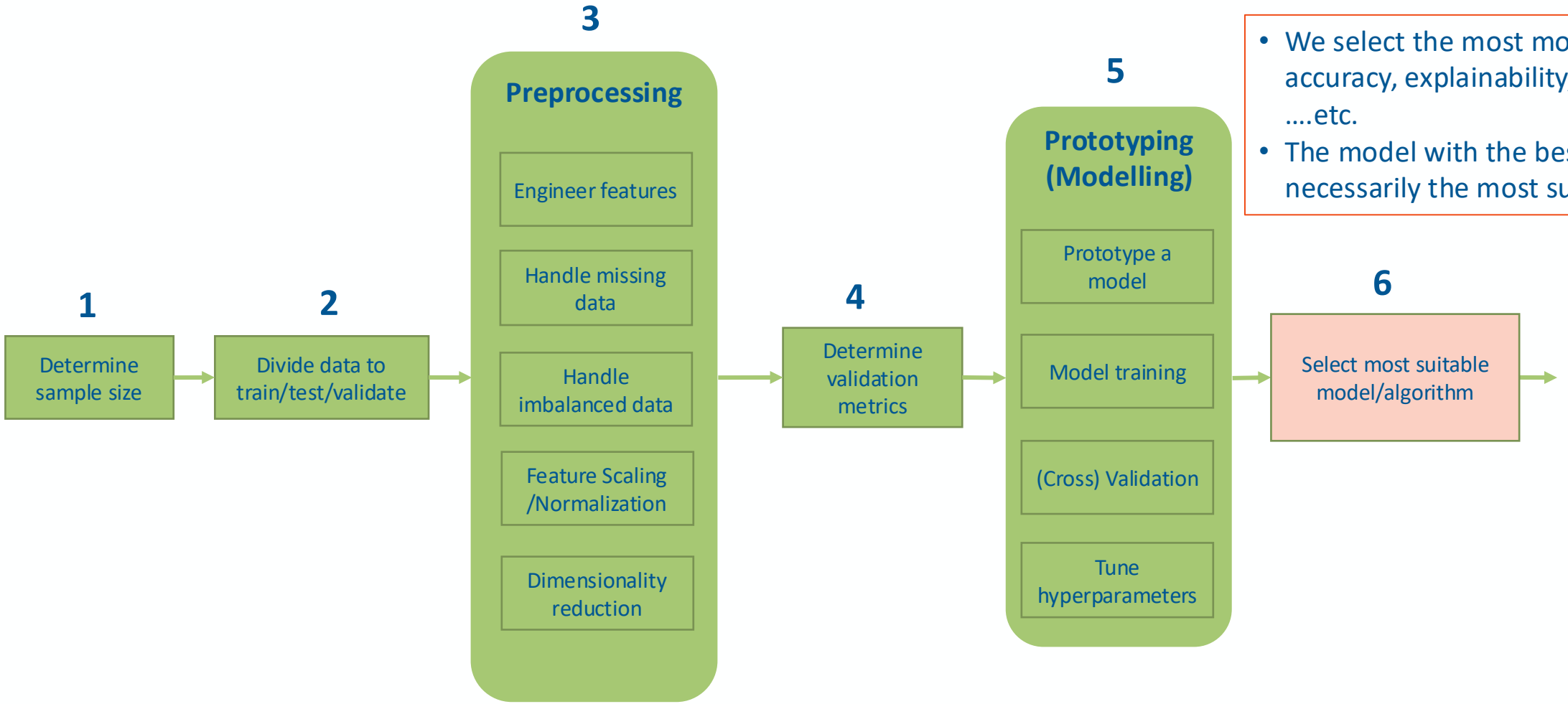
Hyperparametrs optimization



Hyperparameters vs Model Parameters

Feature	Model Parameters	Hyperparameters
Set by	Learned during training	Manually set before training
Purpose	Define the model's behavior	Control the learning process
Examples	Weights, biases	Learning rate, epochs, tree depth
Tuned using	Optimization algorithms	Search or validation methods

Select the most suitable model



- We select the most model in terms of accuracy, explainability, fit for purpose, ….etc.
- The model with the best score is not necessarily the most suitable model

IMPORTANT

What to consider to choose the Most Suitable Model

Generalizability

- Avoids overfitting; performs well on unseen data.

Simplicity & Maintainability:

- Easier to debug, update, and deploy.

Speed & Efficiency:

- Meets latency and resource constraints.

Interpretability

- Critical in domains like healthcare, finance, law.

Stability & Reproducibility

- Performs consistently across data variations.

Summary

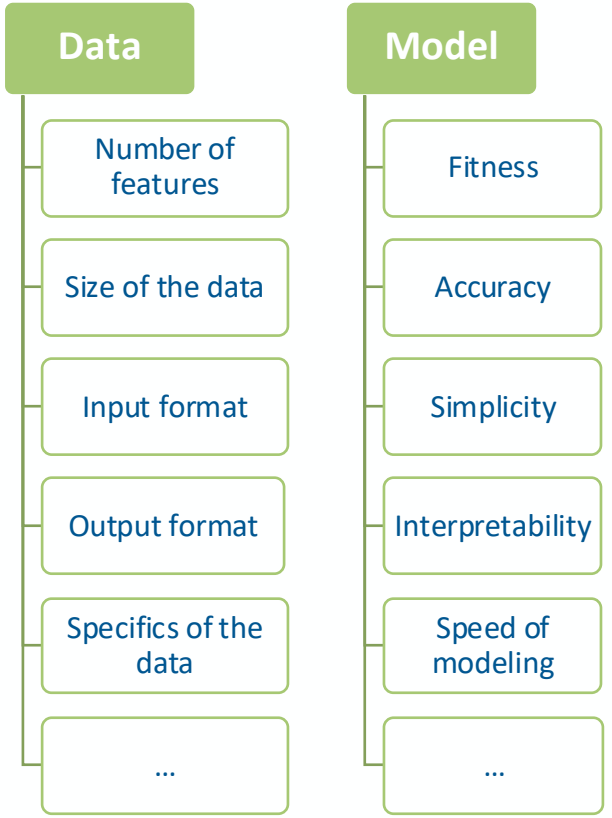
The most suitable model balances performance with interpretability, scalability, and operational constraints.

Statistical modelling, Machine Learning, or deep learning, what to choose?

How to choose the right model?

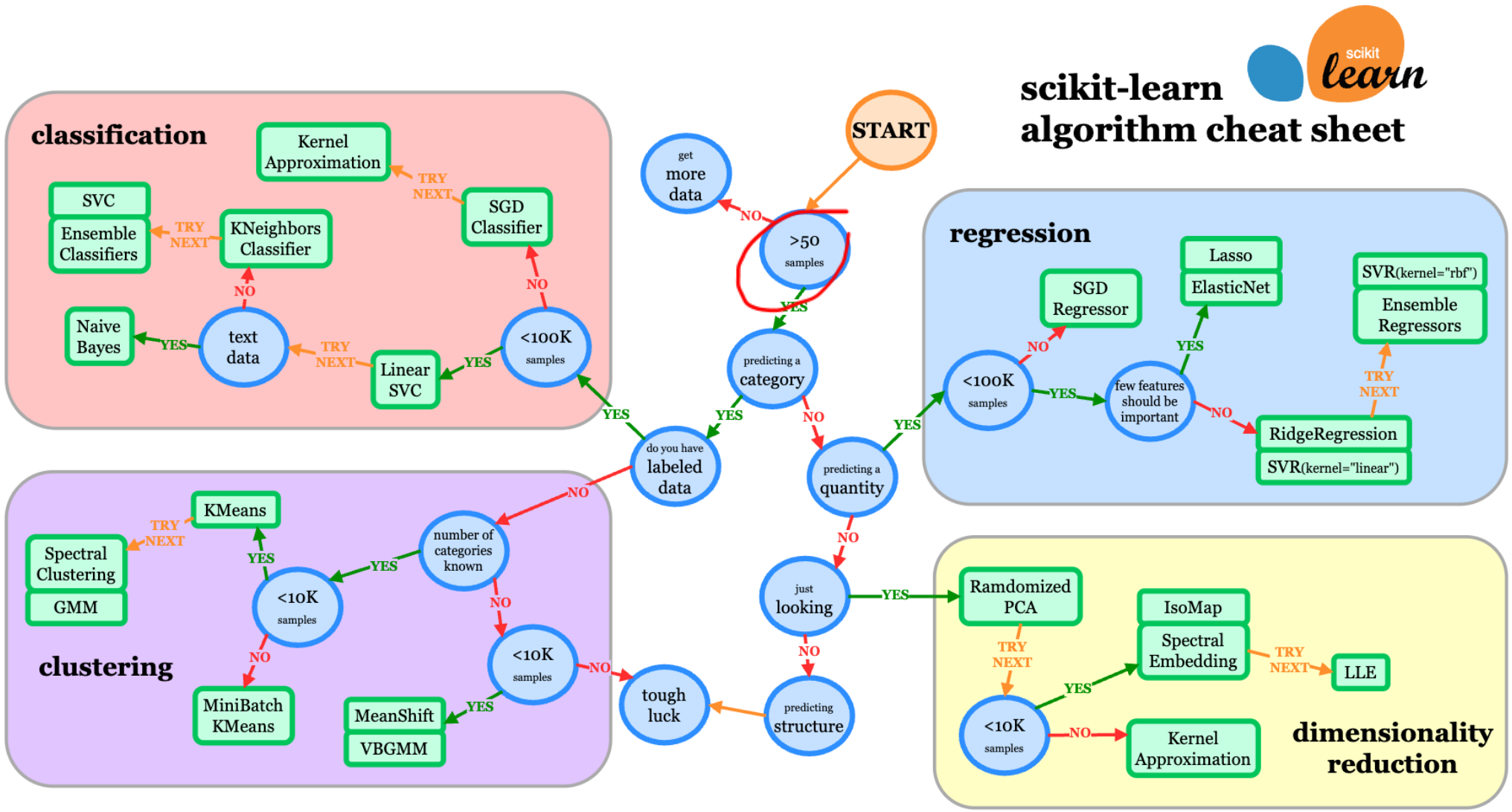
Machine Learning, statistical modelling, or deep learning, what to choose?

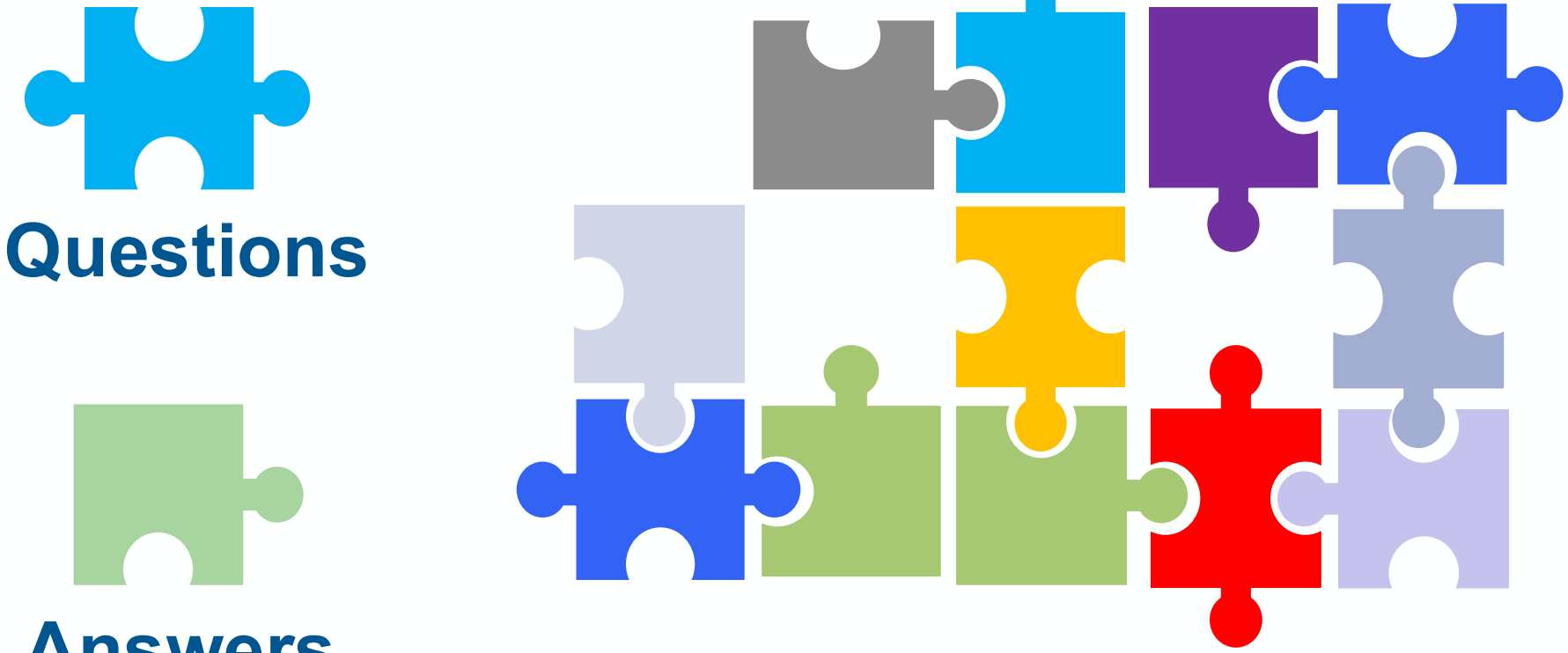
For a successful Data Science project, we need to choose the right ingredients and flavours ideally in advance, but a lot of adjustments will happen while cooking.



Remember: Data Mining is also about visualization and other types of data analytics and not only about Machine Learning

How to choose the right estimator?
Example: scikit-learn algorithm cheat-sheet




Questions
Answers